

面向 CEC 场景的 LLM 推理优化：算法 研究与应用综述

Cloud-Edge-Client Collaborative LLM Inference Survey

研究综述报告

云-边-端协同计算与大模型推理优化

版本： v1.0

日期： 2026 年 5 月

摘要

大语言模型推理正在从云端集中式服务走向端、边、云协同参与的分布式服务形态。随着端侧设备、接入边缘、区域边缘和中心云共同进入推理路径，系统瓶颈不再只是单个模型或单个 GPU 实例的执行效率，而是模型部署、请求调度、KV/上下文状态管理、服务路径选择和业务 QoS 之间的联合权衡。CEC 场景下的 LLM inference 因而不能简单理解为云侧 serving 能力的边缘下沉，而需要在异构资源、动态网络、用户移动、隐私边界和服务连续性约束下重新组织推理系统。

本文围绕 CEC 场景下的大语言模型推理优化展开综述。首先，本文梳理 LLM inference 的基本执行机制、问题空间和以优化对象为主轴的 Layer-Scope 分类框架，并分析 CEC 相较云侧集中式 serving 带来的优化重心迁移。在此基础上，本文重点讨论三类算法问题：其一是资源自适应的模型切分部署方法，包括 layer-wise、tensor/operator-level、expert/component-level、inference phase 以及 semantic/token-level partitioning；其二是业务感知的 batching 参数优化方法，包括 queue-level dynamic/admission batching、continuous/iteration-level batching、chunked prefill/blended batching 和 disaggregated prefill-decode batching；其三是用户移动场景下的 LLM inference 请求调度与状态管理，包括 request/session routing、context/KV/runtime state migration and offloading，并把 multi-model/multi-agent workflow orchestration 作为相关扩展问题讨论。进一步地，本文总结三类算法之间的协同关系，讨论其在华为 Ascend 推理栈、vLLM Ascend、MindIE 和 CANN 等现有基础设施上的工程落点，并给出面向验证环境的实施路线、实验指标、baseline 设计和未来研究方向。本文旨在为云-边-端协同大模型推理系统的算法选型、实验验证和工程落地提供系统化参考。

目录

一、引言	5
1.1 研究背景与应用动机	5
1.2 CEC 场景下边缘 AI 推理的关键挑战	5
1.3 国内外研究现状概述	6
1.4 本文研究范围与主要贡献	7
二、LLM 推理基础、问题空间与优化分类	9
2.1 LLM 推理的基本执行机制	9
2.2 LLM 推理的问题空间：从执行机制到系统瓶颈	12
2.3 LLM Inference Optimization Taxonomy：以优化对象为主轴的分类	14
2.4 工业级 LLM Inference Framework 代表性机制映射	19
2.5 小结	20
三、CEC 场景下的大模型推理	21
3.1 CEC LLM 推理的系统形态	21
3.2 CEC 相较云侧集中式 LLM Serving 新增的问题空间	22
3.3 CEC 场景下 LLM Inference 优化重心的转移	22
3.4 本文采用的 CEC LLM Inference 综述框架	23
3.5 小结	24
四、CEC 下的资源自适应的模型切分部署方法	26
4.1 Layer-wise / Vertical Partitioning	29
4.2 Tensor / Operator-level Partitioning	31
4.3 Expert / Component-level Partitioning	33
4.4 Inference Phase Partitioning	35
4.5 Semantic / Token-level Partitioning	36
4.6 面向华为验证环境的方案建议	38
4.7 小结	40
五、CEC 下的业务感知 Batching 参数优化方法	42
5.1 Queue-level Dynamic / Admission Batching	45

5.2 Continuous Batching / Iteration-level Batching	47
5.3 Chunked Prefill / Blended Batching	48
5.4 Disaggregated Prefill-Decode Batching	50
5.5 面向华为验证环境的方案建议	51
5.6 小结	54
六、CEC 中的 LLM Inference Task Offloading	56
6.1 Request / Session-level Routing and Offloading	57
6.2 Multi-model / Multi-agent Workflow Orchestration	59
6.3 Context / KV / Runtime State Offloading and Migration	60
6.4 面向用户移动性的长期会话感知 LLM 请求卸载方案	62
6.5 小结	70
七、三类算法的协同优化框架	72
7.1 三类算法的基本耦合关系	74
7.2 Model / Service Placement – Request Offloading 协同	75
7.3 Model Partitioning – Batching / Scheduling 协同	75
7.4 Request / Workflow Routing – Runtime State Management 协同	76
7.5 Mobility-aware Hierarchical Control	77
7.6 算法方案对比与面向华为验证环境的建议	78
7.7 小结	79
八、面向 Ascend/CANN 验证环境的 CEC 推理工程实现方案	80
8.1 工程定位	80
8.2 总体架构	80
8.3 分阶段实现路线	81
8.4 关键工程模块	83
8.5 验证路线	84
8.6 小结	84
九、实验评估与对比指标	86
9.1 实验场景	86
9.2 Baseline 设置	87
9.3 指标体系	88

9.4 消融实验	88
9.5 预期收益分析	89
十、挑战与未来方向	91
10.1 异构资源与性能建模	91
10.2 状态迁移与一致性	91
10.3 在线策略稳定性	92
10.4 隐私与合规	92
10.5 未来研究方向	92
十一、总结	94
参考文献	95

一、引言

1.1 研究背景与应用动机

大语言模型推理正在从云端集中式服务，走向端侧、边缘和云端共同参与的协同服务形态。过去几年中，LLM inference 优化大体形成了两条成熟路线：一条是 on-device inference，通过压缩、量化、小模型和端侧 NPU/GPU 加速，使模型尽可能靠近用户运行；另一条是 cloud / datacenter LLM serving，通过高性能 kernel、continuous batching、KV Cache 管理、模型并行和多实例调度，在云侧 GPU 集群中提高吞吐、降低尾延迟并摊薄服务成本。已有 LLM serving 系统与综述工作分别从 runtime、memory management、batching/scheduling 和 distributed serving 等角度总结了这些机制 [1], [2], [3], [4], [5], [6], [7], [8]。

这两条路线分别解决了局部问题，却都难以单独覆盖真实业务中的全部约束。端侧推理具有低时延和数据本地性优势，但受算力、内存、功耗和散热限制，很难长期承载复杂大模型和高并发服务。云侧推理资源充足、框架成熟，却必须经过网络访问，端到端时延、带宽消耗、隐私风险和服务路径依赖都会进入业务关键路径。与此同时，接入边缘、区域边缘和近用户算力节点中存在大量分散、异构且动态变化的资源，它们既无法简单等同于端侧设备，也不同于数据中心中的同构 GPU 集群。

CEC (Collaborative Edge Computing, 协作边缘计算) 正是在这种背景下成为 LLM inference 的重要系统形态。如图 1 所示，CEC 将泛在、地理分布的端侧与边缘设备组织为可协作的资源池，并通过 Wi-Fi 接入点、路侧单元、5G 基站和网关等基础设施连接中心云与多类终端，共同完成计算任务。CEC 并不是简单地把模型从云端搬到边缘，而是把端侧、边缘和中心云视为一个多层协同资源池，使推理请求能够根据模型规模、请求复杂度、网络状态、节点负载、隐私要求和 QoS 目标动态选择模型形态、执行位置和服务路径。端-边-云协同智能研究已经指出，多层系统的关键在于资源、链路和服务位置之间的协同，而不是单点计算能力的替换 [9]。因此，CEC LLM inference 的核心问题不再只是“如何让模型跑得更快”，而是如何在异构、分布式、动态变化的端-边-云环境中，以可控成本满足业务时延、吞吐、服务连续性和数据隐私要求。

1.2 CEC 场景下边缘 AI 推理的关键挑战

CEC 场景下的边缘 AI 推理首先受到资源异构和碎片化的约束。端侧设备、接入边缘、区域边缘和中心云在 CPU、GPU、NPU、内存、显存、功耗预算和加速库支持上差异显著，模型部署不能沿用中心云中“节点规格相近、互联稳定、控制面统一”的假设。同一个模型在不同硬件上可能面临完全不同的显存瓶颈、算子支持和通信代价，部署策略也必须随资源状态动态调整。

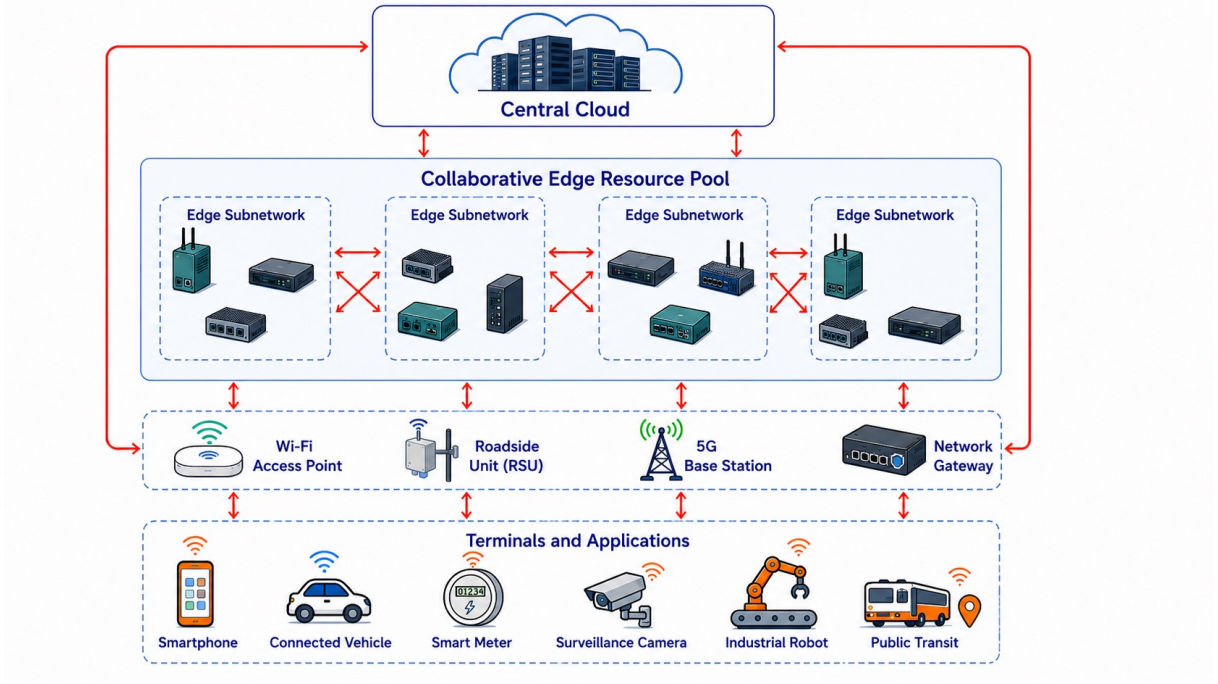


图1 协作边缘计算示意图。协作边缘计算汇聚泛在、地理分布设备的计算资源以共同执行计算任务，从而扩展可用资源池，实现低时延数据处理、灵活设备接入与更广的服务覆盖。

LLM 推理本身还具有显著的状态和内存压力。Prefill 阶段会为后续 decode 生成 KV Cache，decode 阶段又会随着输出长度不断追加状态；在长上下文、多轮会话和高并发场景中，KV Cache、prefix cache 和 session state 往往成为比单次计算更难管理的运行时资源。一旦这些状态跨节点访问、迁移或重算，通信成本、服务连续性和隐私边界都会同时受到影响。

请求动态性进一步放大了调度难度。用户并发数、prompt 长度、输出长度、请求复杂度和 SLA 时延要求都在持续变化，边缘节点又常常缺乏云侧大规模请求池，静态 batching、固定 batch size 和固定服务路径难以稳定满足业务目标。对于移动用户，接入点和最优服务实例还会随位置变化而变化，最近节点不一定是负载最合适或端到端时延最低的节点；若用户会话中的 KV 或上下文状态仍保留在旧边缘节点，服务切换还可能引发状态迁移、远程访问或重新 prefill。

因此，CEC LLM inference 的挑战并不是单一技术点，而是模型部署、batching 调度、task offloading、状态管理和路由之间的耦合。模型切分、prefill/decode 分离、KV 迁移、跨节点执行和多入口路由都可能带来新的 activation、hidden state、KV Cache 或 session state 传输成本。系统需要在计算收益和通信代价之间持续权衡，并在 QoS、隐私和服务连续性之间给出可解释的在线决策。

1.3 国内外研究现状概述

现有研究为 CEC LLM inference 提供了多方面基础。一方面，on-device inference 关注模型压缩、低比特执行、小模型设计和端侧硬件加速，使部分请求能够在用户设备本地完成；FlexGen [10]、Petals [11]、HexGen [12] 等工作则进一步展示了资源受限或异构环境下的大模型推理可以通过分层存储、协作

式 block 放置和异构并行来突破单节点容量限制。另一方面, cloud/datacenter LLM serving 已经围绕 continuous batching、PagedAttention、chunked prefill、prefill/decode disaggregation、KV cache management 和多实例调度形成了较成熟的系统路线, Orca [1]、vLLM [2]、Sarathi-Serve [3]、DistServe [4]、SGLang [5] 和 Mooncake [13] 等系统分别代表了不同层面的关键机制。

边缘计算领域长期研究 task offloading、资源分配、移动性管理和网络感知调度,但传统 MEC/CEC 任务通常被抽象为给定输入大小、计算量、deadline 和能耗约束的 job。LLM inference 改变了这一抽象:推理时间取决于 prompt、输出长度、decode step、batch 状态和 KV 状态,复杂请求还可能涉及多模型、多 agent 或多阶段执行。因此,CEC LLM inference 位于 LLM serving systems 与 CEC task offloading 的交叉处,既要继承 LLM serving 中的 prefill/decode、KV Cache、batching、speculative decoding 和分布式执行问题,又要处理端-边-云资源异构、动态网络、用户移动、隐私约束和服务连续性。

基于这一交叉特征,本文将相关工作组织为三条主线。第一条是 **CEC 下的资源自适应模型部署方法**,关注模型结构、模型组件、执行阶段和生成职责如何在端、边、云之间切分、放置和重配置;这一方向直接对应后文的 layer-wise / tensor-level / expert-level / phase-level / semantic-token-level 等切分范式。第二条是 **CEC 中的业务感知 batching 参数优化算法**,关注请求进入推理实例后如何根据 SLO、队列状态、KV 占用和 prefill/decode 干扰选择 batch 形成与更新策略。第三条是 **CEC 中的 LLM inference task offloading**,关注完整请求、会话、multi-agent workflow 以及 context/KV/runtime state 如何在端、边、云之间路由、卸载、迁移和复用。这三条主线分别从空间部署、时间调度和服务路径三个角度刻画 CEC LLM inference 的核心优化问题。

1.4 本文研究范围与主要贡献

本文聚焦 **CEC 场景下的大语言模型推理优化问题**。研究范围限定在推理阶段,不讨论大模型训练、预训练或微调过程;重点关注端侧、接入边缘、区域边缘和中心云之间的大模型推理协同,包括模型动态切分、自适应 batching、prefill/decode 调度、分层 KV/状态管理、任务 offloading、策略路由、移动性感知请求调度和服务连续性保障。

对于底层 kernel 优化、硬件实现和模型结构设计,本文仅在其影响 CEC 推理系统设计时进行讨论。例如,低比特 kernel 会影响模型能否部署在端侧或边缘设备上;KV Cache layout 会影响状态迁移和缓存复用;partition-friendly model design 会影响模型是否适合跨端-边-云切分执行。本文的重点不是单独评估某个 kernel 或模型结构,而是从 CEC 系统角度分析这些技术如何影响大模型推理的部署、调度、状态管理和策略。

本文首先从 prefill/decode、KV Cache、batching/scheduling、decoding runtime 和主要服务指标出发,梳理 LLM inference 的基本执行机制,并进一步解释模型规模、KV 状态、请求动态性、decode 串行性和多实例协同为何会成为推理系统的核心瓶颈。在此基础上,本文构建面向优化对象的 Layer-

Scope taxonomy, 将 LLM inference optimization 划分为 Model-Level、Kernel-Level、Runtime-Level 和 Serving Policy & Routing-Level, 并以 Local / Distributed 作为执行范围副轴, 用于分析不同方法的 primary optimization object、execution scope 和 secondary effects。

随后, 本文分析 CEC 环境如何改变 LLM inference 的优化重心。相较云侧集中式 serving, CEC 场景需要额外处理端-边-云资源异构、边缘低并发、动态网络、用户移动、状态迁移、隐私边界和 QoS 约束, 因此各优化层都需要从单点效率转向跨节点、跨状态、跨服务路径的协同决策。围绕这一变化, 本文提出三类面向 CEC 的综述主线: 资源自适应模型部署方法、业务感知 batching 参数优化算法和 LLM inference task offloading, 并分别总结其核心问题、代表工作、决策变量、优化目标和面向验证环境的算法建议。最后, 本文从时延、吞吐、QoS、资源利用率、通信开销、状态迁移成本、隐私约束、服务连续性和系统可部署性等角度归纳评估维度, 并讨论跨层协同优化、状态感知 routing、移动性预测、异构 runtime、分层模型服务和真实系统落地等未来挑战。

二、LLM 推理基础、问题空间与优化分类

2.1 LLM 推理的基本执行机制

LLM 推理可以理解为一个在线生成服务过程：系统接收用户输入上下文，并以自回归方式逐 token 生成输出。与传统 DNN 推理相比，LLM 推理的特殊性在于输入长度和输出长度动态变化，decode 阶段存在强状态依赖，KV Cache 会随上下文长度和生成长度持续增长，系统需要在吞吐、时延、显存占用和服务质量之间持续权衡。如图 2 所示，serving policy 是统筹请求准入、路由、排序、排队和执行组织的控制面；其中的 batching/scheduling 模块根据 SLO、优先级、请求长度、实例负载和 KV 容量等状态调整 batching window、token budget、prefill/decode quota 和 active set，形成待执行的活动批次。活动批次中既可以包含尚待处理输入上下文的 prefill tasks，也可以包含正在逐 token 迭代的 decode tasks，调度器分别将两类工作下发到对应执行路径。底层 inference runtime 随后执行 Prefill 和 Decode：Prefill 并行处理输入上下文并生成初始 KV Cache 和首 token；Decode 在每轮迭代中读取历史 KV、写入新增 KV，并将生成结果持续流式返回。运行时再将 TTFT、TPOT/ITL、队列长度和 KV 使用情况反馈给 serving policy，支持完成请求退出和新请求进入。

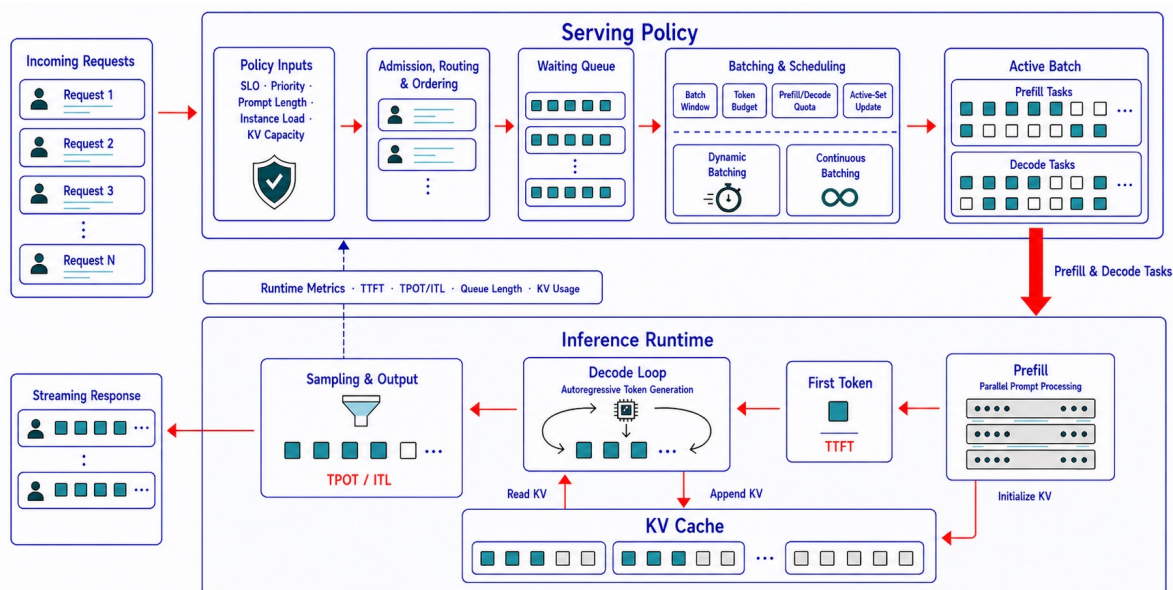


图 2 LLM serving 基础服务流程。Serving policy 作为上层控制面，内部包含准入、路由、排序、请求队列和 batching/scheduling，并根据业务与运行状态形成同时包含 prefill tasks 与 decode tasks 的活动批次；inference runtime 分别执行 Prefill、KV Cache 管理和逐 token Decode，最终形成流式响应并将运行指标反馈给 serving policy。

2.1.1 Prefill 与 Decode

LLM 推理通常分为两个阶段：**Prefill** 和 **Decode**。

Prefill 阶段负责处理完整输入上下文，包括用户 prompt、历史对话、系统提示词和检索增强内

容等。模型会对整段上下文并行计算 attention 和 FFN，并为每一层生成后续 decode 需要复用的初始 KV Cache。该阶段矩阵规模较大、并行度较高，通常更偏 compute-bound，主要影响 TTFT，即 Time To First Token。

Decode 阶段负责逐 token 生成输出。每一步 decode 输入上一步生成的新 token，同时读取历史 KV Cache，计算当前 token 的 attention 和 FFN，并将当前 token 对应的新 KV 写回缓存。该阶段每步计算粒度较小、状态依赖强、访存频繁，通常更偏 memory-bound 或 latency-bound，主要影响 TPOT、ITL 和持续生成吞吐。

Prefill 和 Decode 的资源特征差异是 LLM serving 优化的核心背景之一。Prefill 更像大矩阵并行计算，Decode 更像高频、小粒度、状态密集的在线执行。因此，很多 serving 系统会围绕 prefill/decode 的差异设计 batching、调度、KV 管理和分布式执行策略。

2.1.2 KV Cache

KV Cache 是 LLM decode 阶段最核心的运行时状态。自回归生成中，第 t 个 token 的 attention 需要访问前面所有 token 的 key/value。如果每一步都重新计算历史 token 的 key/value，会产生大量重复计算。KV Cache 的作用是在 prefill 阶段保存历史 token 的 key/value，在 decode 阶段复用这些历史 KV，只为新 token 计算新增 KV。

KV Cache 降低了 decode 阶段的重复计算，但也引入了显著的内存状态问题。KV 占用会随 batch size、上下文长度、输出长度、模型层数和 KV head 数增长。在长上下文、多轮对话和高并发场景中，KV Cache 往往成为显存容量和显存带宽的重要瓶颈。vLLM 的 PagedAttention、vAttention 的动态 KV memory management、Mooncake 的 KVCache-centric serving，以及近期 inference systems survey 都将 memory management 作为 LLM inference systems 的关键主题，覆盖 paged memory、eviction、offloading、quantization 和 cache persistence 等方向 [2], [14], [13], [6], [8]。

典型 KV 相关优化如表 1 所示：

表 1 典型 KV Cache 相关优化及作用

方向	作用
PagedAttention / memory paging	降低 KV 分配碎片，提高显存利用率
Prefix cache / RadixAttention	复用共享前缀，减少重复 prefill
KV compression / quantization	降低 KV 存储和传输成本
KV eviction	在显存不足时淘汰低价值 KV
KV offloading	将部分 KV 放到 CPU、本地存储、远端内存或其他节点
Distributed KV cache	在多 worker / 多节点场景下管理 KV 状态分布

2.1.3 Batching 与 Scheduling

Batching 是 LLM serving 提高硬件利用率和吞吐量的核心机制。单个请求，尤其是 decode 阶段，每一步计算粒度较小。如果只服务单请求，加速器容易出现算力空闲。Batching 通过把多个请求合并执行，扩大矩阵运算规模，提高 GPU/NPU occupancy，并摊薄 kernel launch、调度和访存开销。

但 LLM batching 与传统 DNN batching 不同。传统推理服务系统中的 dynamic batcher 通常在请求执行前按等待窗口和 batch size 形成批次 [15]；LLM 请求则具有输入长度、到达时间和输出长度差异大，decode 阶段请求逐步完成，batch slot 动态空出，长 prompt prefill 阻塞短请求 decode 等特征。因此，现代 LLM serving 系统通常需要 dynamic batching、continuous batching、iteration-level scheduling 和 chunked prefill 等机制。Orca 和 Sarathi-Serve 分别代表 iteration-level scheduling 与 chunked prefill 两条典型路线，近期 inference-system survey 也将 batching 和 scheduling 与 kernel design、memory management 一起作为 LLM inference systems 的核心组成部分 [1], [3], [6], [7], [8]。

典型调度机制如表 2 所示：

表 2 典型 LLM Serving 调度机制及作用

机制	作用
Dynamic batching	根据请求到达情况动态组成 batch
Continuous batching	在每个 decode iteration 动态加入新请求、移除完成请求
Iteration-level scheduling	以 token iteration 为粒度调度请求
Chunked prefill	将长 prompt prefill 拆成多个 chunk，降低对 decode 的阻塞
Priority / fairness scheduling	根据 SLA、优先级和公平性调整请求顺序
Admission control	在资源紧张时控制进入系统的请求量

Batching 的核心目标不是简单最大化 batch size，而是在 throughput、TTFT、TPOT、ITL、显存占用和 SLA 达标率之间做动态平衡。

2.1.4 主要性能指标

LLM inference 通常同时关注表 3 中的指标：

表 3 LLM 推理主要性能指标

指标	含义
TTFT	Time To First Token，首 token 延迟
TPOT	Time Per Output Token，平均每个输出 token 的生成时间
ITL	Inter-token Latency，相邻 token 输出间隔
E2E latency	从请求到完整响应结束的端到端时延
Throughput	单位时间处理的 request 数或 token 数
Goodput	满足 SLA 的有效吞吐
GPU/NPU utilization	加速器计算单元利用率
Memory footprint	权重、activation、KV Cache 等占用
Cost per token	单位 token 的计算、显存和服务成本

2.2 LLM 推理的问题空间：从执行机制到系统瓶颈

LLM 推理的系统瓶颈与其执行机制密切相关。大规模模型参数和复杂的 attention、FFN 计算带来较高的算力与显存开销；Prefill 需要并行处理完整输入上下文，而 Decode 受自回归依赖限制，只能逐 token 生成，二者不同的资源特征容易造成计算效率和时延瓶颈；KV Cache 随上下文长度和并发请求持续增长，又会增加显存容量、访存带宽和状态管理压力。同时，请求到达时间、输入长度和输出长度具有动态性，使 batching 与 scheduling 更加复杂；当推理扩展到多 GPU、多实例或多节点时，还会进一步引入通信、状态传输和服务编排开销。基于这些由执行机制产生的问题，本节从计算复杂度与模型规模、KV Cache 与内存状态、请求动态性与调度、Decode 串行性与生成加速，以及多实例、多 GPU 与服务级编排五个方面分析 LLM 推理的主要系统瓶颈。

2.2.1 计算复杂度与模型规模问题

LLM 参数规模大，单次 forward 的 FLOPs 高，attention 和 FFN 都会带来大量矩阵计算。长上下文进一步放大 attention 的计算和存储开销。相关优化通常关注如何从模型结构、参数表示或算子执行上降低计算复杂度。

这一类瓶颈首先体现为权重加载和常驻显存成本高，其次体现为 attention 与 FFN 带来的高计算量。长上下文会进一步推高 attention 计算和 KV 存储压力；MoE 等条件计算结构虽然降低了单 token 激活参数量，却引入 expert routing、expert placement 和 all-to-all 通信等新的系统问题。因此，相关优化通常沿着两条路径展开：一是通过量化、剪枝、蒸馏、低秩分解等方法降低模型参数和计算量；二是通过 GQA/MQA/MLA、efficient attention、MoE 优化、低比特执行和高效 attention kernel 改善模型结构或执行路径。

2.2.2 KV Cache 与内存状态问题

KV Cache 将 decode 从重复计算历史上下文变为增量计算当前 token，但代价是引入持续增长的运行时状态。上下文越长、batch 越大、并发越高，KV Cache 越容易成为显存容量和带宽瓶颈。

KV 问题的难点在于它不是静态模型权重，而是随请求生命周期动态增长和释放的状态。KV Cache 的显存占用会随上下文长度和并发度增长，动态分配释放又容易产生碎片；在多轮对话和共享 prompt 场景中，不同请求可能重复执行相同前缀的 prefill；在跨 worker 或跨节点 serving 中，KV 的传输、迁移、复用和远程访问会进入端到端时延关键路径。相应地，PagedAttention、prefix cache、KV compression、KV quantization、KV eviction、KV offloading、distributed KV cache 和 remote KV serving 等方法，都是围绕“如何降低状态成本、提高状态复用、控制状态移动”展开。

2.2.3 请求动态性与调度问题

LLM 请求具有高度动态性：prompt 长度不同、输出长度未知、请求到达时间随机、服务质量目标不同。Prefill 和 decode 的资源特征也不同，长 prefill 可能阻塞短 decode，请求完成时间差异又会导致 batch 内部动态变化。

调度问题的核心是吞吐和时延之间的动态权衡。等待凑 batch 可以提高硬件利用率，但会增加 TTFT；过大的 batch 会推高单请求延迟和显存占用；长 prompt prefill 可能阻塞已经进入流式输出的 decode 请求；不同租户、优先级和 SLA 又会使简单 FIFO 策略失效。因此，现代 LLM serving 通常采用 dynamic batching、continuous batching、iteration-level scheduling、chunked prefill、deadline-aware scheduling、priority scheduling 和 admission control 等机制，在 batch 形成、active set 更新和请求优先级之间做在线决策。

2.2.4 Decode 串行性与生成加速问题

LLM decode 是自回归过程，每一步依赖前一步输出，天然存在串行依赖。即使单步 forward 被加速，逐 token 生成仍可能造成较高 TPOT 和 ITL。

这一瓶颈与模型规模无关，即使单步 decode 的 kernel 足够快，自回归依赖仍会限制输出速度。单步 decode 的计算粒度较小，硬件利用率往往不如 prefill；sampling、beam search 或复杂 decoding policy 还会增加控制开销。Speculative decoding、assisted decoding、parallel decoding、tree decoding、beam search optimization、multi-token prediction 和 adaptive drafter selection 等方法，试图通过草稿生成、批量验证或多 token 预测减少目标模型的串行调用次数，但它们也必须在 acceptance rate、额外计算、输出质量和系统复杂度之间取得平衡。

2.2.5 多实例、多 GPU 与服务级编排问题

当模型规模、并发量或服务可用性需求超过单设备能力时，系统需要跨多个 GPU、worker、replica 或节点协同。此时问题从单个 engine 内部执行扩展为分布式执行和服务级调度。

分布式化带来的收益与代价同时存在。Tensor、pipeline、expert 或 context parallel 可以突破单设备容量和吞吐限制，但会引入 activation、KV 和中间状态的跨设备传输；prefill 与 decode 的资源特征差异使 disaggregation 成为可能，也使 KV handoff 成为新的关键路径；多 replica 和多模型服务可以提高可用性和弹性，却需要在负载均衡、质量、时延和成本之间做服务级选择。因此，distributed parallelism、prefill/decode disaggregation、distributed KV cache、KV transfer、replica routing、cluster load balancing、cascade serving 和 speculative serving orchestration 等方法，本质上都在处理“如何将单实例推理扩展成可控的分布式服务”这一问题。

2.3 LLM Inference Optimization Taxonomy: 以优化对象为主轴的分类

围绕 LLM inference 相关优化方向分类的 overview 和 survey 工作较多，表 4 汇总了其中代表性工作的分类方式及其特点。

表 4 现有 LLM Inference Overview 与 Survey 的分类方式

工作	分类视角	主要分类	分类特点
<i>A Survey on Efficient Inference for Large Language Models</i> [6]	优化层次	Data-Level、Model-Level、System-Level	覆盖从输入数据、模型结构到系统实现的多层优化，适合从整体上理解推理效率优化
<i>A Survey of LLM Inference Systems</i> [8]	系统流水线与系统形态	Request processing、model optimization and execution、memory management，以及 single-replica、multi-replica、disaggregated、serverless systems	对 system-level 方法拆分较细，突出 kernel、batching/scheduling、memory management 与不同部署形态
<i>Taming the Titans: A Survey of Efficient LLM Inference Serving</i> [7]	Serving 层级与场景	Instance-Level Approaches、Cluster-Level Strategies、Emerging Scenarios、Miscellaneous Areas	强调从单实例优化扩展到集群部署、负载均衡和云服务等 serving 问题

基于这些工作，本报告进一步整合形成 LLM 推理优化方向的 **Layer-Scope Taxonomy**：以主要优化对象所在的系统抽象层为主轴，将方法分为 Model-Level、Kernel-Level、Runtime-Level 和 Serving Policy & Routing-Level；同时以执行范围为副轴，区分 Local 与 Distributed。图 3 将这一二维分类展开：纵向表示主要优化对象从模型结构、底层算子、在线运行时到服务级决策逐层上升，横向表示执行范围从单实例或本地控制域扩展到跨 device、worker、replica、service 或 node 的分布式协同。后续 2.3.1–2.3.4 将依次介绍四个层级，并在每个层级中讨论其本地与分布式实现。

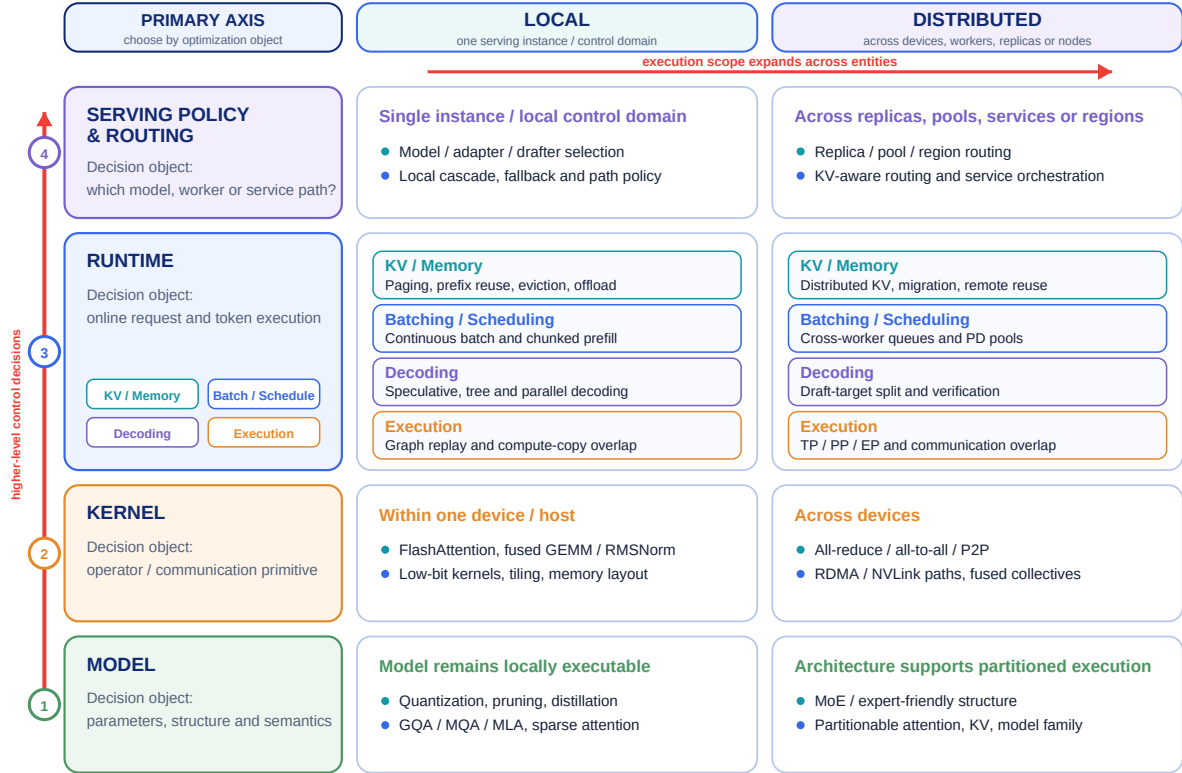


图3 LLM Inference Optimization 的 Layer-Scope Taxonomy。纵轴按照主要优化对象区分 Model、Kernel、Runtime 与 Serving Policy & Routing 四个层级，向上表示决策对象从模型与执行基础逐渐扩展到高层服务控制；横轴按照执行范围区分 Local 与 Distributed，向右表示从单实例或本地控制域扩展到跨 device、worker、replica、service 或 node 的协同。Runtime 层进一步包含 KV/Memory、Batching/Scheduling、Decoding 和 Execution 四类在线机制。

其中，Local 表示优化限制在单个 serving instance 或本地控制域内，可以包含 CPU-GPU/NPU 和本地存储等异构资源协同；Distributed 表示优化需要跨多个 GPU、device、worker、replica、service 或 node 协同，并涉及 computation、request、KV state、activation 或 routing decision 的跨实体分布。需要说明的是，这一分类并不要求所有方法完全互斥，真实 LLM serving 系统通常具有 cross-layer co-design 特征。因此，在归类一项具体工作时，本报告主要考察三个问题：它解决什么 inference problem，主要优化对象位于哪一层，以及该优化发生在 Local 还是 Distributed scope。

2.3.1 Model-Level Optimization

Model-Level Optimization 的核心是改变模型本身，从源头降低 inference complexity。它优化的是模型参数、参数表示、结构设计或算法语义。只要优化后的模型即使脱离具体 serving engine 仍然成立，就可以认为主要属于 Model-Level。这一层的优化对象包括参数量、权重表示、FLOPs、activation 计算量、attention complexity、KV size 以及条件计算路径。

Local scope: 常见方法包括 weight quantization、pruning、distillation、low-rank、weight sharing、GQA/MQA/MLA、linear 或 sparse attention、state space model、early exit、token pruning 和 dynamic sparsity 等，主要服务于单实例或单设备上的模型压缩和结构简化。

Distributed scope: Model-Level 更强调模型结构是否适合跨设备执行, 例如面向 expert parallelism 的 MoE architecture、更适合 tensor/context parallel 的 attention 或 KV 结构、更容易按 layer、expert、context 或 activation 切分的模型结构, 以及由 small / medium / large model 组成的多层级 model family。

需要注意, Model-Level 不包括一般的 execution graph optimization。CUDA Graph、operator fusion、TensorRT execution graph 优化等主要改变的是执行路径或部署图, 而不是模型语义本身, 应归入 Runtime-Level 或 Kernel-Level。

2.3.2 Kernel-Level Optimization

Kernel-Level Optimization 的核心是在不改变模型数学语义的前提下, 优化 tensor operator 或 communication primitive 在硬件上的执行效率。它关注的是 attention、GEMM、normalization、sampling、collective communication 等底层执行单元如何更快、更省显存带宽、更高效地利用硬件, 其直接目标通常是提高 GPU/NPU utilization、降低 HBM traffic、增强 SRAM/cache reuse、提升 Tensor Core 或 NPU special unit 利用率, 并改善 operator throughput。

Local scope: 典型方法包括 FlashAttention、FlashDecoding、paged attention kernel path、fused MLP、fused RMSNorm、fused softmax、low-bit GEMM、FP8/INT8/INT4 kernels、hardware-aware tiling、memory layout 优化和 GPU-side sampling 等。

Distributed scope: Kernel-Level 主要体现为 collective 或 P2P communication primitive 的实现优化, 例如 all-reduce、all-gather、reduce-scatter、all-to-all、RDMA/NVLink transfer path、fused communication primitive、collective algorithm selection 以及 MoE expert dispatch / combine primitive。

需要区分的是, 如果优化对象是 collective / P2P primitive 的实现路径, 归为 Kernel-Level $\times$ Distributed; 如果优化对象是通信与计算在 runtime pipeline 中的编排、重叠和调度, 则归为 Runtime-Level $\times$ Distributed。

2.3.3 Runtime-Level Optimization

Runtime-Level Optimization 是现代 LLM serving 的核心层。它发生在请求进入 inference engine 之后, 围绕在线 token execution、KV state、batch composition、decode loop 和 execution pipeline 做动态管理。Runtime-Level 不改变模型本身, 也不一定改变单个 kernel, 而是决定请求到达后如何高效执行。Runtime-Level 可以进一步细分为四类: KV / Memory Runtime、Batching / Scheduling Runtime、Decoding Runtime 和 Execution Runtime。

A. KV / Memory Runtime KV / Memory Runtime 关注 KV lifecycle、memory state 和 cache reuse。它的核心问题是: KV Cache 如何存储、复用、压缩、淘汰、迁移或远程访问。

Local scope: 主要包括 KV cache management、PagedAttention / memory paging、Prefix cache / RadixAttention、KV compression / quantization、KV eviction、KV offloading、cache persistence 和 runtime memory allocator 等机制。

Distributed scope: 问题进一步扩展为 distributed KV cache、KV transfer / migration、remote KV serving、multi-replica KV reuse、distributed prefix cache、KV-aware placement 和 cross-worker state synchronization。

这一类优化的核心目标是降低 KV 显存占用、减少重复 prefill、提升 cache reuse，并在分布式场景下控制 KV 传输和状态迁移成本。

B. Batching / Scheduling Runtime Batching / Scheduling Runtime 关注请求和 token 在 inference engine 内如何组织执行。它回答的是：哪些请求何时执行，哪些 token 可以组成 batch，prefill 和 decode 如何交错，如何在 throughput、TTFT、TPOT、ITL、公平性和 SLA 之间做权衡。

Local scope: 典型机制包括 dynamic batching、continuous batching、iteration-level scheduling、chunked prefill、prefill/decode interleaving、engine-local admission control、priority / fair scheduling 和 deadline-aware scheduling。

Distributed scope: 调度对象会跨越 worker、replica 或资源池，因此需要 cross-worker scheduling、distributed continuous batching、prefill worker / decode worker scheduling、heterogeneous GPU scheduling、multi-replica queue coordination、distributed admission control 和 SLO-aware worker scheduling。

这一类优化的核心目标是在动态请求负载下提高硬件利用率，同时控制排队时延和服务质量。

C. Decoding Runtime Decoding Runtime 关注在线 token generation procedure。它回答的是：输出 token 如何生成、验证、选择和接受。它不主要决定请求如何组 batch，也不主要决定计算如何 launch，而是改变 decode loop 的生成逻辑。

Local scope: 这类方法包括 speculative decoding、assisted decoding、parallel decoding、tree decoding、beam search optimization、n-gram speculation、prompt lookup decoding 和 serving-time multi-token prediction / verification。

Distributed scope: Decoding Runtime 会涉及 distributed verification、draft / target execution split、multi-draft execution、batched verification across workers、draft model 与 target model 分布式协同，以及 speculative decoding 与 prefill/decode disaggregation 的结合。

Speculative decoding 这类方法有时也被称为 decoding algorithm。本报告将其归入 Runtime-Level，是因为其主要改变 serving 时的在线 token generation procedure，而不是模型参数或架构本身。但如果某种 multi-token prediction 方法引入额外训练 heads 或改变模型结构，则应标注 Model-Level primary 或 secondary。

D. Execution Runtime Execution Runtime 关注 scheduled work 如何 launch、overlap、replay 和执行。当前面的 runtime 已经决定哪些请求要运行、batch 如何组成、decode 如何进行之后，Execution Runtime 负责将计算、数据搬运和通信高效提交到硬件，并组织成流水线执行。

Local scope: 主要包括 CUDA Graph / HIP Graph replay、async execution、stream scheduling、CPU-GPU overlap、host-device communication hiding、compute / memory-copy overlap、runtime metadata / buffer reuse 和 GPU-side sampling execution path。

Distributed scope: 主要包括 tensor parallel runtime、pipeline parallel runtime、expert parallel runtime、context / sequence parallel runtime、prefill/decode disaggregation、distributed execution pipeline、communication-computation overlap scheduling 以及 activation / KV transfer orchestration。

Execution Runtime 与 Kernel-Level 的边界在于：Kernel-Level 优化单个 operator 或 communication primitive 的实现；Execution Runtime 优化一组 operator、copy 和 communication 的 launch、同步、overlap 和流水化组织方式。

2.3.4 Serving Policy & Routing-Level Optimization

Serving Policy & Routing-Level Optimization 关注更粗粒度的服务级控制决策。它不直接优化单个 engine 内部 token 如何执行，而是决定请求进入哪个模型、adapter、replica、worker pool 或 service path，以及是否进行 fallback、cascade、escalation 或多服务编排。这一层的典型优化对象包括 request assignment、model / adapter selection、replica / worker selection、service path selection、fallback / escalation、cascade policy、SLO / cost / latency policy 和 multi-service orchestration。

Local scope: 主要包括 local model selection、adapter / LoRA routing、local cascade serving、normal decoding 与 speculative decoding path selection、drafter selection、local/cloud fallback、budget-aware local inference 和 latency-aware local policy。

Distributed scope: 进一步扩展为 replica routing、worker-pool routing、cluster load balancing、SLO-aware routing、cost-aware routing、region-aware routing、KV-aware routing、speculative serving orchestration、draft-target worker orchestration 和 multi-service graph orchestration。

Runtime-Level 和 Serving Policy & Routing-Level 在真实系统中强耦合，但 primary decision object 不同。Runtime-Level 解决“请求进入某个 engine 后如何执行”；Serving Policy & Routing-Level 解决“请求应该交给哪个模型、worker、replica 或服务路径”。后者经常依赖 runtime feedback，例如 queue length、KV cache pressure、batch state 和 SLO slack。

2.4 工业级 LLM Inference Framework 代表性机制映射

为了避免分类停留在抽象概念层面，表 5 选取若干主流工业级 LLM inference framework 中最具有代表性的机制，并将其映射到前述 Layer-Scope Taxonomy。该表不追求覆盖各框架的全部功能，而是展示每个框架最核心、最能体现其系统设计特点的若干机制。

表 5 工业级 LLM Inference Framework 代表性机制映射

Framework	代表性方法 / 机制	解决的具体问题	优化方向分类层 & Scope	具体方法说明
vLLM [2]	PagedAttention	KV Cache 动态分配和显存碎片问题	Runtime / KV-Memory $\times$ Local	通过分页式 KV 管理降低显存碎片，提高 KV Cache 利用率
	Continuous batching	Decode 阶段请求动态进出，静态 batch 利用率低	Runtime / Batching-Scheduling $\times$ Local	在 decode iteration 动态加入和移除请求，提高高并发吞吐
	Chunked prefill	长 prompt prefill 阻塞 decode，影响 TTFT / ITL	Runtime / Batching-Scheduling $\times$ Local	将长 prefill 拆分为 chunk，与 decode 交错执行
	Prefix caching	多请求共享前缀导致重复 prefill	Runtime / KV-Memory $\times$ Local/Distributed	复用共享 prompt 或 prefix，减少重复 prefill 计算
TensorRT-LLM [16]	In-flight batching	高并发请求长度不同，传统 batching 效率低	Runtime / Batching-Scheduling $\times$ Local	对不同阶段和长度的请求进行动态批处理
	Paged KV Cache	KV Cache 分配、复用和显存管理问题	Runtime / KV-Memory $\times$ Local	通过 paged KV 管理改善 KV Cache 利用率
	FP8 / INT8 / INT4 execution	权重、activation 和算子执行成本高	Model + Kernel $\times$ Local/Distributed	通过低比特模型表示和低比特 kernel 降低显存和计算成本
	Overlap scheduler	compute、copy、communication、prefill/decode 没有充分重叠	Runtime / Execution $\times$ Local/Distributed	将计算、通信和数据搬运流水线化，减少等待
SGLang [5]	RadixAttention	多轮、多分支请求存在大量共享前缀	Runtime / KV-Memory $\times$ Local	用 radix tree 组织 prefix cache，提高共享前缀复用
	Zero-overhead CPU scheduler	高并发场景下 CPU 调度开销影响吞吐	Runtime / Batching-Scheduling $\times$ Local	降低 CPU 侧调度和 metadata 管理开销
	Prefill-decode disaggregation	Prefill 和 decode 资源特征不同，混部互相干扰	Runtime / Execution $\times$ Distributed	将 prefill 与 decode 分离到不同资源池，减少资源干扰
	Multi-LoRA batching	多 adapter 请求混合服务时 batch composition 复杂，吞吐下降	Serving Policy & Routing + Runtime/Scheduling $\times$ Local/Distributed	将不同 LoRA / adapter 请求合并调度，提高多 LoRA serving 效率
Hugging Face TGI [17]	Continuous batching	incoming requests 动态变化，传统 batching 吞吐不足	Runtime / Batching-Scheduling $\times$ Local	动态合并请求，提高在线 serving 吞吐
	Flash Attention	attention HBM traffic 高，operator throughput 受限	Kernel $\times$ Local	通过高效 attention kernel 降低显存访问并提升算子吞吐
	Paged Attention	KV Cache / attention memory 管理效率不足	Runtime / KV-Memory $\times$ Local	通过 paged attention 改善 attention/KV memory 管理
	Tensor parallelism	单 GPU 容量或吞吐不足，需要跨 GPU 执行	Runtime / Execution $\times$ Distributed	将模型执行切分到多个 GPU，提高模型容量和吞吐
LMDeploy / TurboMind [18]	Persistent batching	高并发请求动态调度，提高吞吐	Runtime / Batching-Scheduling $\times$ Local	通过持续批处理机制维持 batch 执行效率
	Blocked KV Cache	KV Cache 显存管理和访问效率问题	Runtime / KV-Memory $\times$ Local	通过 block-based KV 管理改善显存利用率
	Dynamic split & fuse	长 prompt 和 decode 混合执行效率低	Runtime / Batching-Scheduling $\times$ Local	动态拆分和融合 prefill / decode 任务，提高混合负载效率
	Tensor parallelism	大模型需要跨 GPU 执行	Runtime / Execution $\times$ Distributed	支持模型在多 GPU 上并行执行

2.5 小结

本章首先介绍了 LLM inference 的基本执行机制，包括 prefill/decode、KV Cache、batching/scheduling 和主要性能指标。随后，从执行机制出发，总结了 LLM inference 的主要问题空间，包括计算复杂度与模型规模、KV Cache 与内存状态、请求动态性与调度、decode 串行性与生成加速，以及多实例、多 GPU 与服务级编排问题。

在已有 overview 和 survey 工作的基础上，本章进一步整合形成 Layer-Scope Taxonomy：以 Model、Kernel、Runtime、Serving Policy & Routing 作为优化对象主轴，以 Local / Distributed 作为执行范围副轴，并在 Runtime-Level 内部细分为 KV / Memory Runtime、Batching / Scheduling Runtime、Decoding Runtime 和 Execution Runtime。该分类方式能够同时描述优化机制所处的系统层级及其执行范围，并适用于分析具有 cross-layer co-design 特征的 LLM 推理系统。

最后，本章通过主流工业级 inference framework 的代表性机制映射，展示了真实系统通常不是单点优化，而是组合模型表示、kernel、KV/memory runtime、batching/scheduling、execution runtime、distributed execution 和 serving policy 等多层机制。该分类框架将作为后续分析 CEC/MEC 场景下模型部署、请求调度、KV 状态迁移、服务路径选择和端-边-云资源协同优化的基础。

三、CEC 场景下的大模型推理

3.1 CEC LLM 推理的系统形态

CEC 场景下的大模型推理面向端侧设备、接入边缘、区域边缘和中心云组成的多层协同系统。请求可以从终端、基站、边缘网关或区域入口进入，模型权重、adapter、KV Cache、prefix cache、session state 和中间状态也可能分布在不同层级。一次推理既可能在单个端侧或边缘实例内完成，也可能经过端-边、边-云或端-边-云联合执行。

与云侧集中式 LLM serving 相比，CEC 的根本变化在于系统边界被拉长了。云侧 serving 通常假设资源相对集中、互联稳定、控制面统一，优化重点是单实例或集群内部的吞吐、显存和调度效率。CEC 则把地理位置、无线链路、异构硬件、移动接入、状态分布和隐私边界都纳入推理路径。此时，推理系统不只是选择一个 GPU 实例执行请求，而是要决定请求在哪一层服务、模型如何部署、状态如何保留、网络传输是否值得、以及服务路径何时调整。图 4 展示了模型分割部署、计算任务调度和 KV / runtime state 调度在端、边、云多层环境中的协同关系。

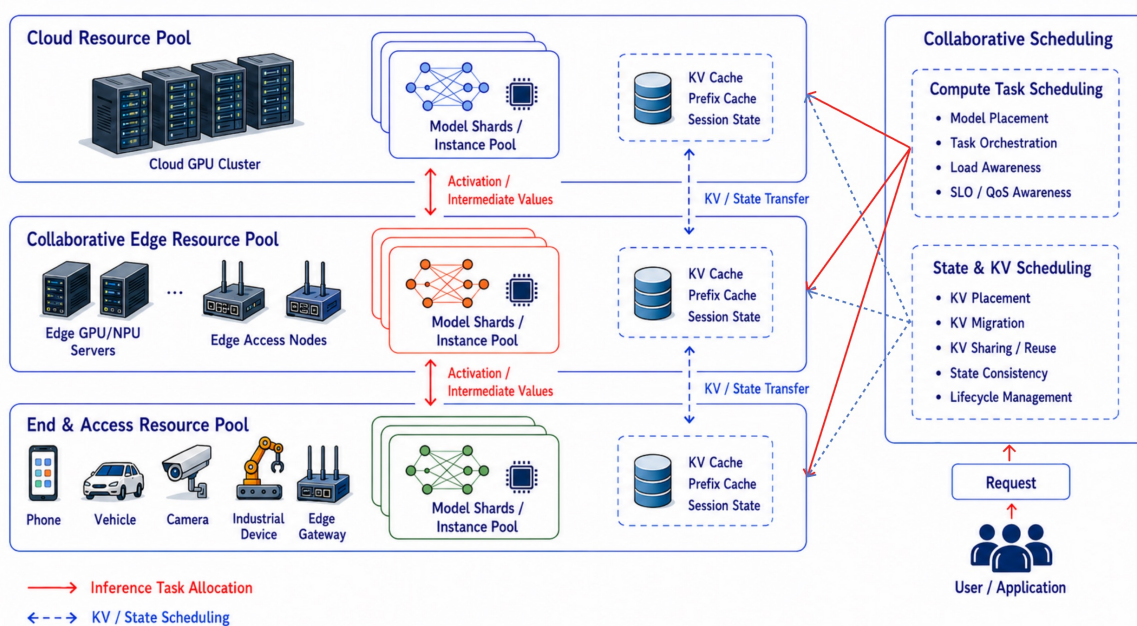


图 4 CEC 环境下的 LLM 推理 Serving 系统形态。Serving Policy 根据节点资源、链路与运行状态，协调端侧、接入边缘、区域边缘和中心云之间的模型分割与部署、Prefill/Decode 等计算任务调度，以及 KV Cache、Prefix Cache 和 Session State 的复用、迁移与远程访问。

因此，CEC LLM inference 可以看作是三类决策的叠加。第一类是空间部署决策，即模型层、张量、专家组件、推理阶段或生成职责如何放在端、边、云不同资源上。第二类是时间调度决策，即请求、token、prefill/decode 阶段和 active batch 如何在动态负载下组织执行。第三类是服务路径决策，即完整请求、会话、workflow 和运行时状态如何在多层网络中路由、卸载、迁移或复用。后续章节

的三条主线正是从这三个角度展开。

3.2 CEC 相较云侧集中式 LLM Serving 新增的问题空间

资源受限与硬件异构：云侧集中式 serving 的核心瓶颈通常来自模型规模、KV Cache、batching、并行执行和多实例调度。CEC 并没有消除这些问题，而是把它们放到一个资源更加受限、硬件更加异构的物理环境中。端侧和边缘节点的算力、显存、内存、功耗和散热预算有限，同一个模型未必能完整放入目标设备；即便能够部署，也可能受到低比特 kernel 支持、operator coverage、runtime backend 和 layout conversion 的限制。由此，模型部署问题从“如何在—组相似 GPU 上高效执行”，转变为“如何在能力差异显著的端-边-云节点之间选择模型形态和执行拓扑”。

网络波动与状态移动成本：端-边、边-边和边-云链路的带宽、RTT、抖动和拥塞状态会直接影响模型切分、task offloading、prefill/decode 分离和 KV 迁移是否值得执行。对于普通 DNN 推理，跨节点传输的主要对象通常是输入或中间 activation；而在 LLM inference 中，KV Cache、prefix cache、session state 和多轮上下文也会成为需要迁移或远程访问的状态资源。网络成本因此不再只是请求入口到服务实例的传输延迟，而是贯穿推理全过程的状态移动成本。

低并发、突发请求与调度失配：单个接入边缘节点往往并发规模有限，请求到达具有地理局部性和突发性，简单等待大 batch 会显著增加 TTFT；但如果完全放弃 batching，又会降低 GPU/NPU 利用率并抬高单位 token 成本。与此同时，prefill 和 decode 的资源画像差异、长 prompt 和短 prompt 的混合、不同业务的 SLA 差异，都会使固定 batch size 和 FIFO 队列难以稳定工作。边缘 runtime 因此更需要短窗口、SLO-aware、phase-aware 和状态感知的在线调度。

用户移动与服务连续性：对于单轮短请求，移动可能只改变入口节点；对于多轮对话、长上下文和 agent workflow，移动会改变服务实例与状态位置之间的关系。KV Cache、prefix cache、session state 或 agent memory 可能仍然停留在旧边缘节点，新请求却已经从新入口进入系统。系统此时需要在迁移状态、远程访问旧状态、重新 prefill 或降低上下文质量之间做选择。这个问题使 CEC LLM inference 的优化目标从单次请求时延扩展到会话连续性、状态复用和移动场景下的 QoS 稳定性。

隐私边界与数据本地性：部分 prompt、传感器数据、业务上下文或用户状态可能不能上传到云端；某些行业应用还要求推理过程中的中间状态不离开边缘域。隐私约束会限制模型选择、offloading 范围、KV 迁移路径和 fallback 策略，使系统不能仅按最小时延或最低成本选择节点。换言之，CEC 中的推理优化必须同时处理资源可行性、链路代价、状态位置、隐私边界和业务 QoS。

3.3 CEC 场景下 LLM Inference 优化重心的转移

Model-Level：优化重心从单模型压缩转向分层模型体系与可切分性。前文提出的 Layer-Scope taxonomy 仍然适用于 CEC 场景，但每一层的关注点都会发生迁移。Model-Level 不再只是压缩模型或降

低 FLOPs，而是要形成端侧小模型、边缘中等模型、云端大模型及其 adapter / MoE / 分层模型族之间的协同体系。模型结构是否容易按 layer、tensor、expert、stage 或 semantic responsibility 切分，会直接影响后续部署和 offloading 的可行性。

Kernel-Level: 优化重心从单一 GPU 性能转向异构硬件适配与稳定执行。CEC 系统可能同时包含端侧 NPU、移动 GPU、边缘 GPU/NPU、CPU 和云端 GPU，不同后端对 precision、operator、attention kernel、KV layout 和通信 primitive 的支持并不一致。一个模型或 runtime 在云端高效，并不意味着它能在边缘设备上稳定运行；unsupported operator、layout conversion 和 fallback 甚至可能抵消协同执行带来的收益。

Runtime-Level: 优化重心从实例内部执行转向网络、计算与状态的联合在线控制。云侧 runtime 主要管理单个 serving engine 或 GPU cluster 内的 KV、batch、decode loop 和 execution pipeline；CEC runtime 则必须把网络状态、节点负载、KV 位置、用户移动和业务 SLA 纳入同一个在线控制面。KV / Memory Runtime 从单实例显存管理扩展为分层状态管理；Batching / Scheduling Runtime 从大请求池吞吐优化转向低并发、短等待、SLO-aware 和 phase-aware 调度；Decoding Runtime 需要考虑端侧或边缘小模型作为 drafter 的可用性；Execution Runtime 则要把端侧执行、网络传输和边缘/云端计算组织成可重配置的流水线。

Serving Policy & Routing-Level: 优化重心从副本选择转向端-边-云服务路径的联合决策。这一层需要联合考虑模型层级、任务 offloading、状态位置和隐私约束。一个请求是否应该本地执行、卸载到接入边缘、升级到区域边缘或转发到云端，取决于请求复杂度、模型能力、链路状态、节点负载、KV/cache 位置、用户移动趋势和 SLA 风险。该层还需要处理 fallback、escalation、state-aware routing 和 mobility-aware routing，使服务策略能够随着环境变化而调整。

总体变化: Distributed Scope 被显著强化，各层之间的耦合更加紧密。CEC 并不是在 LLM inference taxonomy 之外新增一个孤立层，而是显著强化了 Distributed Scope 和 Serving Policy & Routing-Level 的作用，并使 Model、Runtime 和 Routing 之间的边界更紧密。模型是否可切分会影响部署空间，batching 是否可调整会影响请求 offloading 收益，KV 是否可迁移会影响服务路径选择。后续章节不再按单一抽象层展开，而是围绕最能体现 CEC 特征的三类算法问题组织相关工作。

3.4 本文采用的 CEC LLM Inference 综述框架

基于上述分析，本文将 CEC 场景下的 LLM inference 优化归纳为三条主线。第一条是资源自适应模型部署方法，它对应空间部署问题，关注模型结构、模型组件、推理阶段和生成职责如何根据端、边、云资源能力进行切分、放置和动态调整。该方向覆盖 layer-wise / vertical partitioning、tensor / operator-level partitioning、expert / component-level partitioning、inference phase partitioning 和 semantic / token-level partitioning，第 4 章将围绕这一类方法展开。

第二条是业务感知 batching 参数优化算法, 它对应时间调度问题, 关注请求进入推理实例后如何被组织成 batch、active set 和阶段执行流。在边缘低并发、强时延约束、请求动态到达、KV Cache 压力和 QoS 差异条件下, 系统需要动态选择 batching window、batch size、admission threshold、continuous batching 策略、chunked prefill 粒度和 prefill/decode 分池方式。第 5 章将围绕这一类方法展开。

第三条是 LLM inference task offloading, 它对应服务路径问题, 关注完整请求、会话、复杂 workflow、模型/agent 组合以及 context、KV cache、prefix cache、session state 等运行时状态如何在端、边、云之间路由、卸载、迁移、复用或重新计算。该方向包含 request/session-level routing and offloading、multi-model / multi-agent workflow orchestration、context / KV / runtime state offloading and migration, 并与用户移动、隐私边界和 QoS 目标密切相关。第 6 章将围绕这一类方法展开。

如图 5 所示, 这三条主线分别从空间部署、时间调度和服务路径三个角度刻画 CEC LLM inference 的关键优化问题。它们并非彼此独立: 模型部署决定可用的执行拓扑, batching 决定实例内部的时间组织, task offloading 决定请求和状态如何在多层系统中移动。第 7 章将进一步讨论三类算法之间的协同优化关系。

3.5 小结

本章从系统形态出发, 说明 CEC 将 LLM inference 从云侧集中式 serving 推向端、边、云多层协同服务。相较数据中心环境, CEC 额外引入资源异构、网络波动、边缘低并发、用户移动、状态连续性、隐私边界和 QoS 差异等约束, 使推理优化从单点执行效率转向跨节点、跨状态和跨服务路径的联合决策。

基于 Layer-Scope taxonomy, 本章进一步指出: CEC 不改变 LLM inference 优化的基本抽象层, 但会改变各层的优化重心。Model-Level 更关注可部署、可切分和可协同的模型体系; Kernel-Level 更关注异构硬件适配; Runtime-Level 更关注网络、状态、batch 和执行流水线的在线管理; Serving Policy & Routing-Level 更关注服务路径、模型层级、task offloading、状态位置和 QoS 的联合决策。

基于这些变化, 本报告后续围绕三类 CEC LLM inference 优化主线展开: CEC 下的资源自适应模型部署方法、CEC 中的业务感知 batching 参数优化算法, 以及 CEC 中的 LLM inference task offloading。

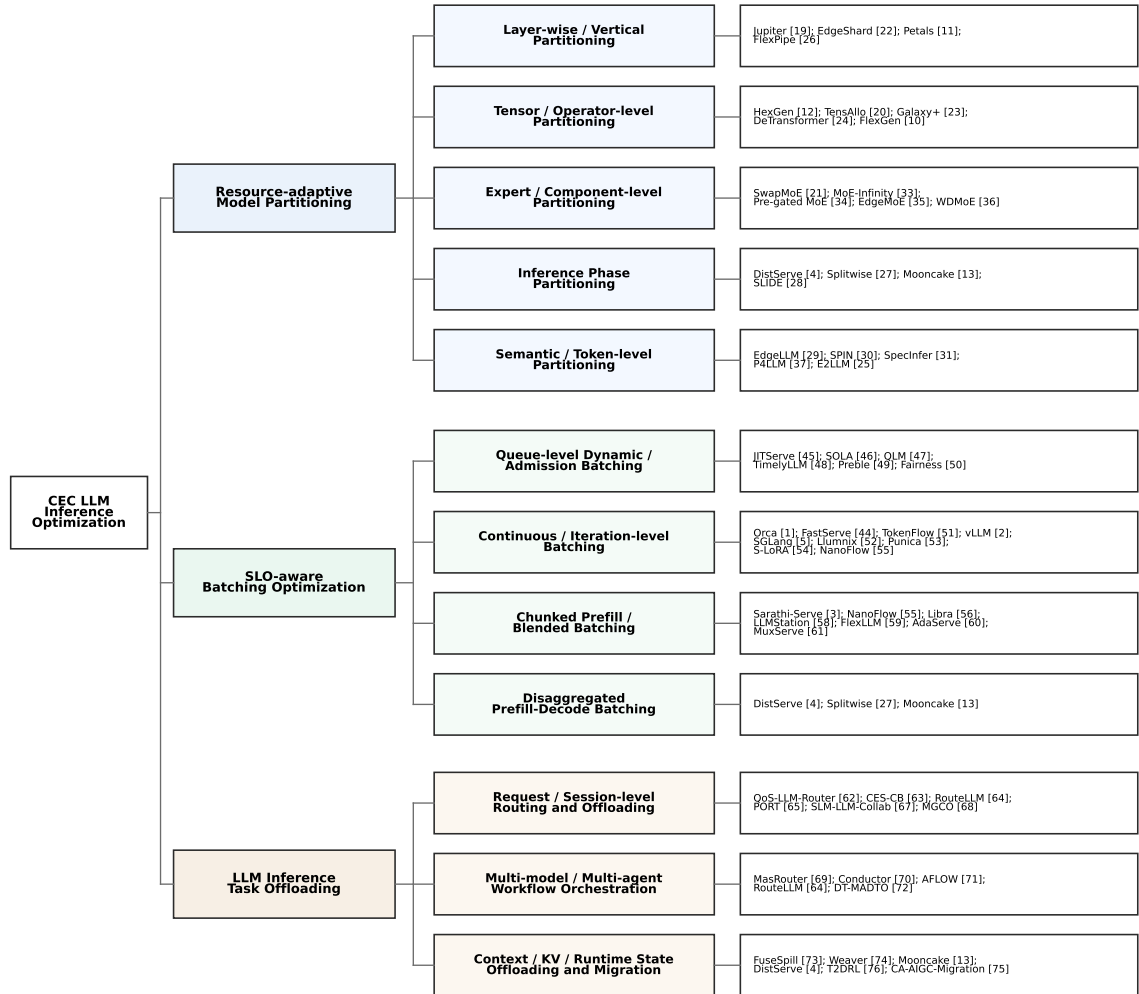


图5 CEC LLM inference 优化主线、子分类与代表工作总览

四、CEC 下的资源自适应的模型切分部署方法

大语言模型推理在 CEC 场景下面临的首要问题，不只是单个模型能否放入某个设备，而是同一次推理中的模型结构、计算阶段和运行时状态能否被合理分布到端侧、接入边缘、区域边缘和云端。端侧设备靠近用户并具有隐私优势，但算力、内存、功耗和散热约束严格；边缘节点提供近用户算力，却常常呈现资源碎片化、设备异构和负载波动；云端算力充足，但网络往返、带宽消耗和数据出域风险会进入端到端服务路径。模型切分部署的价值，正是在这些资源边界之间重新组织一次推理，使大模型不再被单个节点的容量和位置限制。Petals 说明大模型可以通过协作式 block 放置突破单节点容量限制 [11]，HexGen 说明异构 GPU 和异构网络会改变生成式推理的并行策略 [12]，FlexGen 则从单 GPU 场景展示权重、activation 和 KV Cache 的分层放置对吞吐的影响 [10]。这些工作共同表明，LLM 推理的部署瓶颈通常同时来自模型权重、activation、KV Cache、互联带宽和运行时调度，而不是单一计算瓶颈。与一般服务路由不同，模型切分部署关心的是推理内部的执行结构是否发生变化。仅在多个完整模型或完整服务实例之间选择执行路径的方法，本质上属于服务路径选择或负载均衡；它可能影响时延和成本，但没有切分模型本体，也没有改变一次推理中计算、状态或生成职责的组织方式。因此，本章只讨论会改变模型执行拓扑、层内并行方式、推理阶段边界或 token 生成职责分工的方法。这样的边界可以避免把“请求发到哪里”和“模型如何被拆开执行”混为一谈，也便于比较不同方法真正改变的系统对象。本章将 request-level routing 排除在模型切分部署之外，是为了保持分类口径一致；若请求路由同时触发模型段迁移、阶段迁移或 expert 组件迁移，则其模型执行结构变化部分仍可纳入本章讨论。本文按 primary optimization object 将相关方法划分为五类：

1. **Layer-wise / Vertical Partitioning**: 切的是 Transformer 深度方向，决定 layer、block 或 model shard 放在哪些节点上执行。
2. **Tensor / Operator-level Partitioning**: 切的是同一层内部的计算单元，决定 attention head、MLP 矩阵、tensor shard 或 communication primitive 如何并行。
3. **Expert / Component-level Partitioning**: 切的是 MoE expert 等模型组件，决定哪些 expert 常驻、换入、映射或跨层级放置。
4. **Inference Phase Partitioning**: 切的是 prefill、decode、verification、download 等执行阶段，利用不同阶段的资源画像差异。
5. **Semantic / Token-level Partitioning**: 切的是生成过程本身，决定 draft、verify、semantic segment、token-wise partition 等语义单元如何协作。

这五类方法不是严格互斥的。Jupiter 同时涉及边缘协作执行和 prefill/decode 阶段组织 [19]；TensAllo 同时包含设备选择、模型部署和张量并行 [20]；Mooncake 以 prefill/decode 解耦为主，但其收益依赖分布式 KV Cache [13]；SwapMoE 则更接近专家级动态驻留与换入 [21]。本文按主要优化对象

归类，交叉关系在总览表中保留。这个口径的目的不是把每篇工作强行放进唯一格子，而是让读者能看清：某个方法到底是在切模型深度、切层内计算、切专家组件、切执行阶段，还是切生成语义。

表中“切分类型”表示论文中最接近本章 taxonomy 的主优化对象，不等价于论文全部贡献。例如 Mooncake 的核心贡献包含 KVCache-centric 架构，本章将其放入阶段切分，是因为其服务路径首先建立在 prefill/decode disaggregation 之上 [13]。

表 6 汇总了本章五类模型切分部署方法的代表工作、发表位置、切分类型、优化目标与加速机制；图 6 进一步以树状结构展示五类方法、主要切分对象和代表工作之间的对应关系。

表 6 CEC 下资源自适应模型切分部署方法相关工作

序号	System / Framework	Venue / Year	切分类型	Objectives	Optimization Technology	加速机制 / 方法核心
1	Jupiter: Fast and Resource-Efficient Collaborative Inference of Generative LLMs on Edge Devices [19]	IEEE INFOCOM 2025	层级切分 / 推理阶段切分	prefill/decode 协同执行	intra-sequence pipeline parallelism; outline-based pipeline parallel decoding; speculative decoding	Prefill 阶段用序列内流水线并行分摊长 prompt 计算；decode 阶段用 outline-based pipeline parallel decoding 和 speculative decoding 减少自回归串行步数与流水线气泡
2	TensAllo: Adaptive Deployment of LLMs on Resource-Constrained Heterogeneous Edge Devices [20]	IEEE INFOCOM 2025	层级切分 / 张量切分	在资源受限异构边缘设备上部署 LLM	adaptive device selection; tensor allocation; tensor parallelism; quantization	通过设备选择和 tensor allocation 把计算分配到更合适的异构设备；再用量化降低内存占用，使更多 tensor shard 能留在边缘设备上执行
3	EdgeShard: Efficient LLM Inference via Collaborative Edge Computing [22]	IEEE IoT Journal 2025	层级切分 / 垂直切分	模型分片放置	model shard placement; joint device selection; dynamic programming	将 LLM 切成 model shards，并联合选择设备和 shard placement；动态规划寻找计算时延、传输代价和 pipeline 吞吐之间更优的放置方案
4	Petals: Collaborative Inference and Fine-tuning of Large Models [11]	ACL 2023 Demo	层级切分 / 垂直切分	将大模型 block 分布到多个参与节点	decentralized layer/block placement; pipeline across volunteered servers	把 Transformer block 分散存放在志愿节点 GPU 上，客户端按 pipeline 顺序调用连续 block，避免本地 RAM/SSD offloading 反复搬运完整权重
5	HexGen: Generative Inference of Large Language Model over Heterogeneous Environment [12]	ICML 2024	层级切分 / 张量切分	在异构 GPU 和异构网络上组织生成式推理	asymmetric tensor parallelism; pipeline parallelism; constrained scheduling	同时允许 pipeline stage 和 tensor parallel degree 非对称配置，让快慢 GPU、异构链路承担不同计算份额，而不是强制所有 stage 使用同构并行度
6	Resource-Efficient Collaborative Edge Transformer Inference With Hybrid Model Parallelism [23]	IEEE TMC 2025	层级切分 / 张量切分	混合并行协同推理	hybrid model parallelism; heterogeneity- and memory-aware planning; communication overlap	用 hybrid model parallelism 结合 layer/pipeline 与 tensor 并行；通过 ReduceScatter/AllGather 替代部分 AllReduce，并用通信-计算重叠降低边缘弱链路上的同步等待
7	Co-Designing Transformer Architectures for Distributed Inference With Low Communication [24]	IEEE TPDS 2025	张量/算子级切分	降低分布式 Transformer 推理通信	low-communication architecture co-design; block parallelism; two-phase planning	通过把原 Transformer layer 重构为多个 decoupled sub-block 暴露 block parallelism，减少层内强依赖和 all-reduce 次数，再用离线权重重放置与在线并行规划选择低通信执行路径

8	FlexGen: High-throughput Generative Inference of Large Language Models with a Single GPU [10]	ICML 2023	张量/算子级切分	在单 GPU 资源受限环境中服务大模型	tensor/KV/activation placement; GPU-CPU-disk offloading; linear programming	把权重、activation 和 KV cache 在 GPU、CPU、磁盘之间分层放置，用线性规划选择 offloading 和访问模式，并结合压缩扩大 batch size 与吞吐
9	E2LLM: Structure-Guided Efficient Inference for LLMs in Distributed Edge-IoT Environments [25]	IEEE TMC 2026	推理阶段切分 / 语义级协同切分	边缘结构规划与 IoT 并行解码	structure-guided planning; static-dynamic KV cache partitioning; distributed decoding	边缘节点负责结构规划和 segment-specific KV 生成，IoT 设备并行执行轻量 decoding；静态/动态 KV 分离减少重复传输，并用 response skeleton 保持片段语义一致
10	FlexPipe: Adapting Dynamic LLM Serving Through Inflight Pipeline Refactoring in Fragmented Serverless Clusters [26]	EuroSys 2026	层级切分 / 垂直切分	运行时 pipeline stage 重构	fine-grained model partitioning; inflight pipeline refactoring; topology-aware allocation	将模型预切成细粒度 stage，并在请求执行中动态调整 pipeline granularity；通过一致的 cache transition 和拓扑感知资源分配适配碎片化 GPU
11	DistServe: Disaggregating Prefill and Decoding for Goodput-optimized LLM Serving [4]	OSDI 2024	推理阶段切分	prefill/decode 物理解耦	phase-specific workers; phase-aware parallelism; KV transfer	将 compute-heavy prefill 与 memory/latency-sensitive decode 分配给不同 worker pool；分别选择并行度和 batch 策略，再通过 KV transfer 连接两阶段
12	Splitwise: Efficient Generative LLM Inference Using Phase Splitting [27]	ISCA 2024	推理阶段切分	prompt/token 阶段分离	prompt pool; token pool; hardware matching; overlapped KV transfer	将 prompt computation 和 token generation 拆到不同机器池，使两阶段匹配不同硬件；同时用 layer-wise/overlapped KV transfer 降低阶段切换的关键路径开销
13	Mooncake: Trading more storage for less computation – A KVCache-centric architecture for serving LLM chatbot [13]	USENIX FAST 2025	推理阶段切分	prefill/decode disaggregation	distributed KV cache; global scheduling; KVCache-centric architecture	在 prefill/decode 解耦基础上构建分布式 KVCache，把 CPU、DRAM、SSD、NIC 作为共享状态层；调度器按 KV 复用、本地性和 SLO 选择 prefill/decode 实例
14	SLIDE: Simultaneous Model Downloading and Inference at the Wireless Network Edge [28]	IEEE TMC 2026	推理阶段切分	下载与推理重叠	layer-wise download/inference overlap; bandwidth and compute allocation	按 layer 递归依赖组织下载和执行：用户一边接收后续 layer，一边执行已下载 layer；联合优化带宽和计算分配以隐藏模型下载等待
15	EdgeLLM: Fast On-Device LLM Inference With Speculative Decoding [29]	IEEE TMC 2025	语义/token 级协同切分	减少端侧逐 token 解码开销	on-device speculative decoding; branch navigation; self-adaptive fallback; compute-I/O pipeline	小 draft LLM 常驻内存生成候选 token，大 target LLM 批量验证；用分支导航避免把算力浪费在低概率分支，并用 compute-I/O pipeline 隐藏大模型权重加载
16	SPIN: Accelerating Large Language Model Inference with Heterogeneous Speculative Models [30]	IEEE INFOCOM 2025	语义/token 级协同切分	异构草稿模型选择与验证流水线	heterogeneous SSM selection; request decomposition; speculation/verification pipelining	为不同难度请求选择合适的异构 speculative model；把长请求拆分以降低 verification batch padding，并流水化 speculation 与 verification 阶段
17	SpecInfer: Accelerating Large Language Model Serving with Tree-based Speculative Inference and Verification [31]	ASPLOS 2024	语义/token 级协同切分	token tree 草稿与批量验证	tree-based speculative inference; parallel verification	多个小模型生成候选 token tree，目标 LLM 不再逐 token 增量生成，而是作为 verifier 一次并行校验多条候选路径，从而减少目标模型调用次数

18	SwapMoE: Serving Off-the-shelf MoE-based Large Language Models with Tunable Memory Budget [21]	ACL 2024	专家/组件级切分	专家驻留可调	virtual experts; expert swapping; tunable memory budget	只把动态选择的重要专家作为 virtual experts 常驻主存，其余专家按需映射和换入，在不剪枝模型结构的情况下减少 MoE 推理内存占用和换入开销
19	Taming Latency-Memory Trade-Off in MoE-Based LLM Serving via Fine-Grained Expert Offloading [32]	EuroSys 2026	专家/组件级切分	细粒度 expert offloading	fine-grained expert offloading; latency-memory trade-off	以 expert 为换入/卸载单元，在显存预算下决定哪些专家保留在 GPU、哪些下沉到主机或存储层，从而在显存节省和专家加载延迟之间折中
20	MoE-Infinity: Efficient MoE Inference on Personal Machines with Sparsity-Aware Expert Cache [33]	ICML 2025	专家/组件级切分	稀疏感知 expert cache	sparsity-aware expert cache; expert prefetching; memory hierarchy	利用 MoE token 路由稀疏性和专家访问局部性，只缓存即将使用或高频专家，并通过预取和分层内存降低专家缺页造成的 decode 停顿
21	Pre-gated MoE: An Algorithm-System Co-Design for Fast and Scalable MoE Inference [34]	ISCA 2024	专家/组件级切分	提前 expert routing 与系统协同	pre-gating; expert scheduling; algorithm-system co-design	在系统执行前更早获得 expert 选择信息，使 expert 加载、通信和调度可以提前安排，减少 MoE 推理中的路由等待和跨设备同步
22	EdgeMoE: Empowering Sparse Large Language Models on Mobile Devices [35]	IEEE TMC 2025	专家/组件级切分	移动端稀疏 LLM 部署	sparse MoE deployment; expert management; on-device inference	面向移动设备资源约束管理 MoE expert 的驻留与执行，使稀疏大模型能够在端侧或近端资源上以可控内存开销运行
23	WDMoE: Wireless Distributed Mixture of Experts for Large Language Models [36]	IEEE TWC 2025	专家/组件级切分	无线边缘分布式 MoE	wireless distributed experts; expert selection; bandwidth allocation	将 gating/attention 和 experts 分布在基站与移动设备之间，根据 expert 权重、设备延迟和无线带宽联合选择专家与分配通信资源
24	P4LLM: Protecting Embedding Privacy against Model Inversion Attacks for EaaS LLM Serving via Token-wise Partition and Perturbation [37]	IEEE TMC 2026	语义/token 级协同切分	token-wise partition + perturbation	insensitive token filtering; partitioning scheduler; hidden-state perturbation	在用户侧按 token 切分 Transformer 执行，只对敏感 token 的 hidden state 做扰动并上传，非敏感 token 过滤或轻处理，从而减少扰动计算和传输负担

4.1 Layer-wise / Vertical Partitioning

层级切分是最接近传统意义上“模型分割”的一类方法。它把 Transformer 视为由多个顺序 block 组成的计算链，在 layer 或 block 边界上拆分模型，并将不同模型段放置到端侧、边缘或云端节点执行。对于无法完整加载模型权重、activation 和运行时状态的端侧或边缘设备，层级切分提供了一种直接的扩展方式：单个节点不再承担完整模型，而是多个节点共同完成同一次 forward。Petals 展示去中心化 block 放置 [11]，EdgeShard 面向边缘 shard placement [22]，HexGen 讨论异构生成式推理 [12]，FlexPipe 则关注 serverless 集群中的 pipeline 重构 [26]；这些工作共同验证了沿深度方向拆分模型的价值。

为了把这种“沿深度方向展开”的执行方式与普通请求路由区分开，可以参考图 7。示意图中横向排列的 Transformer layers 表示同一个模型在深度方向上的执行链，虚线边界表示可选 split point。Client、Edge 和 Cloud 并不对应固定层数，而是代表不同资源层级；层间箭头表示 activation 或 hidden

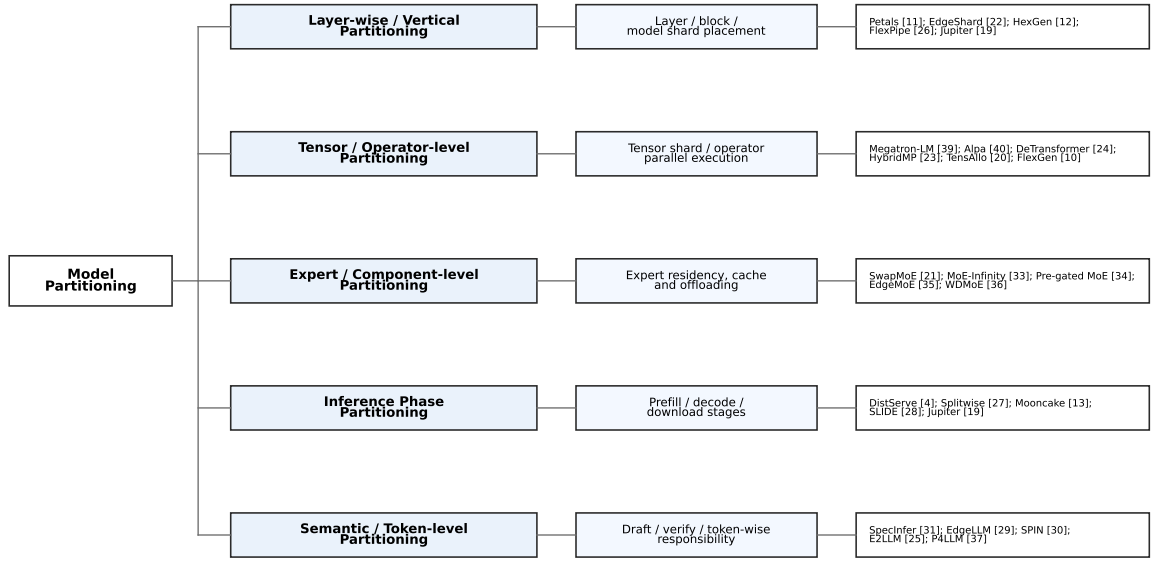


图 6 CEC 下资源自适应模型切分部署方法的分类树

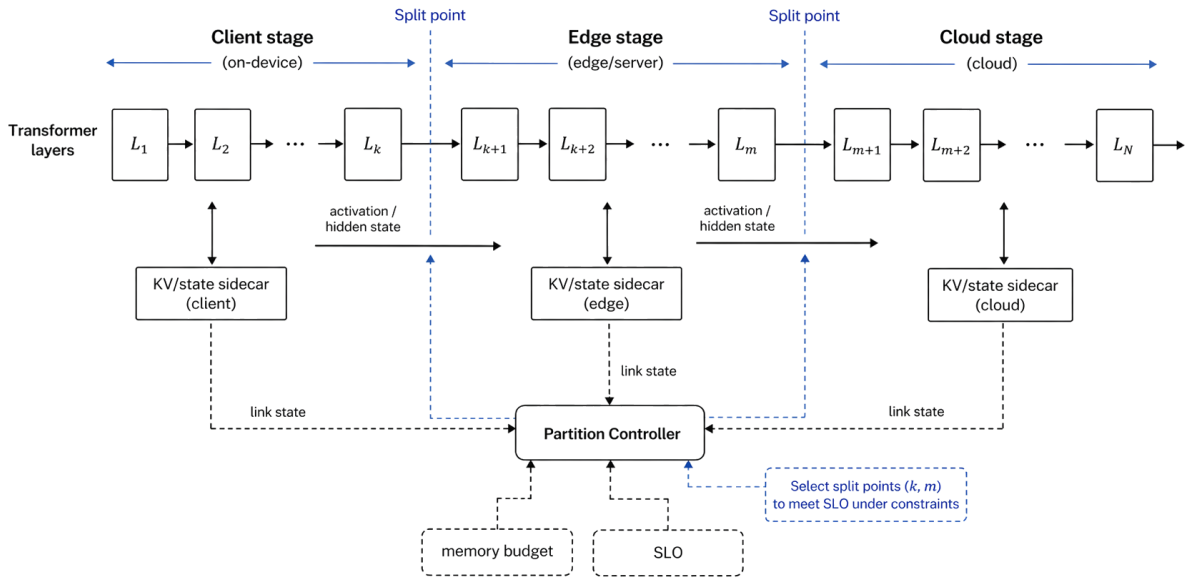


图 7 Layer-wise / Vertical Partitioning 机制示意图

state 的跨节点传输，KV/state sidecar 表示推理过程中需要伴随模型段管理的运行时状态。Partition Controller 根据链路状态、显存余量、节点负载和 SLO 选择 layer placement、pipeline stage 与 fallback path。因此，层级切分的关键不是平均切开模型，而是在计算节省和跨节点状态传输之间动态权衡。

这一类方法的核心不是“把模型平均切开”，而是寻找合适的 split point 和 placement。切分点靠前，可以减轻端侧计算和内存压力，但会更早产生跨节点 hidden state 或 activation 传输；切分点靠后，可以降低网络依赖，却要求端侧执行更多 layer。对于长上下文请求，activation 规模和 KV 状态增长会进一步放大通信成本；对于短请求，通信启动时延可能抵消计算节省。因此，层级切分通常需要同时估计每层计算量、activation 大小、显存占用、链路带宽、RTT、节点负载和功耗约束。

CEC 环境使层级切分比云端 pipeline parallelism 更难。数据中心内的互联带宽相对稳定，而端-边、边-边、边-云链路可能受到无线抖动、跨域转发和拥塞影响。一个在静态 profiling 中最优的切点，到了移动用户、突发请求或边缘节点过载时可能迅速失效。因此，资源自适应的层级切分往往需要三类能力：离线阶段建立 layer-level profile，在线阶段根据节点和链路状态选择切点，运行时阶段在负载变化时进行有限频率的重配置。GPipe 体现了 pipeline parallelism 在稳定集群中的基本价值 [38]，Megatron-LM 展示了大模型 tensor parallelism 的基础路线 [39]，Alpa 则进一步自动化 inter-operator 与 intra-operator parallelism [40]。CEC 需要在这些并行思想之上继续纳入异构链路和在线状态。

这类方法还要处理流水线稳定性。切分后，每个模型段的执行时间如果差异过大，较慢节点会成为 pipeline bottleneck；中间节点失败或链路恶化时，后续 layer 是否迁移、是否重新选择切点、是否回退到云端完整执行，都会影响服务连续性。对于 CEC LLM 推理，层级切分不能只作为部署算法讨论，还必须和 admission control、状态管理和异常回退衔接。

总体来看，层级切分适合模型容量和单节点部署能力不足的场景，尤其适用于模型段执行时间差异明显、节点间链路相对稳定、跨节点 activation 可控的系统。它的主要风险在于切分收益被网络传输和流水线等待抵消。因此，CEC 中的层级切分更适合采用“少量稳定切点 + 在线代价修正”的方式，而不是频繁改变模型执行拓扑。

这里的“少量稳定切点”并不意味着静态平均切分，而是指将在线搜索限制在经过 profiling 验证、可部署且不会 OOM 的候选切点集合内，避免运行时任意切分造成模型重加载和状态迁移震荡。

4.2 Tensor / Operator-level Partitioning

当层级切分仍不足以缓解单层计算和显存压力时，系统需要进入更细粒度的张量/算子级切分。LLM 的每一层内部包含 attention、QKV projection、MLP、归一化和通信同步等多个计算单元，其中 attention head、矩阵乘和中间 activation 都可能成为单设备瓶颈。张量/算子级切分不再以完整 block 为边界，而是在同一层内部拆分 tensor、head、矩阵维度或 operator 执行路径。Megatron-LM 和 Alpa 代表了数

据中心大模型中 tensor / intra-operator parallelism 的基础路线 [39], [40]; DeTransformer 通过结构协同设计降低分布式推理通信 [24], Galaxy+ 采用混合模型并行组织边缘 Transformer 推理 [23], TensAllo 则把异构边缘设备选择和 tensor allocation 纳入部署决策 [20]。

层内拆分的系统关系可以用图 8 概括，它突出的是同一 Transformer 层内部的并行和同步，而不是层与层之间的顺序切点。图中没有展开每个 GEMM 或 attention kernel 的细节，而是保留张量/算子

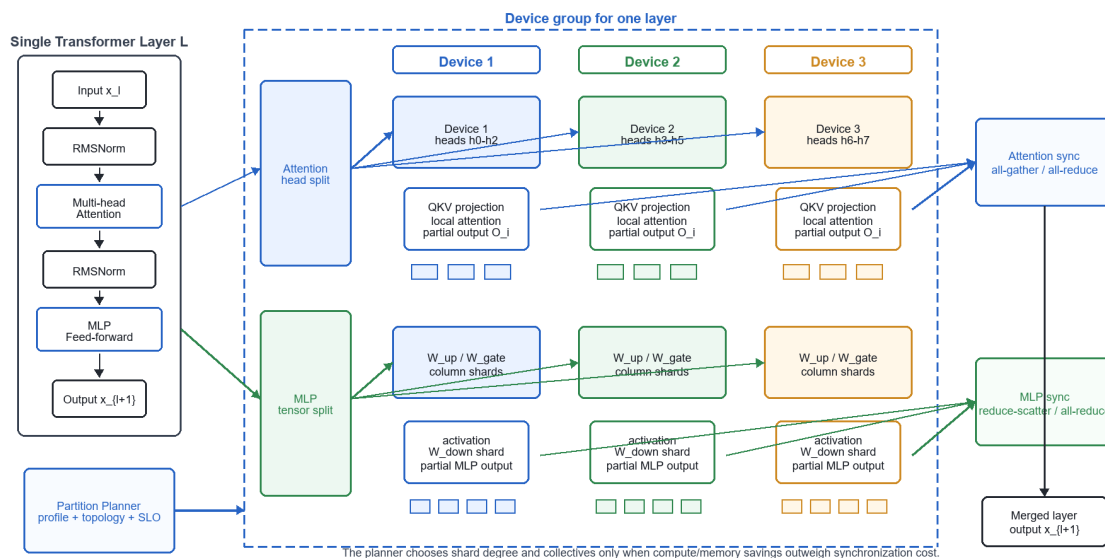


图 8 Tensor / Operator-level Partitioning 机制示意图

级切分最关键的执行关系：左侧是单个 Transformer layer，中间的 Partition Planner 根据 layer profile、硬件拓扑、显存预算和 SLO 判断是否需要层内并行；右侧的 Device Group 表示同一层被分配到多个 Ascend 设备上执行。Attention 部分可以按 head 或 QKV shard 拆分，MLP 部分可以按矩阵维度拆分；各设备得到局部结果后，再通过 all-gather、reduce-scatter 或 all-reduce 完成同步合并。这里的关键不是“多设备一定更快”，而是并行度增加后同步次数和链路依赖也会增加；在 CEC 中，边缘链路的带宽和抖动会直接决定水平切分是否有收益。

这类方法通常保持模型数学语义不变，改变的是层内计算如何分布到多个设备。它可以表现为 tensor parallelism、hybrid model parallelism、block parallelism、低通信 Transformer 结构或 structure-guided partitioning。与层级切分相比，它能更细地利用碎片化资源；与单机 kernel 优化相比，它必须显式处理跨设备通信、同步和拓扑选择。

本章将 Tensor / Operator-level Partitioning 视为比普通 kernel 优化更高一层的系统问题。若某项工作只优化单设备 GEMM/attention kernel，而不改变层内计算的跨设备分布，则不归入本类。

水平切分的核心矛盾是并行收益与通信惩罚。切得越细，单个设备承载的权重、activation 和中间矩阵越少，也更容易把多个弱设备组织起来；但 all-reduce、all-gather、reduce-scatter 或点对点传输

会更频繁。切得越粗，通信开销下降，却可能重新遇到显存或算力瓶颈。数据中心可以依赖高速互联吸收一部分同步成本，CEC 中的边缘设备却往往只有普通以太网、无线链路或多跳边缘互联，过细的 tensor parallelism 很容易变成通信密集型执行。

因此，CEC 下的张量/算子级切分需要比云端并行更强的资源画像。调度器要知道哪些设备适合低比特 GEMM，哪些设备适合 attention，哪些设备只能承担较小 tensor shard；还要估计层内同步频率、通信 primitive 的代价和 batch size 对算子效率的影响。某些工作还通过模型结构 co-design 减少层内依赖，使模型天然更适合低通信分布式推理。这说明水平切分不是单纯调大 parallel degree，而是模型结构、设备选择和通信路径的联合设计。

张量/算子级切分适合单层计算或显存压力成为瓶颈的场景，但在 CEC 中不能简单照搬数据中心 tensor parallelism。边缘链路的带宽、抖动和同步延迟会直接决定并行收益是否存在。因此，这类方法应优先与低通信结构、异构设备选择和静态/动态 profiling 结合，并尽量避免让每个 token 都经过高频跨节点同步 [20], [23], [24]。

4.3 Expert / Component-level Partitioning

专家/组件级切分主要面向 MoE 等具有显式内部组件选择机制的模型。与普通 dense Transformer 不同，MoE 模型通常在 FFN 部分包含大量 expert，而每个 token 只激活其中少数 expert。这个结构使模型天然具备“组件级可调度性”：系统不一定让所有 expert 同时常驻同一设备，而可以根据 expert 热度、token routing 分布、显存预算、链路状态和请求类型，决定哪些 expert 常驻、哪些按需换入、哪些放在边缘或云端节点上。SwapMoE 从 expert swapping 角度降低显存压力 [21]，MoE-Infinity 利用 sparsity-aware expert cache 支撑个人设备上的 MoE 推理 [33]，Pre-gated MoE 通过提前 expert routing 协同系统调度 [34]，EdgeMoE 面向移动端稀疏模型部署 [35]，WDMoE 则讨论无线分布式 expert 放置和带宽分配 [36]。这些工作共同说明，组件级切分已经成为 MoE 推理系统的重要组织方式。

MoE 场景下，组件驻留和 token routing 的关系更适合从图 9 这样的分层缓存视角理解。在这个机制中，MoE Router 为每个 token 选择 top-k experts，Expert Placement Policy 再根据 expert hotness、memory budget 和 link state 决定 expert 的驻留层级。热 expert 可以放入 Edge Expert Cache，次热 expert 放在 Regional Expert Pool，低频或完整 expert 集合保存在 Cloud / SSD Expert Store。箭头表示 token-expert 访问路径和 expert swap/prefetch 路径。组件级切分的收益来自稀疏激活和访问局部性，风险则来自 expert 预测错误后产生的远程访问或换入等待。

这一类方法之所以不直接并入张量/算子级切分，是因为它切的不是矩阵维度或 communication primitive，而是具有模型语义的组件单元。一个 expert 往往对应一组完整参数和计算路径，其是否被激活取决于 token-level routing；系统优化的对象也从“如何并行执行同一层矩阵”转向“如何管理大量稀疏激活组件的驻留、映射和调度”。因此，专家/组件级切分介于层内切分和语义/token 协同之

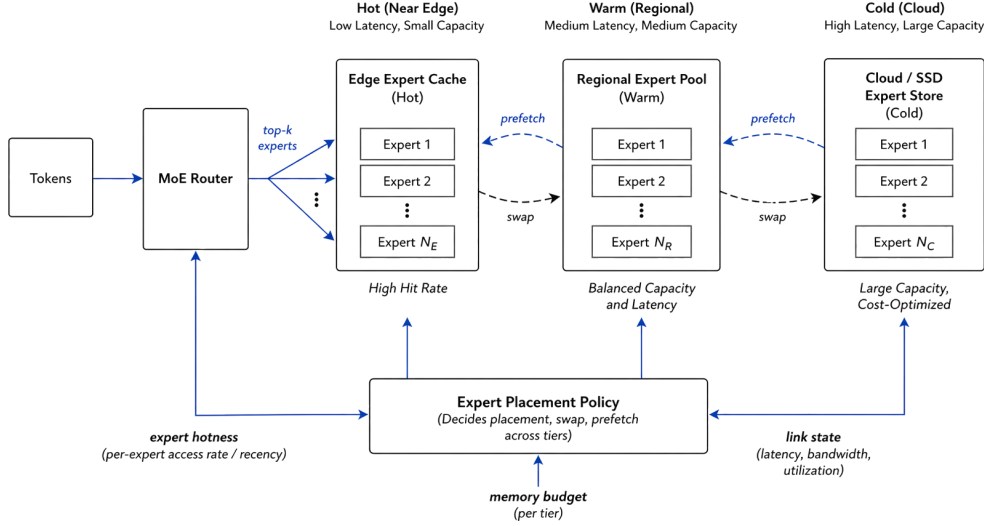


图9 Expert / Component-level Partitioning 机制示意图

间：它发生在模型内部组件层面，但收益取决于 token 分布和专家访问模式。

在 CEC 场景下，专家/组件级切分的价值在于降低大模型组件的常驻压力。端侧或接入边缘难以保存完整 MoE 模型，但可能保存高频 expert、局部业务相关 expert 或经过压缩的 expert cache；区域边缘可以维护更大的 expert pool；云端则保留完整 expert 集合并处理低频或复杂请求。系统需要在 expert 命中率、换入换出成本、跨节点 expert 访问时延和输出质量之间做权衡。如果 expert 热度预测准确，组件级切分可以减少显存占用并提高边缘可部署性；如果预测失误，频繁 swap 或远程访问 expert 会直接破坏 decode 时延。

专家/组件级切分的决策变量包括 expert residency、expert cache size、expert hotness prediction、token routing statistics、swap bandwidth、memory budget、fallback policy 和跨层级 expert placement。与层级切分相比，它更细粒度；与张量切分相比，它更依赖模型稀疏激活；与语义/token 级协同相比，它仍然保持同一个 MoE 模型内部的 expert 语义一致性，而不是把 draft、verify 或语义段分给不同模型完成。

总体来看，专家/组件级切分适合 MoE 模型、expert 热度具有局部性、边缘节点无法常驻完整 expert 集合的场景。它不应被简单理解为普通 request routing，因为请求仍在同一个 MoE 模型语义下执行；也不应被完全归入 token-level collaboration，因为系统主要调度的是 expert 组件而非生成职责。现有代表工作已经覆盖 expert swapping、fine-grained expert offloading、expert cache、pre-gating 和 wireless distributed experts 等不同角度；CEC 场景下仍然缺少把用户移动、区域业务热度、expert cache 迁移和无线带宽联合起来建模的工作 [32], [36]。

若目标模型是 dense LLM，本类方法一般不应作为模型切分部署的主路径；若目标模型或后续业务引入 MoE、专家模型或多 adapter 组件，本类才会成为核心部署问题。

4.4 Inference Phase Partitioning

LLM 推理不是单一形态的计算。Prefill 需要一次性处理完整 prompt，矩阵规模大、并行度高，通常更偏 compute-bound；decode 逐 token 生成，频繁访问 KV Cache，通常更偏 memory-bound 或 latency-bound；speculative decoding 中的 verification 又会形成目标模型批量校验；在无线边缘场景中，模型下载、加载和推理还可能同时出现在关键路径上。推理阶段切分正是利用这些阶段资源画像的差异，把不同阶段放到更合适的节点或资源池上执行。DistServe [4]、Splitwise [27] 和 Mooncake [13] 是 prefill/decode disaggregation 的代表；Jupiter 从边缘协作角度组织 prefill/decode 执行 [19]，E2LLM 从 Edge-IoT 结构化生成角度扩展阶段分工 [25]，SLIDE 则讨论无线边缘中的下载-推理重叠 [28]。

阶段级切分更适合放在推理流水线视角下理解，图 10 给出了 prefill、decode、verification 与 KV/state handoff 的典型组织方式。这里的 Stage Scheduler 根据 prompt length、SLO、link state 和 KV

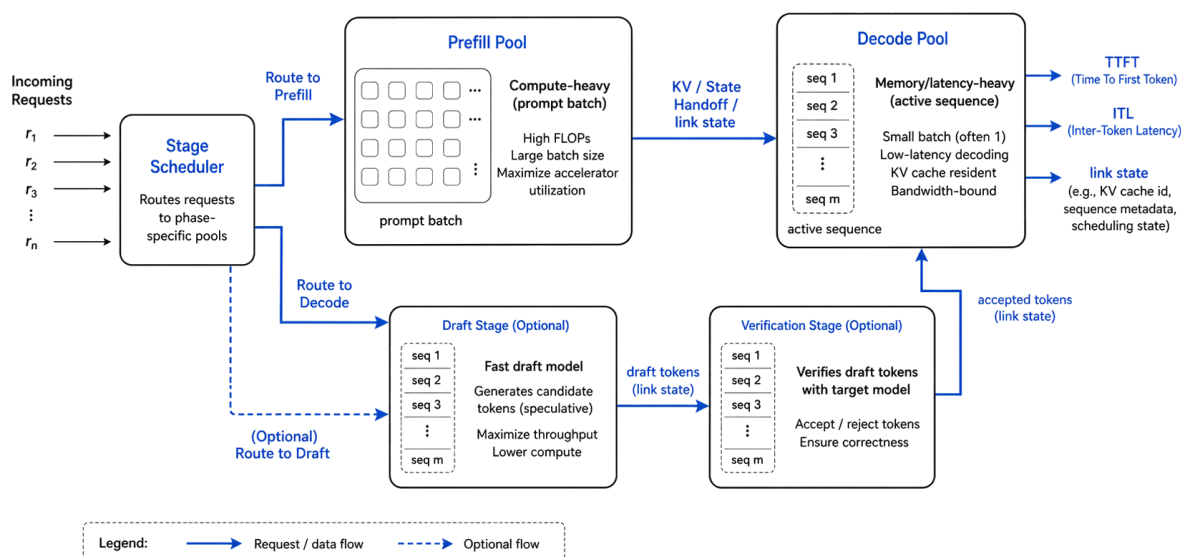


图 10 Inference Phase Partitioning 机制示意图

locality 把请求送入不同阶段资源池。Prefill Pool 处理 compute-bound 的 prompt batch，Decode Pool 维护 memory/latency-bound 的 active sequence，KV / State Handoff 负责在阶段之间传递 KV blocks、metadata 和必要状态。若系统引入 verification 或 draft stage，它们也可以作为独立阶段被调度。阶段切分的核心收益来自资源画像匹配和阶段干扰隔离，核心代价则集中在 KV/state 传输与阶段队列协调。

最典型的形式是 prefill/decode disaggregation。长 prompt 的 prefill 可以放在算力更强、吞吐更高的区域边缘或云端实例，低时延 decode 则由更靠近用户的接入边缘承担。这样做可以减少长 prefill 对 decode 的阻塞，也能让不同硬件分别服务 compute-heavy 和 memory-heavy 阶段。进一步地，系统还可以把 draft generation 和 target verification 分开，或将模型下载与推理执行重叠，从而缩短冷启动

和模型切换时延。

阶段切分的决策变量包括 prefill worker、decode worker、stage queue、pipeline boundary、KV transfer、verification placement、download/inference overlap 和阶段间同步机制。它的收益来自资源匹配和干扰隔离，但代价集中在状态交接。Prefill 和 decode 共享 KV Cache，draft 和 verify 共享候选 token 与概率校验状态，下载与推理重叠则会竞争网络、存储和内存带宽。换言之，阶段切分能否有效，取决于拆开之后能否低成本地把状态交给下一阶段 [4], [27], [13]。

在 CEC 中，阶段切分具有明显的场景价值。接入边缘适合承担低时延 decode，区域边缘适合聚合多个入口的 prefill-heavy 请求，云端适合长上下文、大模型 verification 或复杂推理。但这种分工不是无条件成立：如果 prefill 在云端完成后需要把大量 KV 传回边缘，链路抖动就会直接放大 TTFT 或后续 TPOT；如果边缘并发不足，独立 decode worker 可能无法形成有效 batch；如果用户移动导致入口切换，阶段边界还可能触发状态迁移。因此，阶段切分实际是在阶段资源匹配和状态连续性之间做权衡。

推理阶段切分适合 prefill/decode 负载差异明显、阶段队列规模足够、状态传输可控的场景。若请求很短、边缘并发很低或链路抖动大，复杂阶段拆分可能不如本地顺序执行稳定。因此，阶段切分必须和 KV 状态管理、链路预测、admission control 和回退策略一起设计；它不是独立的部署技巧，而是贯穿推理流水线的资源组织方式。

阶段切分与第五章的 Disaggregated Prefill-Decode Batching 视角不同。第四章关注 prefill/decode 是否跨实例或跨层级部署；第五章关注在已经分离的 prefill pool 与 decode pool 内部如何分别组批、排队和控制参数。

4.5 Semantic / Token-level Partitioning

语义/token 级协同切分与前四类不同。它不一定把同一个模型的 layer、tensor 或 expert 组件拆开，而是把一次生成过程中的职责拆开：草稿由小模型产生，验证由大模型完成；长回答可以按语义段并行生成；隐私敏感 token 或 embedding 可以被单独 partition 和 perturbation。这里“切分”的对象是生成语义和 token 处理职责，而不是模型内部参数组件。SpecInfer 展示 token tree verification [31]，EdgeLLM 讨论端侧 speculative decoding [29]，SPIN 关注异构草稿模型选择 [30]，E2LLM 则展示语义段级并行生成 [25]。

生成职责级方法的结构可参考图 11。它强调的不是参数如何分片，而是 draft、verify、refine 和 privacy filtering 等职责如何分配。从系统路径看，Semantic Planner 根据 task type、confidence 和 privacy tag 判断每段生成职责交给哪个模块。Local Small Model 负责生成 draft tokens，Edge Model 可以承担部分 semantic segment generation，Privacy Guard 对敏感 token 做 partition 或 perturbation，Cloud Large Model 负责 verify/refine。Acceptance Check 决定草稿是否接受、是否回退到大模型重算。token/语义

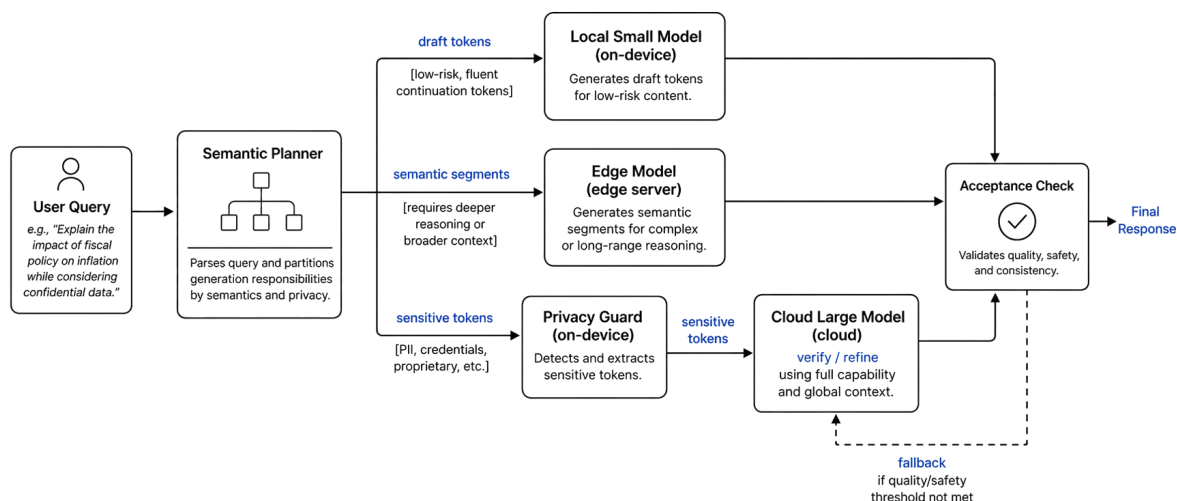


图 11 Semantic / Token-level Partitioning 机制示意图

级方法的关键约束是质量一致性和错误回退，而不仅是加速。

这类方法的出发点，是自回归生成并不必然只能由一个模型连续完成。系统可以在端侧先生成 draft，再由边缘或云端目标模型 verify；可以先生成 response skeleton，再把不同语义段交给多个设备并行生成；也可以将隐私敏感 token 从普通 token 流中分离出来，采用额外扰动或分区策略。与层级切分和张量切分相比，它改变的是生成过程中的职责边界，而不是纯粹的模型拓扑 [29], [37]。

语义/token 级协同的核心变量包括 draft length、acceptance rate、verification frequency、semantic segment boundary、token partition、privacy budget、fallback condition 和输出质量约束。它的收益通常来自减少大模型调用、并行化长回答生成或细粒度保护敏感信息；但风险也更直接地体现在输出质量上。draft 接受率过低会浪费验证计算，语义段边界选择不当会破坏上下文连贯性，过强的 token perturbation 可能损害语义一致性。

在 CEC 中，这类方法的意义在于把“端侧小模型、边缘模型、云端大模型协同”从请求级选择推进到生成级协作。端侧可以承担低成本 draft 或局部补全，边缘可完成常见语义验证或中等复杂度生成，云端大模型只在关键语义点介入。这样不仅有机会降低云端调用成本，也能改善交互时延和隐私边界。不过，这类方法对在线质量估计和回退机制要求最高，不能只依据资源指标判断收益。

语义/token 级协同切分适合端侧小模型可用、任务分布相对稳定、局部生成质量可被估计的场景。它的风险也最明显：低接受率、语义段不连贯或隐私扰动过强都会破坏质量。因此，这类方法需要和质量评估、置信度估计、安全回退以及外层服务策略联合设计，不能只用系统吞吐指标来评价。

本类中的 speculative decoding 工作不等价于“模型切分”本体；本章保留它们，是因为它们切分了 draft、verify、token tree 或语义段等生成职责，属于生成过程级的协同执行。

4.6 面向华为验证环境的方案建议

面向异构边缘验证环境，资源自适应模型切分部署首先要回答的是一个具体问题：在当前 Ascend 设备拓扑、显存水位、链路带宽和业务 SLA 下，模型应该在哪些 layer/block 之间切开，每一段应该放到哪个设备组上执行。因而推荐方案应以层级切分为主线，先让 Ascend GPU/NPU 推理栈支持连续 layer/block 形成跨卡、跨节点 stage 的执行，再用动态规划生成 stage boundary 和 stage placement 策略。TP、PP、EP、DP replica、KV migration 或 PD 分离等能力不作为人工挑选的“大方法”，而是在 runtime 支持且资源检查通过时被用于降低某个 stage 的显存、时延或吞吐压力。该思路与 EdgeShard 将 model shard placement 转化为动态规划搜索的思想一致，但落点更贴近华为验证环境中的层级切分执行能力建设 [22]。

具体而言，系统先采集三类画像。第一是设备资源与能力画像：根据硬件环境信息构建算力拓扑图 $H = (V, E)$ ，节点记录 Ascend 卡或 vNPU 的可用算力、可用显存、显存带宽、当前负载和 runtime capability，边记录 HCCS、RDMA、以太网或跨主机链路的带宽、时延和抖动。第二是模型画像：解析 PyTorch 或 HuggingFace 模型的 config、权重和模块结构，得到 layer 数、每层参数量、activation 峰值、单 token KV Cache 增量、prefill/decode 计算量以及是否包含 MoE expert 或可拆分子件。第三是业务画像：从配置中读取目标并发、输入输出长度分布、P90/P99 TTFT、TPOT、E2E latency、SLA 等级、请求优先级以及是否允许 KV 迁移、PD 分离或降级回退。这些画像共同决定每个连续 layer 区间在不同设备组上的执行代价、显存风险和链路代价。

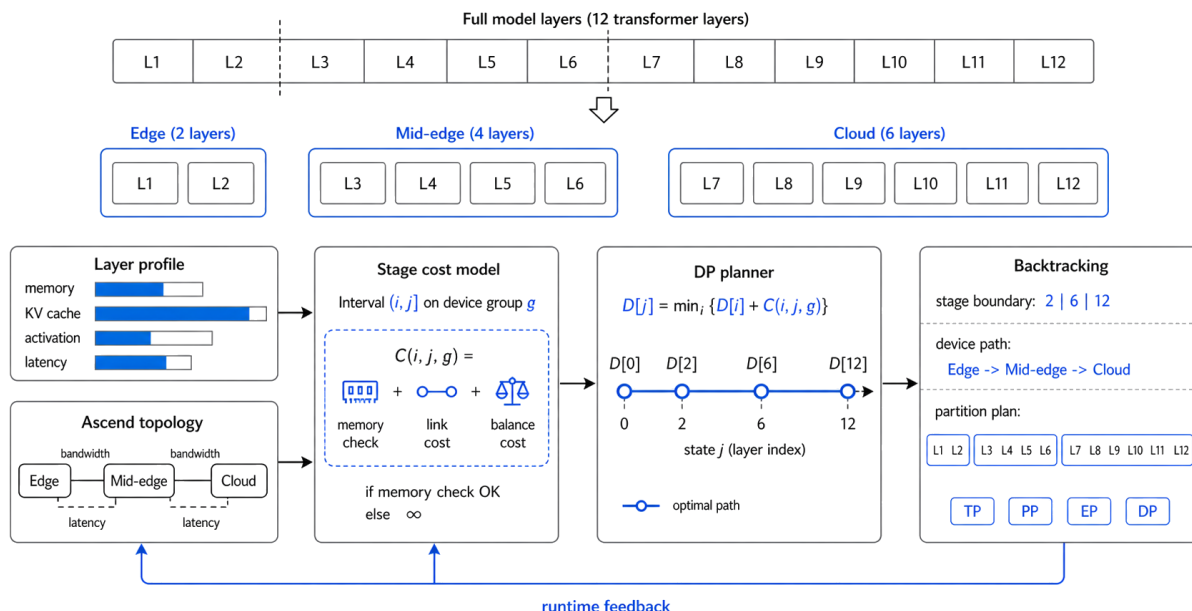


图 12 基于动态规划的层级切分部署策略生成流程

如图 12 所示，层级切分策略由 layer profile、Ascend topology、stage cost model 和 DP planner 共

同生成。为了避免把“切分方案”写成一个过于抽象的黑盒，第四章的决策变量可以明确写成

$$x = \{K, \{l_k, g_k, e_k\}_{k=1}^K\}.$$

其中， K 表示模型被切成多少个连续 stage； l_k 表示第 k 个 stage 的右边界，因此第 k 个 stage 覆盖 layer 区间 $(l_{k-1}, l_k]$ ，且 $l_0 = 0, l_K = N$ ； g_k 表示该 stage 放置到哪个 Ascend 设备组或节点组； e_k 表示该 stage 在当前 runtime 已支持范围内启用的执行能力，例如单设备执行、TP、PP+TP、EP、KV/offload 或阶段间状态传输。这样，第四章真正要决策的是“切在哪里、每段放哪里、每段用什么已支持执行能力”，而不是在几类并行方法之间做人工选择。

在论文建模上，可以把问题写成从安全可部署取值空间中选择低代价层级切分方案：

$$x^* = \arg \min_{x \in \mathcal{X}_{\text{safe}}} J(x),$$

$$J(x) = w_t T(x) + w_c C(x) + w_m M(x) + w_b B(x) - w_g G(x).$$

其中， $\mathcal{X}_{\text{safe}}$ 表示经过 runtime 支持检查、资源检查和 SLA 检查后的安全取值空间。 $T(x)$ 表示 TTFT、TPOT、E2E latency 和 pipeline bubble 等时延代价； $C(x)$ 表示 activation/hidden state、KV 或 stage 交接带来的通信代价； $M(x)$ 表示显存水位、runtime reserve 和 OOM 风险； $B(x)$ 表示 stage 执行时间不均衡造成的流水线阻塞； $G(x)$ 表示满足 SLA 的有效吞吐收益。这个目标函数不需要展开到每个 kernel，而是服务于验证环境中的部署决策：在不违反显存、链路和 SLA 约束的前提下，选择总代价更低、有效吞吐更高的层级切分方案。

这些决策变量的取值由系统自动生成。首先，规划器根据模型 layer 数和 layer profile 生成所有连续区间 $(i, j]$ ，并用前缀和快速得到该区间的权重、activation 峰值、KV 增量和 prefill/decode 计算量。其次，根据 Ascend topology 生成可承载 stage 的设备组 g ：单卡、同机多卡、跨节点设备组或端-边-云层级组合都要带有可用算力、可用显存、链路带宽和 RTT 信息。再次，runtime capability 决定 e_k 的取值：如果连续 layer stage 加载和跨设备 hidden state 传输可用，则允许层级 stage；如果 TP collective 可用且链路代价可接受，则允许该 stage 使用 TP；如果模型包含 MoE expert 且 runtime 支持 expert 驻留或换入，则允许 EP；如果 KV/offload 或阶段间状态传输已经被 runtime 支撑，也可以作为该 stage 的执行能力进入评估。未通过能力检查或资源检查的取值不会进入后续 DP 递推。

动态规划负责在这些可执行取值中生成最终层级切分策略。规划器先计算把连续区间 $(i, j]$ 放到设备组 g 上的 stage cost $C(i, j, g)$ 。这里的 $C(i, j, g)$ 与图 12 中的写法一致，是一个经过能力检查后的综合代价：它内部会在 $\mathcal{E}_{\text{ok}}(i, j, g)$ 中选择可用执行方式 e ，并同时计入执行时间、显存水位、同步/传输代价和 pipeline balance。图中 Edge、Mid-edge 和 Cloud 承载不同数量的连续 layer，表示 DP 输出的不是平均切分，而是受设备算力、显存和链路代价共同约束的不均匀层级分配。若某个区间超过

显存预算、链路预算或 SLA 风险阈值，则该取值直接被丢弃。对保留下来的可执行取值，DP 按 layer 顺序递推，记录到达每个 block 位置的最小代价和前驱切点。若记 $D[j]$ 为完成前 j 个 block 的最小综合代价，则可简化写成

$$D[j] = \min_{i < j, g \in \mathcal{G}_{ok}} \{D[i] + C(i, j, g)\},$$

其中， $\mathcal{G}_{ok}$ 表示当前可用于承载 stage 的设备组。若需要展开 $C(i, j, g)$ ，它可以理解为 profiling 得到的 stage 执行代价、由 activation/hidden state 大小和链路状态估计的传输代价、显存检查代价以及 pipeline balance 惩罚的加权和； $\mathcal{E}_{ok}(i, j, g)$ 表示该区间在该设备组上已通过能力和资源检查的执行方式。递推完成后，系统从最后一个 block 反向回溯前驱切点，得到完整的 stage boundary 序列；再把每个 stage 对应的设备组和执行能力写入部署计划。这样，DP 不是抽象地给出一个分数最低的方案，而是实际生成“第几层到第几层放在哪个设备组、每段采用什么已支持执行方式、失败时回退到哪条路径”的部署策略。

在线阶段不应频繁重切模型。系统可以在 P99 TTFT 超标、显存水位接近阈值、链路退化、请求长度分布明显变化或 runtime capability 发生变化时触发重规划；新方案只有在预估收益超过阈值且安全检查通过后才执行，否则保持当前部署或回退到固定 PP/TP、单节点部署、平均层级切分等基线。最终的分割策略可按“先层级、再并行、后扩展”的顺序组织：先用动态规划确定 layer-wise stage boundary、stage placement 和 pipeline balance；再在具体 stage 上按 runtime 支持情况启用 TP、EP、DP replica 或 KV 迁移等能力；最后把 PD disaggregation、KV Cache Pool、semantic/token-level collaboration 等作为扩展能力纳入验证。这样既避免把方案写成空泛的人工选择，也能把第四章落到可测试、可对比、可部署的华为验证路径上。

4.7 小结

本章围绕“一次推理中首先被切分的对象”组织 CEC 下的资源自适应模型切分部署方法。Layer-wise / Vertical Partitioning 关注 layer、block 或 model shard 如何跨节点展开；Tensor / Operator-level Partitioning 关注同一层内部的 tensor、head、MLP 或通信 primitive 如何并行；Expert / Component-level Partitioning 关注 MoE expert 等组件如何驻留、换入和跨层级访问；Inference Phase Partitioning 关注 prefill、decode、verification、download 等阶段如何匹配异构资源；Semantic / Token-level Partitioning 则关注 draft、verify、semantic segment 或 token-wise responsibility 等生成职责如何拆分。这个分类强调的是系统优化对象，而不是论文使用的术语本身。

这五类方法之间存在明显的互补关系。层级切分和张量/算子级切分主要解决模型容量、显存和并行度问题；专家/组件级切分更适合 MoE 等具有稀疏激活结构的模型；阶段切分用于缓解 prefill 与 decode 之间的资源画像差异；语义/token 级切分则把协同推理推进到生成过程内部。CEC 场景下的

关键不在于选择某一种“万能切分”，而在于根据模型结构、节点能力、链路状态、KV 位置、业务 SLA 和隐私边界选择合适的切分粒度。

从工程落地看，资源自适应切分不宜完全依赖一次性离线规划。离线 profiling 可以建立 layer cost、tensor cost、activation size、KV 增长和设备吞吐基线，并与 runtime capability 一起形成能力门控和代价估计；在线阶段仍需要根据负载、链路和请求长度修正这些代价。更稳妥的路线是“离线 profiling + 能力门控 + 事件触发动态规划重算 + 安全回退”：先判断 Ascend 推理栈在当前硬件拓扑上支持哪些切分和并行能力，再由动态规划搜索满足约束的低成本层级切分方案，最后通过收益阈值、冷却时间和回退路径避免频繁重配置。

结合本报告面向的验证环境，第四章的主线应放在以 Layer-wise / Vertical Partitioning 为基础的能力门控动态规划算法上。Layer-wise stage boundary、stage placement 和 pipeline balance 是主决策；tensor/operator-level、phase partitioning 和 expert/component-level 等方法不再作为人工挑选的“大方法”，而是映射到 vLLM-Ascend / MindIE 或 CANN 生态可支持、可压测、可回退的执行能力，例如 TP、PP、PP+TP、EP、layer sharding、context parallel、PD 分离、KV pool/offload 和 speculative decoding。Expert/component-level 与 semantic/token-level 方法适合作为扩展能力：前者用于 MoE 或组件化模型，后者适合端侧小模型可用且质量回退机制成熟的场景。这样既能覆盖研究前沿，也能将综述结论自然转化为可测试、可对比、可部署的算法方案。

五、CEC 下的业务感知 Batching 参数优化方法

LLM serving 中的 batching 不再只是传统 DNN 推理里的固定 batch size 选择。一次 LLM 请求通常先经历 prefill，再进入逐 token decode；请求到达时间、prompt 长度、输出长度、SLO、KV Cache 占用和上下文复用机会都会随时间变化。因此，batching 的核心问题可以定义为：在给定请求队列、实例负载、KV 状态和端-边-云链路条件下，动态决定哪些请求进入 batch、batch 在什么粒度更新、prefill 与 decode 是否混合、长 prompt 是否切块、不同阶段是否解耦，从而在 TTFT、ITL、TPOT、E2E latency、throughput、goodput 和资源利用率之间取得平衡。Orca 将生成式 Transformer serving 的调度粒度推进到 iteration level [1]；vLLM 通过 PagedAttention 为 continuous batching 提供了 KV 内存管理基础 [2]；Sarathi-Serve 代表 chunked prefill 路线 [3]，DistServe 则代表 prefill/decode disaggregation 路线 [4]。在 CEC 场景中，这个问题进一步复杂化。接入边缘的请求池通常小于云端数据中心，单纯等待大 batch 会破坏 TTFT；边缘 GPU/NPU 显存有限，KV Cache 会限制可并发 decode 请求数；端-边、边-云链路状态会影响 prefill offloading、decode placement 和状态传输收益；不同业务还可能具有差异化 SLO，例如机器人控制、车联网辅助决策、工业现场问答和普通对话服务的最大等待时间并不相同。因此，本章不再把 SLO-aware、KV-aware、multi-tenant 或 speculative verification 作为并列一级分类，而是按 batching 机制和 batch 更新粒度组织相关工作。SLO-aware、KV-aware、multi-tenant、speculative verification 和 multi-model multiplexing 不是被忽略，而是在本章中作为横向约束或补充机制嵌入各类 batching 机制。例如 SOLA/JITServe 属于 SLO-aware admission，vLLM 属于 KV-aware continuous batching 基础设施，S-LoRA/Punica 说明多租户 adapter serving 会直接改变 batch 的组成方式和执行算子，AdaServe 和 MuxServe 则分别说明 speculative verification 与多模型时空复用也会引入额外的批处理决策。本文将 CEC 下的业务感知 batching 参数优化划分为四类：

1. **Queue-level Dynamic / Admission Batching**：batch 在请求进入执行前根据到达情况、等待窗口、优先级、SLO slack 或 admission threshold 动态组成。
2. **Continuous Batching / Iteration-level Batching**：以 decode iteration 或 token iteration 为边界动态维护 active batch，使完成请求退出、新请求或被抢占请求加入。
3. **Chunked Prefill / Blended Batching**：把长 prefill 切成 chunk，并与 decode token 或其他请求混合执行，缓解长 prompt 对 decode 的阻塞。
4. **Disaggregated Prefill-Decode Batching**：将 prefill 和 decode 分配给不同实例、资源池或硬件层级，分别形成阶段 batch，并通过 KV/state transfer 衔接。

这四类不是严格互斥的，而是从 batching 形成与更新粒度描述不同批处理组织方式。传统 Static Batching 仍可作为 baseline，但不再作为本章一级方法，因为它主要代表固定批次假设，并不能体现 CEC 下的业务感知和在线自适应。Queue-level dynamic/admission batching 可以叠加 SLO-aware

priority; Continuous batching 可以叠加抢占、KV-aware admission 和 active set 调整, Orca 和 FastServe 分别体现了 iteration-level scheduling 与 preemptive scheduling 的代表路线 [1], [41]; Chunked Prefill / Blended Batching 仍然可能运行在 continuous batching runtime 上, Sarathi-Serve 是这一方向的典型系统 [3]; Disaggregated Prefill-Decode Batching 则进一步把两个阶段的 batch 从同一实例内部拆到不同资源池, DistServe 是阶段解耦路线的代表 [4]。这个分类的目的不是把每篇论文强行放入唯一格子, 而是让读者看清: 一个方法主要是在请求接纳/动态组批、迭代级 active batch、prefill chunk 混合, 还是阶段解耦上做优化。

表 7 汇总了与 batching 执行机制直接相关的代表工作, 同时保留少量会改变批处理决策的横向机制; 图 13 给出了本章四类 batching 方法在批次形成粒度上的分类主线。SLO-aware、multi-tenant、KV-aware、speculative verification、multi-model serving 和 inference/finetuning co-serving 等内容在本章中作为相关机制或约束出现, 不再作为一级分类。这里纳入 LoRA serving、多模型 serving 和 inference/finetuning co-serving 工作, 是因为它们的核心系统问题并不只是“是否共置多个模型”, 而是如何让不同 adapter、不同模型或不同任务类型共享 batch、共享 GPU 时间片, 并在 SLO、显存和阶段干扰约束下决定执行顺序。

表 7 CEC 下业务感知 Batching 参数优化方法相关工作

序号	代表工作	发表在哪	主归属	相关机制	方法关键词
1	JITServe [42]	NSDI 2026	Queue-level Dynamic / Admission Batching	SLO-aware request admission	根据请求 SLO deadline、剩余时间预算和当前服务带宽动态决定接纳、排序和组批
2	SOLA [43]	MLSys 2025	Queue-level Dynamic / Admission Batching	SLO-aware scheduling; phase-aware scheduling	根据成本模型预测 TTFT/IPOT 相对 SLO 的紧迫程度, 对等待请求排序并决定谁优先进入 batch
3	QLM [44]	AIOps@ASPLOS 2024	Queue-level Dynamic / Admission Batching	Queue management	以满足端到端延迟 SLO 为目标, 通过队列控制减少 head-of-line blocking 并提高 SLO attainment
4	TimelyLLM [45]	MobiSys 2026	Queue-level Dynamic / Admission Batching	time utility; iteration-level control	面向时间敏感应用, 将 deadline 和 time utility 纳入 batch 接纳与请求优先级
5	Preble [46]	ICLR 2025	Queue-level Dynamic / Admission Batching	KV-aware admission; distributed prompt scheduling	在分布式 serving 中把 prefix/KV cache 命中和负载均衡共同纳入请求分配, 使共享 prompt 请求更容易进入可复用状态的实例
6	Fairness in Serving Large Language Models [47]	OSDI 2024	Queue-level Dynamic / Admission Batching; Continuous / Iteration-level Batching	multi-tenant fairness; VTC	在 continuous batching 机制上引入 Virtual Token Counter, 用已服务 token 成本调整调度顺序, 减少租户之间的服务不公平
7	Orca [1]	OSDI 2022	Continuous / Iteration-level Batching	selective batching	提出 iteration-level scheduling, 新请求在 decode iteration 边界加入, 完成请求及时退出 active batch
8	FastServe [41]	NSDI 2026	Continuous / Iteration-level Batching	preemptive scheduling	在 continuous batching 基础上引入迭代级抢占, 缓解长请求占用和尾延迟问题

9	TokenFlow [48]	EuroSys 2026	Continuous / Iteration-level Batching	streaming token scheduling	以 token stream / scheduling interval 为控制粒度，动态做 admission、preemption 和 resumption
10	vLLM [2]	SOSP 2023	Continuous / Iteration-level Batching	KV block memory management	PagedAttention 用逻辑/物理 KV block 映射降低显存碎片，为更大的 active batch 提供内存基础
11	SGLang [5]	NeurIPS 2024	Continuous / Iteration-level Batching	RadixAttention; prefix reuse	通过 RadixAttention 自动匹配和复用共享前缀 KV，使复杂多轮、RAG 和 agent workload 的 active batch 更少重复 prefill
12	Llumnix [49]	OSDI 2024	Continuous / Iteration-level Batching	runtime rescheduling; live migration	在多个 vLLM 实例之间迁移运行中请求及其内存状态，缓解 active batch 负载不均、优先级变化和尾延迟问题
13	Punica [50]	MLSys 2024	Continuous / Iteration-level Batching	multi-tenant LoRA batching	用 SGMV 等算子支持不同 LoRA adapter 请求共享同一基础模型 batch，降低多租户 LoRA serving 的显存和计算浪费
14	S-LoRA [51]	MLSys 2024	Continuous / Iteration-level Batching	heterogeneous adapter batching; unified paging	通过统一分页管理 adapter 权重和 KV cache，并用定制 LoRA 算子支持大量 adapter 请求在同一 serving 实例中批处理
15	NanoFlow [52]	OSDI 2025	Continuous / Iteration-level Batching; Chunked Prefill / Blended Batching	nano-batch scheduling	将执行拆成 nano-batches 并搜索数量、大小和顺序，同时对长 prefill 做 token 粒度分块
16	Sarathi-Serve [3]	OSDI 2024	Chunked Prefill / Blended Batching	chunked prefill; decode interleaving	将长 prefill 切成 chunk，并与 decode 混合交错执行，平滑两阶段负载
17	Libra [53]	NSDI 2026	Chunked Prefill / Blended Batching	phase quota control	在混合批次中调控 prefill 与 decode token 配额比例，降低两阶段算子干扰
18	A Hybrid Online and Offline Requests Inference Serving System for LLM in Private Computer Environment [54]	IEEE TC 2026	Chunked Prefill / Blended Batching	online/offline mixed batching	在同一 continuous batch 内组织在线 prefill/decode 与离线任务，在线优先、离线填充空闲资源
19	LLMStation [55]	USENIX ATC 2025	Chunked Prefill / Blended Batching	iteration-level multitasking; inference/tuning co-serving	将 fine-tuning 任务改造成可挂起流水线，并在 iteration 粒度批处理 inference 与 tuning 请求，以提高 GPU 利用率同时满足 serving SLO
20	FlexLLM [56]	NSDI 2026	Chunked Prefill / Blended Batching	token-level co-serving; SLO-aware scheduling	将 inference 和 PEFT finetuning 融合到 token 级执行，并优先保护 latency-sensitive inference 的 SLO，再利用空闲 token 预算推进 finetuning
21	AdaServe [57]	EuroSys 2026	横向机制: Speculative Verification Batching	multi-SLO speculative decoding	按请求 SLO 定制 speculation tree，并通过 speculate-select-verify pipeline 形成验证批处理；它主要补充 multi-SLO 场景下的验证调度，而不是 prefill chunking
22	DistServe [4]	OSDI 2024	Disaggregated Prefill-Decode Batching	phase-specific workers	将 compute-heavy prefill 与 memory/latency-sensitive decode 解耦到不同 worker pool
23	Splitwise [27]	ISCA 2024	Disaggregated Prefill-Decode Batching	prompt pool; token pool	将 prompt computation 和 token generation 放到不同机器池，并分别匹配硬件资源
24	Mooncake [13]	USENIX FAST 2025	Disaggregated Prefill-Decode Batching	distributed KV cache; global scheduling	在 prefill/decode 解耦基础上构建分布式 KVCache，并按 KV 本地性和 SLO 调度实例

25	MuxServe [58]	ICML 2024	横向机制: Multi-model Multiplexed Batching	adaptive batch scheduling: 面向多 LLM serving 联合做空间共置和时间复用, 并 spatial-temporal multiplexing	利用 prefill/decode 阶段特征设计 adaptive batch scheduling, 以提升多模型场景下的 SLO 吞吐
----	---------------	-----------	---	--	--

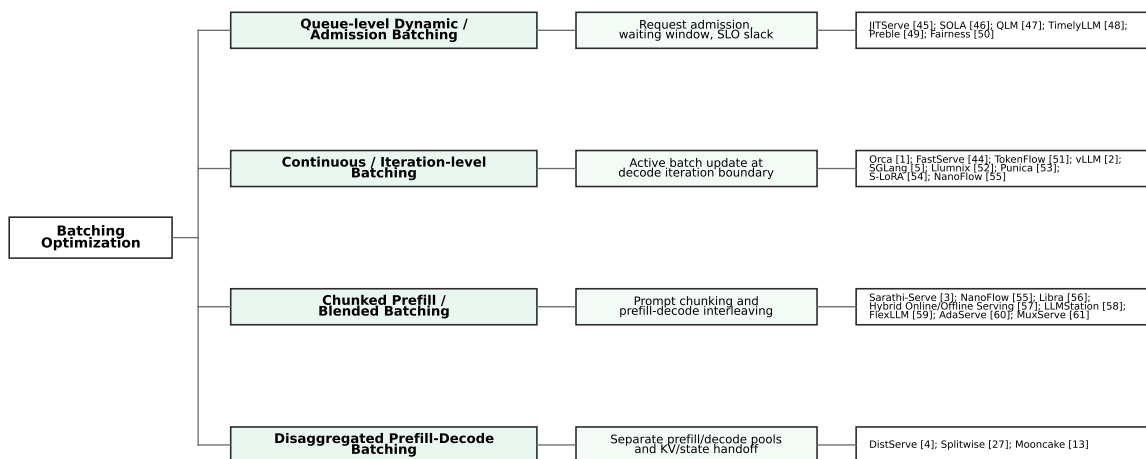


图 13 CEC 下业务感知 Batching 参数优化方法的分类树

5.1 Queue-level Dynamic / Admission Batching

Queue-level Dynamic / Admission Batching 要解决的是“固定等待固定 batch size 过于僵硬”的问题。真实 LLM 服务中，请求到达是随机的，prompt 长度、输出长度和 SLO 也不一致。如果系统仍按固定 batch size 或固定时间窗口组批，低负载时会等待过久，高负载时又可能把紧急请求排在队尾。因此，这类方法在请求进入执行前动态决定 batching window、batch membership、admission threshold 和请求优先级。JITServe 从 SLO-aware serving 角度建模请求服务带宽 [42]，SOLA 以 state-aware scheduling 提高 SLO attainment [43]，QLM 关注队列管理和 head-of-line blocking [44]，TimelyLLM 则面向时间敏感机器人应用组织请求调度 [45]。Preble 说明 prefix/KV locality 会改变请求分配 [46]；Fairness in Serving Large Language Models 则说明多租户公平性也会改变请求排序和 token 服务顺序 [47]。

这种队列级决策可以抽象成图 14 中的闭环：请求先等待，控制器再根据剩余时间预算决定立即执行还是继续合批。Incoming Requests 携带 SLO 和 priority 进入 Waiting Queue，Admission Controller 持续读取 SLO slack、wait time、service estimate 和 queue length。临近 deadline 的请求进入 Urgent Batch，slack 较充足的请求进入 Throughput Batch 或继续等待。Monitoring Feedback 将 TTFT、E2E latency 和 SLO violation 反馈给控制器，用于调整 batching window 与 admission threshold。由此可见，dynamic batching 与 static batching 的区别不只是 batch size 会变，而是组批本身围绕剩余时间预算在线决策。

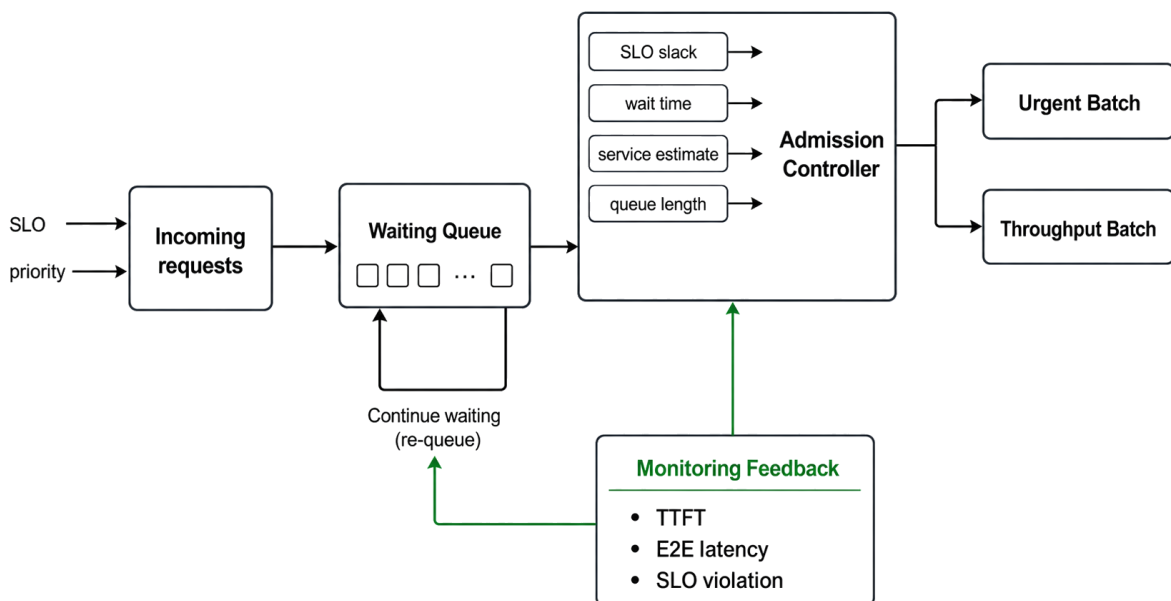


图 14 Queue-level Dynamic / Admission Batching 机制示意图

这类方法的核心思想是把组批从静态规则改为在线决策。调度器可以根据队列长度、请求等待时间、SLO slack、prompt 长度、预计输出长度、业务优先级和当前实例负载决定是否立即发起 batch，还是继续等待更多请求。对于 slack 充足的请求，可以适当等待以提高吞吐；对于即将违反 TTFT 或 deadline 的请求，应尽快进入执行队列。与 static batching 相比，dynamic batching 的 batch 仍然主要在执行前形成，但形成过程已经具备业务感知能力。

Queue-level Dynamic / Admission Batching 的关键变量包括 batching window、max wait time、admission threshold、request priority、deadline slack、estimated service time 和 batch size limit。很多 SLO-aware request batching 工作可以放在这一类中理解：它们并不一定改变 decode iteration 的 active batch 机制，而是先在请求队列和接纳阶段决定谁应该进入下一批执行 [42], [43]。

在 CEC 中，dynamic batching 需要把网络时延纳入剩余时间预算。接入边缘的优势是 RTT 低，但请求池小；区域边缘或云端请求池更大，但链路更长。因此，同一个请求是否应该在接入边缘立即小批次执行，还是转发到区域边缘等待更大 batch，取决于剩余 SLO、入口到实例链路、当前排队时间和模型能力。Dynamic batching 是 CEC 中最自然的业务感知入口，因为它在请求尚未进入长期 decode 循环前就可以完成接纳、排序和路径选择。

Queue-level Dynamic / Admission Batching 的风险在于调度器需要可靠估计请求服务时间。如果输出长度预测错误、队列状态滞后或网络时延估计不准，batching window 和优先级选择可能反而放大尾延迟。因此，在 CEC 中应避免过长等待窗口，并将 admission control、优先级老化和 fallback 策略作为配套机制。

5.2 Continuous Batching / Iteration-level Batching

Continuous Batching 进一步打破了 batch 生命周期。它要解决的是 LLM decode 阶段中请求完成时间不同导致的空槽浪费问题。传统 static 或 dynamic batching 即使在执行前能组成合适 batch，一旦进入自回归 decode，短请求会先完成，长请求还在继续生成。如果 batch 在整个生成过程中固定不变，已完成请求留下的 slot 无法被新请求利用，新到请求也必须等待当前 batch 全部结束。Orca 首先系统化提出 iteration-level scheduling 和 selective batching，成为后续 continuous batching 系统的基础 [1]。

当调度边界从请求级下降到 decode iteration 后，batch 的含义就变成图 15 中持续变化的 active set。每个 Iteration 表示一次 decode iteration，Active Batch 中的序列共同生成 next token。到达 iteration

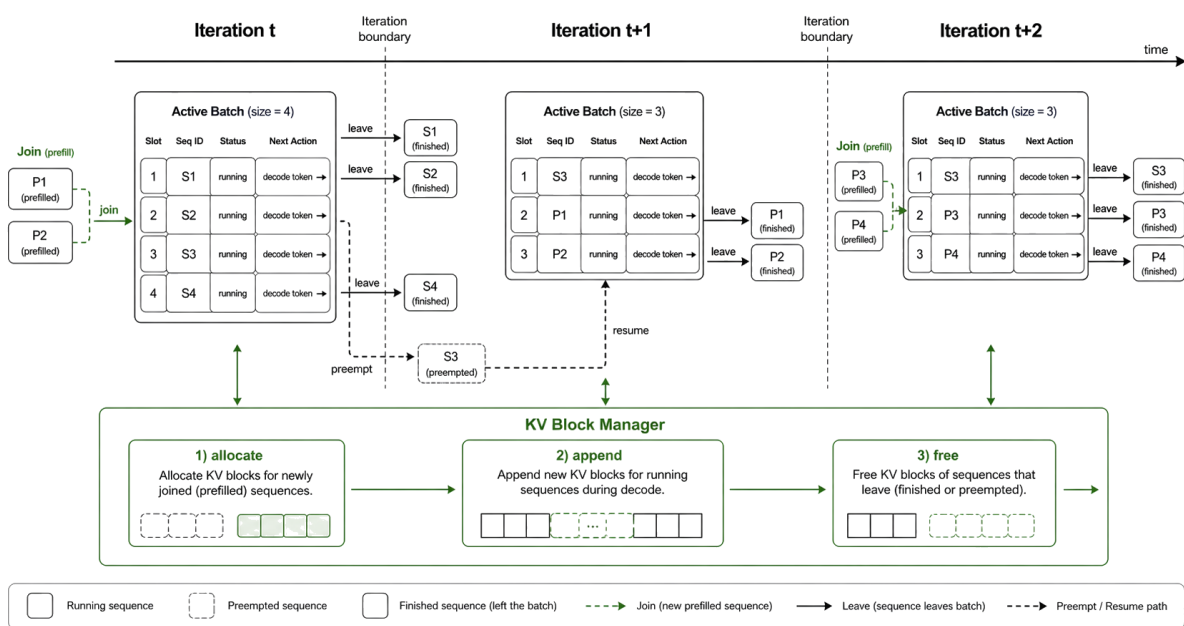


图 15 Continuous Batching / Iteration-level Batching 机制示意图

boundary 后，已完成序列退出，新完成 prefill 的序列加入，被抢占序列可以恢复。KV Block Manager 同步执行 allocate、append 和 free 操作，因为 active batch 的变化必然伴随 KV block 的增长和释放。因此，continuous batching 的 batch 不是固定请求集合，而是一个随 token 生成持续变化的运行时集合。

Continuous Batching 的解决思路，是把 batch 的更新边界缩短到 decode iteration。每生成一轮 token 后，系统都可以移除已经完成的请求，加入完成 prefill 的新请求，或者恢复被抢占的请求。这样 batch 不再是一组固定请求，而是一个随 token 生成持续变化的 active set。Orca 将这一思想形式化为 iteration-level scheduling，使 LLM serving 从“按完整请求批处理”转向“按 token iteration 批处理” [1]；FastServe 在这一基础上引入迭代级抢占 [41]，TokenFlow 面向 streaming token 服务做 preemptive scheduling [48]，NanoFlow 则用 nano-batch execution 进一步细化执行流 [52]。

这类方法的核心变量包括 active sequence set、decode batch size、token iteration order、join policy、leave policy、preemption policy、resume policy 和 KV block allocation。由于每轮 decode 都要读取并追加 KV Cache，continuous batching 必须和内存管理绑定：请求能否加入 active batch，不仅取决于是否有计算 slot，也取决于是否有足够 KV block 和状态本地性。vLLM 的 PagedAttention 正是从 KV block 管理角度支撑更大的 active batch 和更低的显存碎片 [2]；SGLang 用 RadixAttention 复用共享前缀 [5]；Llumnix 则把运行中请求迁移纳入多实例调度 [49]。这些工作说明，continuous batching 的 active set 会进一步受到 prefix cache 和实例负载的共同约束。

多租户 LoRA serving 也可以从 continuous batching 角度理解。Punica 和 S-LoRA 的目标不是改变自回归生成逻辑，而是让请求不同 adapter 的序列仍能共享同一个基础模型执行批次。为此，它们需要解决 adapter 权重分页、异构 adapter batching 和定制 LoRA 算子问题；这些机制改变的是 batch 内请求能否被合并执行，以及合并后是否仍能保持较低的显存和算子开销 [50], [51]。

在 CEC 中，Continuous Batching 不能简单照搬云端大请求池策略。接入边缘并发较低，短时间内能加入 active batch 的请求有限；如果为等待更多 token stream 而延长调度周期，TTFT 会恶化。因此，边缘侧更适合短窗口、低等待的 micro-continuous batching；区域边缘则可以跨多个入口聚合请求，形成更稳定的 active batch。对于移动用户，还需要考虑请求是否会在 decode 期间切换入口，避免 active batch 中的 KV 状态和用户路径频繁迁移。

Continuous Batching 是现代 LLM serving 的基础能力，但不是完整的业务感知调度。它提升 decode 阶段资源利用率，却不天然感知 SLO、应用等级、KV 状态迁移、边缘链路和隐私约束。因此，在 CEC 中应把 Continuous Batching 视为 runtime 基座，再叠加 dynamic priority、phase-aware interleaving 和状态感知 admission。

5.3 Chunked Prefill / Blended Batching

Chunked Prefill / Blended Batching 要解决的是长 prefill 阻塞 decode 的问题。Prefill 处理完整 prompt，矩阵规模大、并行度高，通常更偏 compute-bound；decode 逐 token 生成，频繁访问 KV Cache，更偏 memory-bound 或 latency-bound。如果长 prompt prefill 连续占用 GPU，已经在流式输出的请求会出现 ITL 抖动；如果完全优先 decode，新请求 TTFT 又会被推迟。两阶段混部时，关键问题不是是否 batching，而是 prefill 和 decode 如何在同一执行资源上交错。Sarathi-Serve 将长 prefill 切成 chunks 并与 decode 交错，是这一类方法的代表性工作 [3]。

长 prompt 不能直接整段 prefill 的原因，可以通过图 16 看到：系统需要把 prefill chunk 和 decode token 交错安排，在 TTFT 与 ITL 之间控制阶段干扰。Long Prompt 先经过 Prefill Chunker 被拆成 P1、P2、P3 等 prefill chunks；Decode Stream 表示已经进入流式输出的 active decode tokens。Blended Scheduler 根据 chunk size、prefill budget、decode budget 和 phase quota，在同一调度周期内混合安排

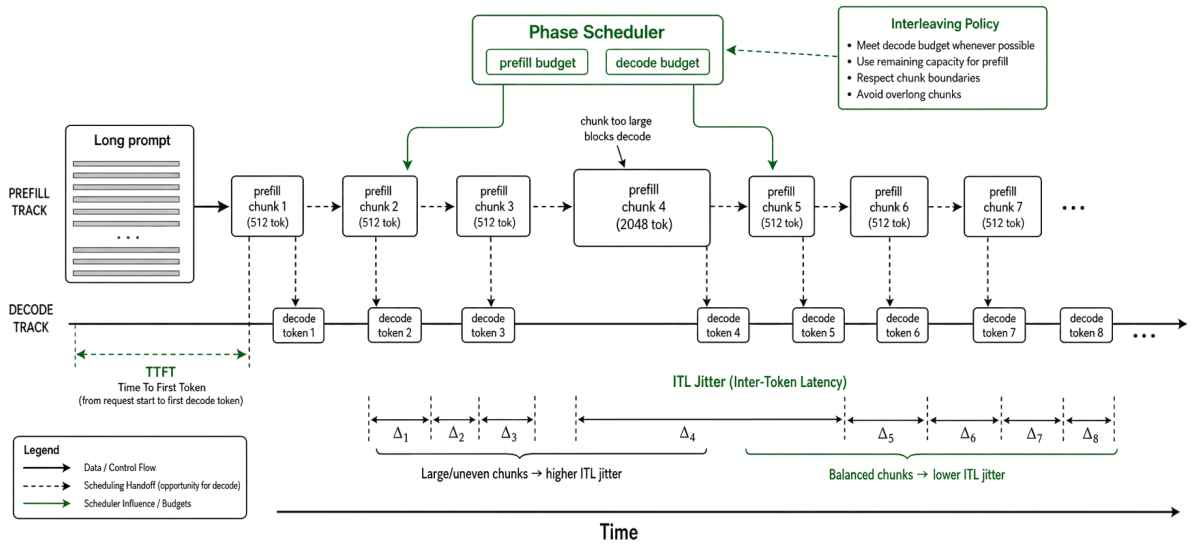


图 16 Chunked Prefill / Blended Batching 机制示意图

prefill chunk 与 decode tokens。TTFT、ITL 和 stall feedback 用于修正阶段配额。chunked prefill 的目标不是把 prefill 拆得越细越好，而是让长 prompt 不再一次性阻塞 decode。

Chunked Prefill 的思想是把长 prompt 拆成多个 prefill chunk，而不是一次性执行完整 prefill。Blended Batching 则进一步把 prefill chunk 与 decode token 混合在同一个调度周期或同一个 batch 中，使 compute-heavy 的 prefill 和 memory/latency-sensitive 的 decode 交错执行。这样可以平滑每轮计算量，减少长 prefill 对 decode 的阻塞，也能避免 decode 完全占用调度周期导致新请求 TTFT 过高。

这类方法的核心变量包括 chunk size、prefill token budget、decode token budget、prefill-to-decode ratio、interleaving policy、phase quota、online/offline mixing ratio 和 stall control。Chunk 太大，会重新造成 prefill 阻塞；chunk 太小，会增加调度和 kernel overhead。Prefill/decode 比例太偏向 prefill，会伤害 ITL；太偏向 decode，会伤害 TTFT。因此，Chunked Prefill / Blended Batching 的本质是阶段内混合调度和阶段间干扰控制。Libra 可以理解为阶段配额控制的扩展 [53]，NanoFlow 代表更细粒度执行流优化 [52]，LLMStation 和 FlexLLM 则把混合对象扩展到 inference/finetuning co-serving 等更复杂 workload [55], [56]。这些工作本质上仍然需要在同一批执行资源上安排不同类型 token 或任务片段的执行顺序。MuxServe 同样涉及 batch scheduling，但其主问题是多模型 serving 的空间共置与时间复用，更适合作为 multi-model multiplexed batching 的横向机制，而不是 chunked prefill 的直接代表 [58]。

在 CEC 中，这类方法特别适合接入边缘和区域边缘之间协同。接入边缘可以保持较短 decode 间隔，区域边缘可以处理更长 prefill chunk；当链路稳定且 KV 传输成本可控时，系统还可以把长 prompt prefill 拆分到更强节点，再将可复用状态交给近用户 decode 节点。若链路抖动大或边缘并发不足，过度切分 prefill 可能增加状态传输和调度开销，因此 chunk size 和 interleaving ratio 必须纳入

在线代价模型。

Chunked Prefill / Blended Batching 比 continuous batching 更直接处理 prefill/decode 干扰。它适合长 prompt、多轮对话、RAG 增强上下文和流式输出并存的场景，但需要准确控制 chunk size 和阶段配额。对于 CEC，关键是把网络传输和 KV 状态位置纳入阶段混合决策，否则 prefill 拆得越细，跨节点状态交接越可能成为新的瓶颈。

5.4 Disaggregated Prefill-Decode Batching

Disaggregated Prefill-Decode Batching 是对 phase-aware 思路的进一步扩展。Chunked Prefill / Blended Batching 仍然假设 prefill 和 decode 在同一个实例或同一类资源上交错执行；而 disaggregated batching 直接把两阶段拆到不同 worker pool、不同 GPU 实例、不同机器池甚至不同 CEC 层级中。Prefill batch 和 decode batch 分别形成，分别扩缩容，并通过 KV Cache 或中间状态传输衔接。DistServe 从 goodput 优化角度论证阶段解耦 [4]，Splitwise 强调 prompt pool 与 token pool 的硬件匹配 [27]，Mooncake 则在 prefill/decode 解耦基础上构建 KVCache-centric serving 架构 [13]。

PD 分离后的 batching 可以按图 17 理解：prefill instance 内部形成 prompt batch，decode instance 内部维护 active sequence batch，两个池之间通过 KV/state handoff 连接。Global Scheduler 根据 SLO、

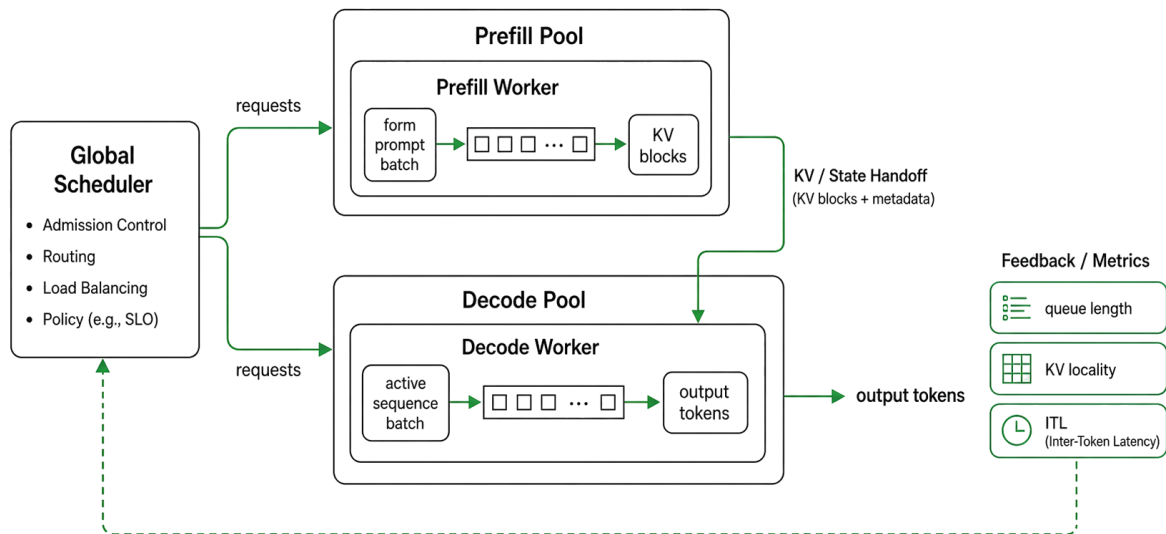


图 17 Disaggregated Prefill-Decode Batching 机制示意图

queue length、KV locality 和 link state 将请求送入 Prefill Pool；Prefill Worker 形成 prompt batches 并生成初始 KV blocks。随后 KV / State Handoff 将 KV blocks、state transfer 和 metadata 交给 Decode Pool，Decode Worker 再维护 active sequence batches 并输出 tokens。图中的反馈路径表示 queue length、KV locality、ITL 等运行时指标回流到全局调度器，用于调整后续请求进入哪个 pool 以及是否需要扩缩容。因此，PD 分离后的 batching 实际上变成两个 pool 的独立组批与跨池状态协调问题。

这类方法的出发点是 prefill 和 decode 的资源画像差异非常明显。Prefill 更偏 compute-bound，适合吞吐导向、更强算力或更大 batch 的资源池；decode 更偏 memory/latency-bound，需要稳定 ITL、低排队和快速状态访问。若两者混部在同一资源池中，长 prefill 可能阻塞 decode，decode 小步迭代又可能降低 prefill 吞吐。Disaggregation 通过物理或逻辑隔离减少两阶段干扰，并允许系统分别为 prefill 和 decode 选择 parallelism、batch size 和扩缩容策略。

Disaggregated Prefill-Decode Batching 的核心变量包括 prefill worker pool、decode worker pool、phase queue、KV transfer、state locality、worker scaling、prefill/decode routing 和 global scheduling。它的收益来自阶段资源匹配和干扰隔离，但代价集中在 KV 传输、状态连续性和跨节点调度。Prefill 完成后，decode 必须访问初始 KV Cache；如果 KV 传输慢、远程访问不稳定或用户入口发生变化，disaggregation 的收益会被状态成本抵消 [4], [13]。

这里的 batching 不是指“是否启用 PD 分离架构”本身，而是指 PD 分离之后 prefill pool 如何形成 prompt batch、decode pool 如何维护 active sequence batch，以及两个 pool 之间如何通过 KV/state handoff 协调。

在 CEC 中，disaggregated batching 具有直接映射关系：区域边缘或云端可以承担 prefill-heavy 请求，接入边缘负责低时延 decode；也可以让区域边缘维护较大的 prefill pool 和 KV state layer，接入边缘只在需要低 ITL 时承担 decode。对于移动用户，系统还需要判断 decode worker 是否应该跟随入口迁移，还是远程访问原有 KV。也就是说，CEC 下的 disaggregated batching 不能只看 GPU 利用率，还必须同时看链路稳定性、KV 迁移成本和服务连续性。

Disaggregated Prefill-Decode Batching 适合 prefill/decode 负载差异明显、阶段队列规模足够、状态传输成本可控的场景。若请求很短、边缘并发很低或链路抖动大，复杂阶段拆分可能不如本地 blended batching 稳定。因此，CEC 中应把 disaggregation 视为高层资源组织方式，而不是默认打开的单点优化。

5.5 面向华为验证环境的方案建议

面向华为验证环境，batching 参数优化需要在请求持续到达、prompt 长度变化、KV 水位变化和业务 SLO 不同的情况下，在线决定下一轮接纳哪些请求、按什么顺序执行、给 prefill 和 decode 分配多少 token 预算，以及是否启用 chunked prefill、长请求隔离或 PD 分离。该决策必须兼顾吞吐、尾延迟、公平性和显存安全，避免高优先级请求超时、普通请求长期饥饿或 KV Cache 触发 OOM。

因此，本章建议采用分阶段增强的轻量 actor-critic 强化学习在线 batching 优化器。算法输入包括推理实例集合 $\mathcal{S}$ 、等待请求集合 $\mathcal{Q}_t$ 、当前 active set、KV Cache 状态、硬件/模型画像和业务 SLO

配置；算法输出为当前控制窗口的 batching 动作。该问题的决策变量是控制窗口 t 内的动作向量

$$a_t = (b_t, u_t, w_t, c_t, \rho_t, \alpha_t, \beta_t, \eta_t).$$

其中, b_t 表示 max running requests 或目标 active batch 规模; u_t 表示 max batched tokens 或 token budget; w_t 表示最大等待窗口; c_t 表示 chunk size; ρ_t 表示 prefill/decode token quota; α_t 表示 admission threshold; β_t 表示请求排序权重, 用于综合 SLO slack、等待时间、业务优先级、预估 prefill 代价和 KV 增长; η_t 表示离散控制开关, 包括长请求隔离、chunked prefill 以及在具备稳定 KV handoff 时可选的 PD 分离。actor 网络用策略 $\pi_\theta(a_t | s_t)$ 根据当前状态 s_t 生成动作 a_t , 其中 θ 为策略参数, $\pi_\theta(a_t | s_t)$ 表示在状态 s_t 下选择动作 a_t 的策略分布或打分函数; critic 网络估计该动作对 SLA 达标率、端到端时延、ITL 稳定性和 KV 风险的长期收益。算法核心是在 runtime 可执行动作空间中学习“接纳谁、按什么顺序执行、用多大 token/batch 预算、是否切 chunk、是否启用阶段控制”。

这些动作变量也不是在无限连续空间中任意生成。系统先根据 runtime 能力和当前资源状态构造可执行动作空间: 由剩余 KV block、显存水位和实例并发上限给出 b_t 与 u_t 的取值范围; 由业务 TTFT、deadline slack 和当前排队长度给出 w_t 与 α_t 的范围; 由长 prompt 比例、prefill/decode 时间占比和 ITL 抖动给出 c_t 与 ρ_t 的候选值; 由请求业务等级、等待时间、预估服务时间和 KV 增量生成排序权重 β_t 的候选组合; 由 runtime 是否支持 chunked prefill、长请求隔离和 PD/KV handoff 决定 η_t 中哪些开关可以打开。在此基础上, actor 只负责从可执行动作空间中选择或微调动作, 安全层再根据 KV 上限、P99 TTFT 预测、OOM 风险和切换幅度对 a_t 做裁剪或回退。因此, 第五章的强化学习不是让模型凭空发明 batching 参数, 而是在业务和资源约束限定的动作空间内学习更合适的参数、排序和阶段控制组合。图 18 给出了该 actor-critic 控制器的状态观测、动作生成、安全过滤和策略输出链路。

在线 batching 优化器不是简单输出一个固定 batch size, 而是根据请求队列、KV 水位、active batch、prefill/decode backlog 和 SLO slack 生成一组可执行控制量, 包括 batch/token 预算、等待窗口、准入阈值、排序权重、chunk size 和 prefill/decode quota。在论文建模上, 可以先把每个控制窗口的决策写成带安全约束的动作选择问题:

$$\begin{aligned} & \max_{a_t \in \mathcal{A}_{\text{safe}}(s_t)} R_t(a_t) \\ \text{s.t.} \quad & K_t(a_t) \leq K_{\max}, \\ & \hat{T}_{\text{tff}}(r_i, a_t) \leq D_i, \quad \forall r_i \in \mathcal{A}_t, \\ & a_t \in \mathcal{A}_{\text{exec}}(s_t), \\ & \Delta(a_t, a_{t-1}) \leq C_{\text{switch}}. \end{aligned}$$

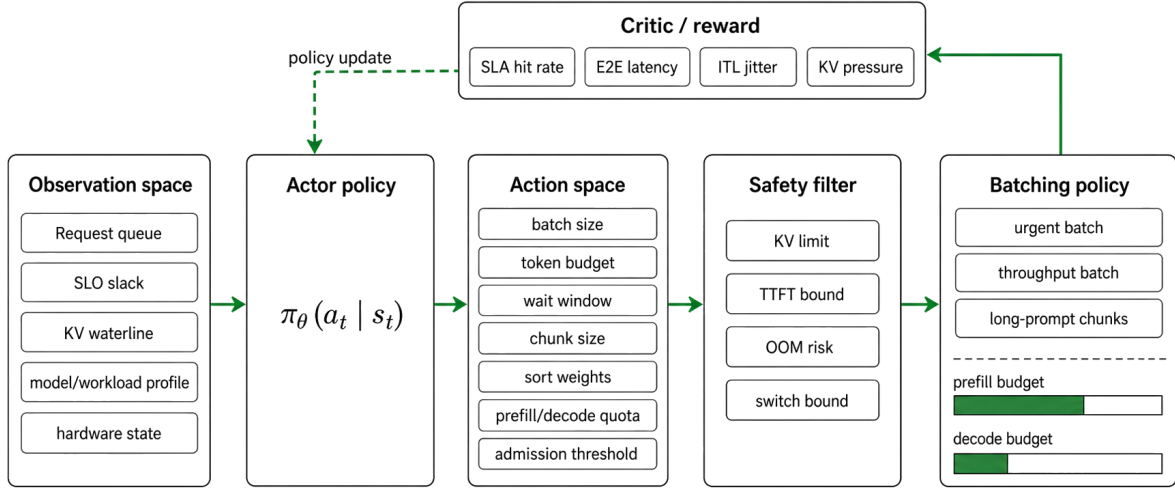


图 18 基于 actor-critic 强化学习的在线 batching 优化架构

其中 $R_t(a_t)$ 是动作在当前窗口带来的综合收益， $\mathcal{A}_{\text{safe}}(s_t)$ 是经过安全过滤后的动作集合， K_t 是动作执行后的 KV block 或显存占用， D_i 是请求 r_i 的 deadline 或 TTFT 约束， $\mathcal{A}_{\text{exec}}$ 表示当前 runtime 可执行动作集合， $\Delta(a_t, a_{t-1})$ 限制相邻窗口的参数变化幅度。actor-critic 不需要在公式中展开复杂的长期期望项；可以理解为 actor 根据 s_t 产生动作，critic 根据当前收益和后续窗口反馈估计动作的长期价值。验收目标直接对应收益设计：相对于“静态 batching 配置、不调整请求处理顺序”的基线，满足 SLA 的请求处理速度或 rps 提高 20%，所有请求平均 E2E 时延下降 20% [59]。

状态 s_t 由三类画像构造。第一类是硬件与实例状态，包括 GPU/NPU 型号、可用显存、KV block 剩余量、active sequence 数、waiting queue 长度、prefill queue 长度、decode backlog、最近 P90/P99 TTFT、TPOT/ITL、prefix cache 命中率和链路状态。第二类是模型与 workload 画像，包括模型类型、上下文长度、单 token KV Cache 增量、prefill/decode 计算比例、prompt token 估计、输出长度估计、VLM 或长 prompt 比例。第三类是业务需求配置，包括请求业务等级、HTTP header 中的特殊时延要求、deadline、SLO slack、优先级比例、是否允许长请求隔离、是否允许 chunked prefill、是否允许 PD 分离和降级回退。

可选动作的生成需要体现不同 batching 能力的适用条件。默认动作先保留框架现有 continuous batching 能力，只在安全范围内调整 max running requests、max batched tokens 和等待窗口；当长 prompt 或 VLM 请求比例升高，并且 prefill 对 decode ITL 的干扰明显时，再生成 chunked prefill、长请求隔离和 prefill/decode quota 调整动作；当存在稳定的 prefill pool、decode pool 和 KV handoff 观测时，才生成 PD 分离动作。图 18 中 urgent batch、throughput batch 和 long-prompt chunks 表示策略输出会同时改变请求接纳、排序和 token 预算，而不是只调一个全局参数。请求排序和准入动作同时考虑 slack、请求 age、预估 prefill 代价、KV 增长和对已有 decode 流的干扰，使优先级只是排序信号之一，而不

是唯一出队规则。

是否采用某个动作由安全过滤和强化学习策略收益共同决定。动作执行前先检查 KV 水位、OOM 风险、P99 TTFT 预测、runtime 支持能力和参数切换幅度；不满足约束的动作被裁剪或丢弃。通过过滤后，轻量 actor-critic 根据实际反馈更新策略，奖励可写为：

$$R_t = b_1 R_{sla} - b_2 L_{e2e} - b_3 V_{itl} - b_4 M_{kv} - b_5 R_{oom} - b_6 F_{starve}.$$

其中 R_{sla} 是满足 SLA 的请求处理速度或 rps， M_{kv} 是 KV 水位压力， R_{oom} 是 OOM 或回退风险。若某个动作提高了吞吐但导致高优先级请求 TTFT 违约、普通请求饥饿或 ITL 抖动恶化，奖励会下降，策略后续应减少选择该动作。

算法验证可采用三阶段递进方式。第一阶段只学习 batch 参数自适应，保持 FIFO 或框架默认排序不变，用 shadow mode 对比固定参数和在线参数策略，确认收益是否来自 max running requests、max batched tokens、等待窗口和 chunk size 的调整。第二阶段加入请求排序和准入控制，对 10% 高优先级、10% 低优先级和 80% 普通请求的 workload [59]，比较 FIFO、EDF/slack、priority queue 与策略学习到的排序权重，观察高优先级违约率、普通请求饥饿比例、P99 TTFT 和 ITL 抖动。第三阶段再形成混合策略，允许策略同时调节参数、排序权重、chunked prefill、长请求隔离和可选 PD 分离动作，重点观察满足 SLA 的请求处理速度、平均 E2E 时延、策略切换次数、回退次数和链路扰动下的稳定性。

最终的落地策略可概括为“先参数、再排序、后混合”。先学习 batch 参数自适应，保证在不改变请求顺序的情况下相对静态配置有稳定收益；再学习 SLO/代价感知的请求排序和准入权重；最后把参数、排序和阶段控制放入同一个受安全约束的在线策略学习闭环。Continuous batching、dynamic batch、chunked prefill 和 PD 分离只是底层可用能力，真正需要交付的是可观测、可回退、可解释的业务感知 batching 控制器。

5.6 小结

本章按 batch 形成和更新粒度，将业务感知 batching 参数优化划分为四类：Queue-level Dynamic / Admission Batching、Continuous Batching / Iteration-level Batching、Chunked Prefill / Blended Batching 和 Disaggregated Prefill-Decode Batching。Queue-level 方法决定请求进入执行前如何等待、排序和接纳；Continuous Batching 将 batch 更新边界推进到 decode iteration；Chunked Prefill / Blended Batching 通过切分长 prompt 并与 decode 混合执行来控制阶段干扰；Disaggregated Prefill-Decode Batching 则把 prefill 和 decode 放入不同实例或资源池中分别组批。

这四类方法不是互相替代的层级，而是一组可以叠加的 runtime 能力。SLO-aware、KV-aware、

multi-tenant 和 speculative verification 不再作为本章一级分类，是因为它们更多是横向约束或附加机制：SLO 会影响 admission 和排序，KV 状态会影响 active batch 容量和迁移成本，多租户会影响队列公平性，speculative verification 会引入额外的验证 batch。用 batch 更新粒度作为主线，可以更清楚地区分“请求进不进 batch”“active set 何时更新”“prefill 是否切块混合”“两阶段是否分池组批”这几类核心问题。

CEC 场景改变了 batching 的优化重心。云端 serving 通常依赖大请求池和高速互联来扩大 batch，而接入边缘往往请求池较小、链路波动更强、SLO 更紧、KV 显存更有限。因此，边缘侧更适合短等待窗口、低延迟 continuous batching 和保守的 chunked prefill；区域边缘适合聚合多个入口的 prefill-heavy 请求；云端适合承载长上下文、大 batch 和复杂解耦 serving。真正有效的业务感知 batching，需要同时读取请求长度、SLO、优先级、KV 占用、实例负载和链路状态，而不是只调一个固定 batch size。

面向验证环境，第五章的主线应是分阶段增强的轻量 actor-critic 强化学习在线 batching 优化器：先学习 batch 参数自适应，再学习请求排序和准入权重，最后学习参数、排序和阶段控制的联合策略。这样的安排既能对应静态 batching/FIFO 基线，也能把算法收益落实到满足 SLA 的请求处理速度、平均 E2E 时延、TTFT/ITL 稳定性、高优先级请求违约率、策略切换次数和回退次数等可测指标上。

六、CEC 中的 LLM Inference Task Offloading

传统 CEC 计算卸载通常将任务建模为具有给定输入数据量、计算量、deadline 和能耗约束的计算 job，其核心问题是在本地设备、邻居设备、边缘服务器和云端之间选择执行位置，并联合优化通信资源和计算资源分配。在这一范式下，任务卸载的主要决策对象通常是完整任务、子任务、任务链断点或 DAG 节点，优化目标也多集中在时延、能耗、系统成本和队列稳定性等方面。

LLM inference task 改变了这一问题形态。与普通计算任务不同，LLM 推理的执行时间不仅由输入 prompt 决定，还受到输出长度、decode step 数、batch 状态、模型队列和上下文长度影响；一次请求也不一定只由单个模型独立完成，而可能需要多个模型、多个 agent、多个工具或多个 verifier 协作完成；多轮对话和长上下文服务则进一步引入 KV cache、prefix cache、session state、agent memory 和 user context 等运行时状态。因此，在 CEC 场景下，LLM inference task offloading 不再只是决定“一个任务是否卸载到边缘”，而是需要决定完整请求如何路由、多个模型或 agent 如何编排，以及支撑后续推理的 context/KV/runtime state 如何迁移、复用或重新计算。

基于上述特性，本章从一次 LLM 服务请求在 CEC 系统中的执行过程出发组织相关工作。具体而言，当一个 LLM request 到达系统后，首先需要决定该请求或会话应由哪个设备、边缘节点、云端服务或模型实例处理；对于复杂请求，系统可能进一步需要编排多个模型、agent 或工具形成协作式 workflow；在请求执行和多轮交互过程中，context、KV cache、prefix cache、session state 和 agent memory 等运行时状态还需要在不同节点之间保留、迁移、复用或重新计算。因此，本章将现有方法划分为三类：request/session-level routing and offloading、multi-model/multi-agent workflow orchestration，以及 context/KV/runtime state offloading and migration，三类代表性工作分别在表 9、表 10 和表 11 中展开。

代表工作总览。表 8 汇总了 LLM inference task offloading 相关工作的主归属、相关机制和方法关键词。前三类对应本章 6.1–6.3 的文献脉络，最后一类面向项目场景，将 request routing、KV/state management、SLA/memory 约束和长期 session cost 放入同一决策闭环。

表 8 CEC 下 LLM Inference Task Offloading 代表工作总览

序号	代表工作	发表在哪儿	主归属	相关机制	方法关键词
1	QoS-LLM-Router [60]	TMC 2025	Request / Session-level Routing and Offloading	QoS-aware routing; state-aware scheduling	结合请求质量、响应长度、GPU memory 和队列状态，将 edge LLM expert routing 建模为长期 QoS 优化问题
2	CES-CB [61]	TSC 2025	Request / Session-level Routing and Offloading	request cost prediction; combinatorial bandit	利用 prompt embedding 和历史执行数据预测请求成本，在 edge/cloud 之间选择可提升吞吐并缓解长请求阻塞的请求集合
3	RouteLLM [62]	ICLR 2025	Request / Session-level Routing and Offloading	strong/weak model routing	基于 query 难度和偏好数据判断是否调用强模型，在保持质量的同时降低强模型调用成本

4	Training-Free Router [63]	NeurIPS 2025	Request / Session-level Routing and Offloading	budget-aware multi-LLM routing	通过近邻检索估计不同 LLM 的质量和成本，并用预算影子价格实现低训练成本的在线模型路由
5	ActiveInfer-Offload [64]	TMC 2024	Request / Session-level Routing and Offloading	offloading; resource allocation	使用 active inference 在不确定 cloud-edge 环境中联合决定 LLM inference task offloading 和资源分配
6	SLM-LLM-Collab [65]	TMC 2026	Request / Session-level Routing and Offloading	SLM caching; D2D / edge collaboration	在本地 SLM、邻居 SLM 和 edge LLM 之间做短时卸载决策，并将长期 SLM caching 作为请求路由前置条件
7	MGCO [66]	TSC 2026	Request / Session-level Routing and Offloading	mobility-aware offloading	用 Transformer MHA encoder 建模历史决策和未来移动状态，使 generative computation offloading 避免短视路由
8	MasRouter [67]	ACL 2025	Multi-model / Multi-agent Workflow Orchestration	multi-agent routing	联合选择协作模式、agent 数量、角色分工和底层 LLM，使 multi-agent system configuration 匹配 query 难度
9	Conductor [68]	ICLR 2026	Multi-model / Multi-agent Workflow Orchestration	natural-language workflow control	训练元控制器用自然语言生成多模型 workflow，控制 subtask、model id 和上下文访问范围
10	AFLOW [69]	ICLR 2025	Multi-model / Multi-agent Workflow Orchestration	automatic agentic workflow search	将 agentic workflow 表示为代码形式的 LLM 调用图，通过搜索和执行反馈自动优化复杂任务 workflow
11	DT-MADTO [70]	TMC 2025	Multi-model / Multi-agent Workflow Orchestration	mobility-aware DAG offloading	用 digital twin 预测移动和资源状态，联合决策 DAG 子任务卸载、带宽分配和中间数据等待时间
12	FuseSpill [71]	TPDS 2026	Context / KV / Runtime State Offloading and Migration	KV spillover; auxiliary GPU scheduling	在显存受限 GPU 上按代价模型选择 KV cache 溢出策略，并将溢出序列调度到辅助 GPU 继续 decode
13	Weaver [72]	USENIX ATC 2025	Context / KV / Runtime State Offloading and Migration	attention offloading	通过 attention offloading 缓解 multi-LLM serving 中的资源压力，提高多模型服务吞吐和资源利用率
14	Mooncake [13]	USENIX FAST 2025	Context / KV / Runtime State Offloading and Migration	distributed KV cache; global scheduling	将 CPU/DRAM/SSD/RDMA 组织为全局 KVCache Store，并按缓存命中、节点负载和 TTFT/TBT SLO 调度请求
15	CA-AIGC-Migration [73]	TSC 2026	Context / KV / Runtime State Offloading and Migration	context migration; mobility-aware service migration	将模型迁移转化为 context migration，在 reward 中联合考虑 context relevance、AoI、计算延迟和传输成本
16	长期成本感知请求卸载方案 本报告		Project-oriented Long-term Cost-aware Offloading	mobility/length prediction; action masking; block-level KV management	联合选择计算节点与 KV reuse/sync/recompute 方式，在满足 P99 E2E latency 和 memory 约束下最小化 session 长期服务成本

6.1 Request / Session-level Routing and Offloading

Request/session-level routing 是最接近传统 task offloading 的 LLM inference offloading 形式。它将完整 LLM 请求、query、conversation turn 或 session 作为基本决策对象，决定其应该由本地设备、邻居设备、接入边缘、区域边缘还是云端服务处理。与普通任务不同，LLM request 的处理时间往往难以在执行前准确确定，因为它不仅取决于 prompt length，还受到 output length、decode step 数、batch 状态和模型当前队列影响。对于多个 edge LLM experts、多个 SLM/LLM 服务或多层 device-edge-cloud 网

络，系统还需要根据请求复杂度、模型能力、响应质量、时延约束、GPU memory、队列状态和云端成本进行联合决策。

这一类问题的核心矛盾是：edge 具有低时延、低回传开销和隐私优势，但计算资源和显存有限，长输出请求可能占据 batch 并造成 head-of-line blocking；cloud 算力充足且模型能力更强，但会带来更高 RTT、带宽开销和数据出域风险。对于 CEC 系统而言，请求路由不应只回答“是否卸载到边缘”，还需要回答“卸载到哪个 edge server”“调用哪个 LLM expert”“是否先由 SLM 处理”“是否需要递归升级到更强模型或云端模型”等问题。表 9 汇总了该方向的代表性工作。

表 9 Request / Session-level Routing and Offloading 代表性工作

代表工作	发表在哪 & 时间	问题表述	核心方法	优化目标
QoS-LLM-Router [60]	TMC, 2025	多个 edge LLM experts 在生成质量、响应长度、GPU memory 和队列状态上异构；动态到达的请求需要实时选择 expert，以同时保证 response quality 和 latency requirement。	将请求路由建模为 infinite-horizon MDP；使用 DistilBERT-based predictors 预测 response quality 和 length；使用 heterogeneous graph attention network 抽象系统状态，并用 SAC 学习 QoS-aware routing policy。	最大化长期 accumulated QoS，同时考虑 response quality 和 latency constraint。
CES-CB [61]	TSC, 2025	LLM request 的 output length 和处理时间不可预测；如果长请求留在 edge，会阻塞后续短请求并降低 edge throughput。	将 prompt embedding 作为 context，将每轮 edge batch selection 建模为 contextual combinatorial semi-bandit；使用 neural prediction 与 UCB 机制平衡 exploration/exploitation。	最大化 edge throughput，减少因长请求导致的 head-of-line blocking。
RouteLLM [62]	ICLR, 2025	给定一个 query，如何在推理前决定调用强模型还是弱模型，使系统在满足质量目标的同时尽量少调用强模型。	使用偏好数据训练 router，预测强模型相比弱模型是否具有明显优势；根据阈值决定调用强模型还是弱模型。	在保持回复质量的同时降低强模型调用比例和推理成本。
Training-Free Router [63]	NeurIPS, 2025	在高流量、多 LLM 在线服务中，如何在不知道未来请求、预算有限且不能频繁训练 router 的情况下，快速把每个 query 分配给最合适的 LLM，以最大化整体回答质量并控制成本。	先用历史 query 的近邻检索 (ANNS) 快速估计当前 query 在各个 LLM 上的质量和成本，再用最开始一小部分在线 query 做一次轻量优化，学习每个 LLM 的预算“影子价格” γ 。之后每个新 query 只需按“估计质量收益 - 预算成本惩罚”打分，选择得分最高的 LLM，从而实现无需训练、低延迟、预算感知的在线路由。	在每个 LLM 的 token 预算约束下，为在线到达的 queries 选择路由模型，使所有已处理 queries 的总体回答质量/性能最大化。
ActiveInfer-Offload [64]	TMC, 2024	Cloud-edge LLM inference 需要同时考虑任务卸载和资源分配；传统 DRL 方法数据效率低，且难以适应动态任务负载和 latency requirement。	使用 active inference 框架建模 LLM inference task offloading 和 resource allocation，在不确定环境下进行在线决策。	优化 cloud-edge LLM inference 的服务性能、资源利用和时延表现。
SLM-LLM-Collab [65]	TMC, 2026	在给定 SLM/LLM 服务能力和缓存状态下，每个请求需要决定本地 SLM 执行、D2D 卸载给邻居 SLM，还是卸载到 edge LLM 与 cached SLM plugins 协同推理。	在短时间尺度上使用 factor graph 和 belief propagation 进行分布式 inference offloading 决策；长期 SLM caching 部分作为请求路由的前置条件，而非本节重点。	在时延和显存约束下最大化长期平均推理准确率。
MGCO [66]	TSC, 2026	移动场景下，generative computation offloading action 会受到历史决策和未来移动状态影响；短程决策容易短视，且传统 RNN/LSTM 对长序列建模能力有限。	使用 Transformer MHA encoder 建模长程移动和历史决策信息；通过候选移动区域约束缩减 action space，提高收敛效率。	在移动 edge-cloud 系统中实现更高效的 generative computation offloading，降低长期服务开销。

从 CEC 角度看，上述工作共同说明，LLM request routing 不能简单沿用传统 task offloading 中“输入数据量 + CPU cycles + deadline”的建模方式。一方面，请求执行时间受到输出长度和 batch 状态影响，具有更强的不确定性；另一方面，请求质量取决于模型能力、prompt 语义和模型选择，而不是单纯取决于计算资源是否充足。因此，CEC 中的 request-level offloading 需要同时感知 semantic complexity、model capability、network condition 和 queue state。

多用户和移动性会进一步放大这一复杂性。用户位置变化会改变接入边缘、链路质量、候选服务节点和区域请求热点；请求执行过程中可能发生 handover，多轮会话的 context 或 KV cache 也可能留在旧 edge。因此，request/session routing 不能只根据当前时隙的队列和链路状态做短视选择，而需要同时考虑未来位置变化、会话连续性、状态位置和模型可用性。未来可以进一步研究 mobility-aware session routing，使请求路径选择同时感知用户移动趋势、后续多轮会话、模型缓存位置和 context/KV 可复用性。

6.2 Multi-model / Multi-agent Workflow Orchestration

随着 LLM 从单轮问答走向复杂推理、代码生成、工具调用、科学问答和长期任务执行，一个 request 到来后，系统不一定只调用一个模型完成回答。更一般的形式是：系统根据请求复杂度和任务类型，动态选择多个模型、多个 agent 或多个工具，并决定它们之间的角色分工、通信拓扑、执行顺序、上下文可见性和结果聚合方式。这一类问题可以被称为 multi-model 或 multi-agent workflow orchestration。

这一类问题与传统 stage-level offloading 或 splittable-request offloading 不同。Stage-level offloading 关注 prefill、decode、verification、diffusion steps 等底层执行阶段如何分布到不同计算节点；而 multi-agent workflow orchestration 关注的是语义层和任务层的协作结构，例如是否需要 planner、solver、critic、verifier、researcher、programmer 等角色，是否采用 Chain-of-Thought、Self-Consistency、LLM Debate、chain topology 或 complete graph 等协作模式，以及每个 agent 背后调用哪个 LLM 或 SLM。换言之，这里的决策对象不是 token、模型层或推理阶段，而是一个 request 的 problem-solving workflow。

这一类问题的核心矛盾是：多 agent 协作可以提升复杂任务的推理能力、鲁棒性和可解释性，但也会带来额外模型调用成本、通信开销、上下文传输开销和调度复杂度。对于 CEC 系统而言，agent 背后的模型、工具、数据库、memory 和 verifier 可能分布在用户设备、边缘服务器、区域云和中心云上。因此，multi-agent workflow orchestration 不仅是算法层的协作模式选择问题，也是计算、通信、状态和隐私的联合编排问题。表 10 列出了相关代表性工作。

从 LLM serving 角度看，RouteLLM 可以被视为 multi-agent routing 的基础形式：它只在强模型和弱模型之间做 query-level routing，尚未涉及 agent 数量、角色分配或协作拓扑。MasRouter 则进一步把 routing 扩展到 multi-agent system configuration，即根据 query 决定采用哪种协作模式、需要多少个 agent、每个 agent 扮演什么角色，以及每个角色调用哪个底层 LLM。Conductor 更进一步，不再只在预定义配置集合中选择，而是训练一个控制器用自然语言生成 workflow，包括任务拆解、模型调用和上下文访问控制。AFLOW 则是将 workflow 表示为代码形式，通过搜索方法，实时根据当前任务执行情况自适应生成新的 workflow。

从 CEC 角度看，multi-agent workflow orchestration 需要被进一步物理化。现有 multi-agent orchestration 工作大多默认所有 worker LLM 都可以通过 API 调用，较少考虑模型、工具和 memory 的实际

表 10 Multi-model / Multi-agent Workflow Orchestration 代表性工作

代表工作	发表在哪 & 时间	问题表述	核心方法	优化目标
RouteLLM [62]	ICLR, 2025	现有 LLM serving 中，所有 query 都调用强模型会带来高成本，而固定使用弱模型又可能损害回答质量；需要根据 query 难度选择合适模型。	使用偏好数据训练一个 query-level router，预测强模型是否会显著优于弱模型；根据阈值进行强/弱模型选择。	在保持目标回答质量的同时减少强模型调用比例，降低推理成本。
MasRouter [67]	ACL, 2025	现有 LLM routing 多解决“给单个 agent 选哪个 LLM”的问题，但在多智能体系统中，真正需要 routing 的还包括协作模式、agent 数量、角色分工以及每个角色对应的底层 LLM。	通过将 query 映射到 latent space 选择合作模式和 agent 数量；使用 structured probabilistic cascade 为 agent 分配角色；再用多项分布为每个角色选择底层 LLM；训练时使用 policy gradient 优化。	为每个 query 选择合适的 multi-agent system configuration，使回复质量和准确率尽可能高，同时控制推理成本。
Conductor [68]	ICLR, 2026	如何训练一个元控制器，让它用自然语言自动编排多个已有 LLM，使整体系统超过任一单个 worker。	使用一个 SLM 作为 Conductor，根据 query 生成自然语言 workflow；每个 step 包含 subtask、model id 和 access list；训练阶段采样多个 candidate workflows，根据最终答案正确性给 reward，并用 GRPO 更新 Conductor。	让 Conductor 学会任务拆解、模型选择、上下文可见性控制和 workflow 生成，使多模型协作后的最终答案尽可能正确。
AFLOW [69]	ICLR, 2025	如何在尽量减少人工设计的情况下，自动生成并优化由多次 LLM 调用组成的 agentic workflow，从而提升 LLM 在问答、代码生成和数学推理等复杂任务上的表现。	AFLOW 将 agentic workflow 表示为代码形式的 LLM 调用图/流程，并把 workflow 优化转化为一个搜索问题。选择已有高分 workflow，让 LLM 修改 prompt 或代码结构生成新 workflow，再执行评估，并把成功/失败经验回传到搜索树中，从而逐步找到更优工作流。	给定任务 T 和评价函数 G，在所有可能的 workflow 中找到使任务表现 G(W,T) 最大的工作流 W
DT-MADTO [70]	TMC, 2025	移动用户产生具有 DAG 依赖关系的复杂任务；设备移动导致链路状态和中间数据流传输代价动态变化，同时多设备并发卸载会造成 flow congestion。	构建 device、physical CEC、digital twin 三层架构；使用 digital twin 预测未来移动和资源状态，并用 DDPG / actor-critic 联合决策 subtask offloading、bandwidth allocation 和 flow waiting time。	最小化任务完成时间和能耗的加权 QoS 指标。

部署位置。但在 CEC 系统中，不同 agent 可能对应不同物理节点：本地设备上的 SLM 可以承担隐私敏感的初步理解任务，近端 edge 上的 LLM 可以承担低时延交互任务，区域边缘或云端的大模型可以承担复杂推理和 verification 任务，外部工具或数据库 agent 也可能位于不同网络位置。因此，agent workflow 的拓扑会直接影响端到端时延、带宽消耗、隐私风险和状态迁移成本。

多用户移动性进一步使 multi-agent workflow orchestration 变得更复杂。用户移动会改变可访问的 edge agents、工具服务和 memory 节点；workflow 中间状态可能在多个 agent 之间传递，如果用户在执行过程中发生 handover，后续 agent 是否继续在旧 edge 执行、是否迁移到新 edge、是否远程访问旧 context，都会影响服务质量。虽然 DT-MADTO [70] 并不是 LLM agent workflow 工作，但其对 DAG 依赖关系、中间数据流和移动预测的建模，为 CEC 中的 multi-agent workflow offloading 提供了可借鉴思路。未来可以进一步研究 mobility-aware agent workflow orchestration，在 request 到达时联合决定协作模式、agent 位置、模型选择、上下文可见性和中间状态传输路径。

6.3 Context / KV / Runtime State Offloading and Migration

LLM inference 与普通计算任务的另一个关键区别在于运行时状态会持续影响后续请求。多轮对话、长上下文服务、prefix reuse 和 agent workflow 会产生 KV cache、prefix cache、session state、agent memory、tool outputs 和 intermediate reasoning traces。这些状态的位置会直接影响后续 TTFT、decode latency、重复计算成本、跨节点传输开销、隐私边界和服务连续性。因此，context/KV/runtime state

offloading 关注的不是计算任务本身去哪执行，而是支撑后续推理的状态应该保留在哪里、何时迁移、是否远程访问、是否复用或是否重新计算。

这一类问题的核心矛盾是：状态迁移可以保持多轮服务连续性、减少重复 prefill 计算并提升生成质量；但迁移或远程访问状态会消耗带宽、显存和存储资源，也可能增加一致性管理和隐私风险。对于长上下文 chatbot，KV cache 和 prefix cache 可能比单次请求本身更有价值；对于 multi-agent workflow，中间结果、工具调用结果和 agent memory 也可能在后续步骤或后续会话中被反复使用。因此，LLM inference offloading 的优化目标正在从 computation-aware 转向 computation-state co-optimization。表 11 汇总了 context、KV 和 runtime state 卸载迁移方向的代表性工作。

表 11 Context / KV / Runtime State Offloading and Migration 代表性工作

代表工作	发表在哪 & 时间	问题表述	核心方法	优化目标
FuseSpill [71]	TPDS, 2026	在显存受限 GPU 上进行 LLM 推理时，KV cache 容易溢出，现有 swap/recompute 策略处理低效，导致吞吐下降和溢出请求延迟增加。	FuseSpill 通过代价模型自适应选择 KV cache 溢出策略，并把溢出或主动卸载的序列调度到辅助 GPU 继续 decode，从而释放主 GPU 显存、提高 prefill/decode 利用率。	在显存受限 GPU 上提高 LLM 推理吞吐量，同时降低发生 KV cache spillover 的请求延迟。
Weaver [72]	USENIX ATC, 2025	多 LLM serving 中 attention computation 和模型实例管理会造成资源压力；attention 和相关状态如何 offload 会影响多模型服务效率。	通过 attention offloading 提升 multi-LLM serving 的资源效率，并缓解多模型服务中的资源压力。	提高多模型服务吞吐和资源利用率。
Mooncake [13]	USENIX FAST, 2025	在长上下文 LLM 聊天服务中，如何用全局分布式 KV cache 复用历史前缀计算，减少昂贵的 prefill GPU 计算，从而在满足 TTFT/TBT 延迟要求的同时提升系统吞吐。	把 LLM 推理拆成 prefill 和 decoding 两个独立资源池，并把集群中 CPU/DRAM/SSD/RDMA 组织成全局分布式 KVCache Store；调度器根据缓存命中、节点负载和 TTFT/TBT SLO，把请求路由到合适的 prefill/decoding 节点。	用更多存储换取更少重复计算，提高长上下文 LLM serving 效率。
DistServe [4]	USENIX OSDI, 2024	Prefill 和 decode 混部执行会造成阶段间干扰，并耦合两阶段资源分配；两阶段分离后，KV handoff 和状态传输成为连接 prefill 与 decode 的关键问题。	将 prefill 和 decoding 分离到不同 worker/resource pool，并根据 TTFT/TPOT 需求分别优化资源分配和并行策略。	最大化满足 TTFT 和 TPOT 约束下的 goodput。
T2DRL [74]	TON, 2026	Mobile edge 难以支撑 long-context LLM agents 的多轮交互，因为 context window 会同时影响准确率、延迟和资源消耗；模型缓存和请求执行策略需要随上下文和使用模式变化。	提出 T2DRL 框架，在训练和测试阶段共同学习 cached models 与服务请求的管理策略，并结合 double Dutch auction 做资源匹配。	降低系统成本，同时保证 long-context LLM agents 在感知和推理任务中的服务性能。
CA-AIGC-Migration [73]	TSC, 2026	用户移动到新边缘节点后，旧节点上的 long context 是有价值但迁移成本较高的状态资产；传统 service migration 迁移完整模型成本过高。	利用 LLM in-context learning 特性，将 model migration 转化为 context migration；在 reward 中联合考虑 context relevance、AoI、计算延迟和传输成本，并用 Transformer DRL 学习迁移策略。	在保持 AIGC 回复质量和准确性的同时降低 context migration 成本和服务时延。

从系统角度看，KV cache、prefix cache 和 context 不再只是 LLM inference 的内部实现细节，而是 CEC 中可以被 offload、cache、migrate 和 schedule 的重要资源。Mooncake 说明了长上下文服务中 KV cache 可以作为全局分布式存储资源被管理；DistServe 说明了 prefill/decode disaggregation 会使 KV handoff 成为连接不同资源池的关键状态传输；FuseSpill [71] 和 Weaver [72] 则从显存约束和 attention offloading 角度说明，runtime state 的位置会直接影响吞吐和时延。

在 CEC 中，context/KV/runtime state offloading 需要进一步考虑边缘节点之间的异构资源和网络差异。对于多轮会话，如果每轮请求都重新 prefill 长上下文，会产生大量重复计算；如果跨节点远

程访问 KV cache, 则会引入额外 RTT 和带宽开销; 如果主动迁移 context 或 KV, 则可能占用边缘链路和存储资源。因此, 系统需要在远程访问、状态迁移、状态压缩、状态丢弃和重新计算之间进行选择。

多用户移动性使这一问题更加突出。用户移动后, 计算路径可以重新选择, 但 long context、KV cache、agent memory 或 session state 可能仍保留在旧 edge。此时系统需要在四种选择之间权衡: 迁移状态到新 edge、远程访问旧状态、重新计算状态, 或丢弃部分状态并降低上下文质量。不同选择会影响后续 decode latency、生成质量、带宽消耗和隐私边界。CA-AIGC-Migration [73] 已经开始从 context migration 的角度研究这一问题, 但对 KV cache placement、remote KV access、prefix cache sharing 和 agent memory migration 的移动 CEC 场景研究仍然不足。未来可以进一步研究 mobility-aware KV and context management, 将用户轨迹预测、session continuation probability、KV reuse probability、edge storage pressure 和链路状态联合纳入决策。

6.4 面向用户移动性的长期会话感知 LLM 请求卸载方案

6.4.1 问题背景与挑战

CEC 任务卸载通常面向普通计算任务: 系统根据输入数据量、计算量、链路状态、队列负载和 deadline 选择执行位置。这类任务的工作量一般可在调度前估计, 任务完成后对后续请求的状态影响也较弱。LLM inference request 则具有明显不同的运行特征: 服务以多轮 session 持续进行, 每轮 request 依赖历史上下文和 KV cache, 同时又生成新的 token 与 KV state; decode step 数在调度时未知, 使真实执行时间、返回流量、显存占用和新增 KV 规模都具有不确定性。因此, CEC 中的 LLM request offloading 不只是普通任务卸载的直接扩展, 而是一个同时包含执行不确定性、状态依赖和长期状态演化的调度问题。

- **LLM 推理任务的卸载挑战。**输出长度是主要不确定来源。调度器在 request 到达时只能观测 prompt、上下文和业务 SLA, 无法预知模型最终生成多少 token。输出长度决定 decode 时间、E2E latency、返回流量和峰值显存压力, 也决定本轮新增 KV blocks 的规模。低估输出长度可能导致 SLA violation 或显存不足; 过度保守则会降低并发和资源利用率。同时, LLM session 具有强历史依赖, 后续 request 的 prefill 成本、状态准备成本和可行动作集合都与此前保留的 KV cache 有关。
- **用户移动性下的状态位置挑战。**现有 CEC 移动任务卸载研究多面向普通计算任务, 主要处理接入链路变化、任务迁移和结果回传路径; 已有 LLM 推理调度工作则更多关注静态或低移动性的服务集群, 尚未充分讨论多基站 CEC 中用户移动、session 连续性和 KV 状态位置之间的耦合关系。对于 LLM 推理任务, 移动性不仅改变后续 request 的入口节点, 还会改变历史 KV cache 与

未来请求入口之间的相对位置：当用户在多轮 session 中从一个基站覆盖区移动到另一个覆盖区时，历史 KV cache 可能仍保留在旧计算节点。系统因此需要同时决定当前计算节点和 KV 获取方式，即就地复用、跨节点同步或在目标节点重算。

6.4.2 耦合关系与方案概览

上述两个挑战相互交织，使得调度策略难以用单次 request 的局部最优来刻画。移动性决定未来 request 从哪里进入系统，输出长度决定当前 request 生成多少新的 KV state；二者共同影响未来 KV cache 与请求入口之间的相对位置、同步字节数、显存压力和 SLA 风险。若仅优化当前 request，系统可能得到较低即时成本，却在后续多轮中产生额外状态同步或重算开销。因此，本节将 CEC LLM request offloading 建模为面向 session 的长期成本优化问题。

本文采用“预测感知调度-显式可行性过滤-分布式 KV 管理”协同框架。调度层利用移动预测和输出长度预测构造长期价值输入，在满足 SLA、显存和 KV 可用性约束的动作集合中选择计算节点与 KV 获取方式；KV 管理层则将调度结果落实到 block 级状态查询、同步、复用、重算和副本维护，从而同时优化当前 request 的服务质量和后续 session 的状态复用成本：

- **不确定性感知的状态预测。** Mobility predictor 根据用户历史轨迹、当前入口节点和相邻 request 间隔预测未来入口分布；Output length predictor 根据 prompt、上下文长度、任务类型和会话历史预测输出长度分布。二者分别刻画未来请求入口和 decode 长度不确定性，使状态位置、返回流量、显存压力和新增 KV 规模在决策阶段提前进入估计。
- **约束感知的长期路由决策。** 调度器将“计算节点 + KV 获取方式”作为联合动作，把请求路由与 KV reuse、sync、recompute 选择放入同一决策空间。随后，系统根据 E2E SLA、显存余量、节点可用性和 KV 版本一致性构造可行动作集合，将明显违反服务约束或状态约束的动作提前屏蔽；Double DQN 仅在该集合内评估长期成本，使策略在保证当前请求可服务的前提下，进一步选择对后续 session 状态复用更有利的执行位置和 KV 获取方式。
- **面向执行效率的 KV 管理。** KV manager 在底层维护 block directory 和版本化副本，将上层动作落实为 block 级查询、同步、保留/驱逐和混合恢复；其中混合恢复可让部分 KV 同步与部分上下文重算流水重叠，以减少通信或计算单独等待造成的 bubble，并可结合移动预测进行 proactive placement。

6.4.3 系统场景与在线处理流程

本项目包含三个部署相同模型的推理节点，每个节点既是请求入口，也是候选计算节点；两条链路为 100 Gbps，另一条为 25 Gbps。对于一个 LLM session，实际 request 数量在在线服务时通常未知；下文用 K 表示分析窗口内的 session 长度。系统在用户移动和输出长度未知的条件下，联合选择计

算节点与 KV 获取方式，并在满足 E2E latency SLA 和显存约束的前提下最小化长期成本。

为了支撑分布式路由，系统后台维护一个 CEC network/state info manager，可实现为逻辑上的共享元数据目录而非集中式调度器。每个推理节点周期性上报本地队列长度、GPU 显存余量、KV cache 水位、模型/adapter 可用性和执行统计；网络监测模块上报节点间有效带宽、RTT、链路拥塞和可用性；KV manager 则同步 prefix hash、block 版本、block 大小和所在节点等目录信息。入口节点的 router 在 request 到达时读取该目录的最近快照，构造 $\mathbf{L}_k, \mathbf{M}_k, \mathbf{W}_k$ 和 C_k ，并将其用于 per-action estimator、SLA/memory/KV 过滤以及 Double DQN 的状态输入。由于该目录以周期性快照工作，读到的状态可能存在 staleness，因此过滤条件中的安全裕量 $\Delta_k(a)$ 需要吸收状态同步延迟和测量误差。

主要符号。

符号	含义
$k, K, \mathcal{N}$	当前请求编号、分析窗口内的 session 长度和候选节点集合
i, j, n, s	入口节点、目标/计算节点、一般节点和 KV 同步源节点索引
$U_k, J_k, \Delta\tau_k$	当前入口、所选计算节点和相邻请求间隔
$P_{ij}(\Delta\tau_k), b_k$	入口转移概率和多步移动预测分布
$x_k, Y_k, Y_k^{\text{obs}}, \rho_k(y), \hat{Y}_k^{(p)}$	请求特征、实际/观测输出长度、长度预测分布和第 p 分位预测
$\mathcal{B}_k, \mathcal{B}_{k,n}$	当前完整 KV block 集合和节点 n 已有 block 集合
$v_{k,n}, C_k$	节点 n 的最高连续 KV 版本和版本化多副本状态，用于 KV 可用性检查
$\mathcal{M}_{k,j}, q_b, \Delta\mathcal{B}_k$	节点 j 缺失 blocks、block b 的大小和本轮新增 blocks
ψ_k^{cache}, H	KV 副本保留/驱逐策略和完整系统状态转移函数
$a_k = (J_k, Z_k)$	计算节点与 KV 获取方式组成的联合动作
a, a', y, u'	当前候选动作、下一候选动作、输出长度取值和下一入口节点取值
Z_k	KV 获取方式：reuse、sync 或 recompute
$\mathcal{A}_k^F(s_k)$	满足 SLA、显存、版本和节点可用性约束的动作集合
$\mathcal{N}_k^{\text{avail}}, \chi_k^{\text{KV}}(a)$	当前可用节点集合和动作 a 的 KV 版本/block 可用性指示函数
$D_k^{\text{input}}, D_k^{\text{request}}$	输入数据量和跨节点请求/响应总字节数
$D_k^{\text{KV}}, BW_{i,j}^{\text{eff}}$	KV 同步字节数和节点间实测有效带宽
$L_k^{\text{new}}, L_k^{\text{ctx}}$	未被 KV 覆盖的新输入长度和完整上下文长度
$\tau_j^{\text{prefill}}(L)$	节点 j 处理 L 个 prefill tokens 的实测时间
$T_k^{\text{comm}}, T_k^{\text{state}}, T_k^{\text{prefill}}$	请求通信、KV 状态准备和 prefill 时间
$T_k^{\text{queue}}, T_k^{\text{decode}}, T_k^{\text{return}}$	排队、decode 和结果返回时间
T_k^{E2E}	从请求发出到全部输出返回的端到端时延
$C_k^{\text{eviction}}, w_1, \dots, w_4, \bar{c}_k, c_k$	跨 session KV 驱逐外部性、成本权重、条件即时成本和当前期望成本
$\hat{T}_k^{\text{E2E}}, \Delta_k(a), SLA_k$	预测端到端时延、动作时延安全裕量和请求 SLA 上限
$\hat{M}_k, M_k^{\text{reserve}}, M_{J_k}^{\text{free}}$	预测显存需求、安全预留显存和计算节点剩余显存
$s_k, s_{k+1}^{a,y,u'}, s_{k+1}^{\text{obs}}$	当前系统状态、模型化下一状态和观测下一状态

符号	含义
$\mathbf{L}_k, \mathbf{M}_k, \mathbf{W}_k$	系统状态中的负载、显存和链路状态向量
$\pi, \gamma, \mathcal{Q}^*$	路由策略、折扣因子和最优状态-动作价值
$\mathcal{Q}_\theta, \mathcal{Q}_{\bar{\theta}}$	Double DQN 在线网络和目标网络
$a_k^{\text{greedy}}, a_k^*, a_{k+1}^{\text{sel}}$	Greedy 动作、在线执行动作和下一状态动作选择
V_{k+1}, z_k	目标网络估值和 Bellman 训练目标

图 19 概括了单个 request 到达后的在线处理闭环，具体模块关系见图注说明。

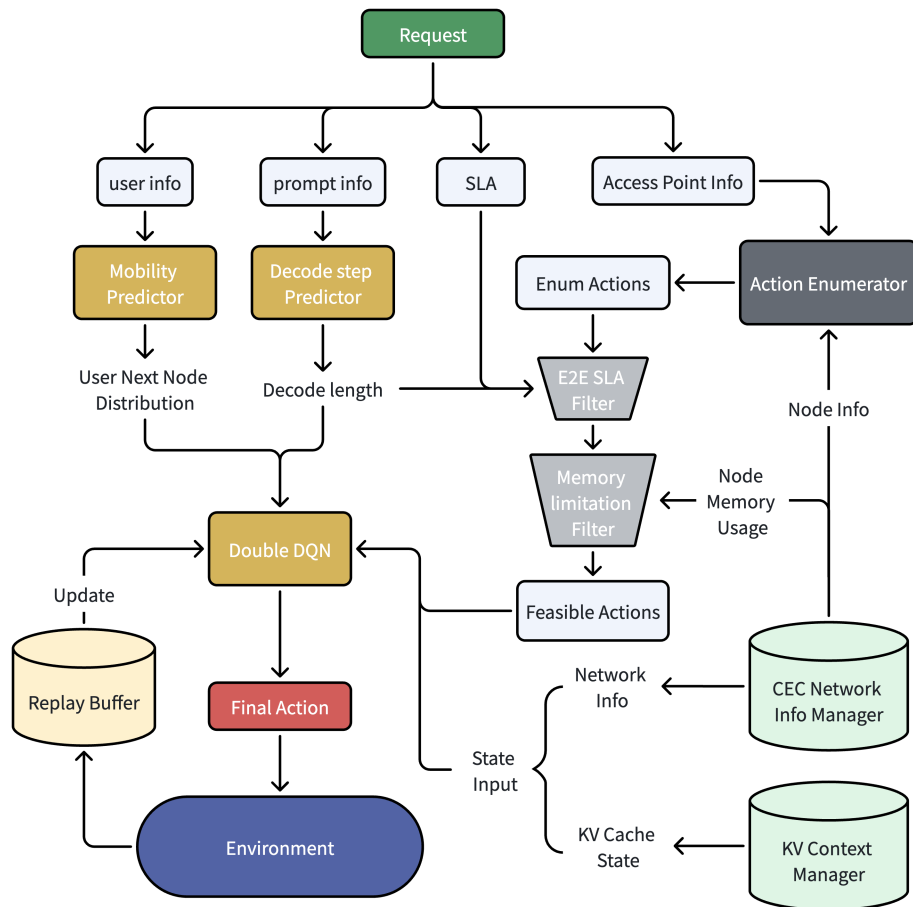


图 19 单个请求到达后的长期成本感知卸载决策流程。Request 被拆分为用户信息、prompt 信息和 SLA 信息：用户信息进入 mobility predictor 生成未来入口分布，prompt 信息进入 output length predictor 估计 decode 长度，CEC network/state info manager 与 KV context manager 分别提供节点负载、链路带宽、剩余显存以及 KV block 位置/版本等运行状态。State builder 将移动预测、长度预测、SLA、系统遥测和 KV 状态汇聚为 feature bundle；action enumerator 枚举“计算节点 + KV 获取方式”候选，per-action estimator 对每个候选动作估计请求传输、KV 同步/重算、prefill、decode 和显存占用。E2E SLA filter 与 Memory/KV filter 先处理时延、显存和 KV 可用性硬约束，得到 feasible actions 作为 DQN action mask；Double DQN router 只在可行动作中比较长期成本并输出 final action。环境执行后的实际 E2E 时延、输出长度、KV 增量、传输量和显存变化写入 replay buffer，用于回传更新预测器、估计器和路由策略。

6.4.4 移动、长度与 KV 状态

令 $U_k \in \mathcal{N}$ 为第 k 个请求的入口节点，移动模型给出

$$P_{ij}(\Delta\tau_k) = \Pr(U_{k+1} = j \mid U_k = i, \Delta\tau_k).$$

固定请求间隔时可采用 Markov chain，间隔变化较大时可采用 continuous-time 或 semi-Markov 模型；LSTM 或 Transformer 则输出多步入口分布 b_k 。这一设计借鉴 MGCO 对历史决策和未来移动状态的利用 [66]。

输出 token 数 Y_k 在决策时未知，长度预测器根据请求特征 x_k 给出回复长度分布

$$\rho_k(y) = \Pr(Y_k = y \mid x_k), \quad \hat{Y}_k^{(p)} = \inf \left\{ y : \sum_{\ell \leq y} \rho_k(\ell) \geq p \right\}.$$

其中 $\hat{Y}_k^{(p)}$ 是第 p 分位长度预测，用于 SLA 与显存等硬约束过滤；完整分布 ρ_k 则用于长期价值估计。类似的长度预测已用于不可预测请求调度 [61]。长度预测不仅用于当前请求的 E2E latency 和显存需求估计，也进入动作价值评估：长输出会增加 decode 时间、返回流量、KV 增量和后续显存压力，从而改变 DQN 对不同计算节点与 KV 获取方式的偏好。

KV 允许存在多个版本化副本：

$$\mathcal{C}_k = \{(n, v_{k,n}, \mathcal{B}_{k,n}) \mid n \in \mathcal{N}\},$$

其中 $v_{k,n}$ 和 $\mathcal{B}_{k,n}$ 分别表示节点 n 的最高连续版本和 block 集合；节点 j 缺失的 blocks 为 $\mathcal{M}_{k,j} = \mathcal{B}_k \setminus \mathcal{B}_{k,j}$ 。同步后不默认删除源副本，保留或驱逐由 cache policy 决定。

6.4.5 联合动作、成本与约束

动作定义为

$$a_k = (J_k, Z_k), \quad Z_k \in \{\text{reuse}, \text{sync}, \text{recompute}\},$$

其中 J_k 是计算节点， Z_k 是该节点获取 KV 的方式。若 $U_k \neq J_k$ ，请求自动转发；存在多个副本时，系统自动选择同步时间最短的源节点。由于动作价值同时考虑当前入口、预测未来入口和 KV 状态位置，策略可以选择并非当前入口最近的计算节点，以减少后续多轮 request 的状态同步或重算成本。reuse 仅在目标节点 J_k 已经持有当前请求所需的连续 KV prefix 或 blocks 时可行；若目标节点缺失必要 block，则该节点只能选择 sync 或 recompute。

请求通信、KV 状态准备和 prefill 时间分别为

$$T_k^{\text{comm}}(a_k) = \mathbf{1}\{U_k \neq J_k\} \frac{D_k^{\text{input}}}{BW_{U_k, J_k}^{\text{eff}}},$$

$$T_k^{\text{state}}(a_k) = \begin{cases} \frac{\sum_{b \in \mathcal{M}_{k, J_k}} q_b}{BW_{s, J_k}^{\text{eff}}}, & Z_k = \text{sync}, \\ 0, & Z_k \in \{\text{reuse}, \text{recompute}\}, \end{cases}$$

$$T_k^{\text{prefill}}(a_k) = \begin{cases} \tau_{J_k}^{\text{prefill}}(L_k^{\text{new}}), & Z_k \in \{\text{reuse}, \text{sync}\}, \\ \tau_{J_k}^{\text{prefill}}(L_k^{\text{ctx}}), & Z_k = \text{recompute}. \end{cases}$$

其中 $\tau_{J_k}^{\text{prefill}}(L)$ 是节点 J_k 处理 L 个 prefill tokens 的实测时间；因此 recompute 会处理完整 context，而 reuse/sync 只处理尚未被 KV 覆盖的新输入。完整请求时延为

$$T_k^{\text{E2E}} = T_k^{\text{comm}} + T_k^{\text{queue}} + T_k^{\text{state}} + T_k^{\text{prefill}} + T_k^{\text{decode}}(Y_k) + T_k^{\text{return}}(Y_k),$$

它已经包含首 token 响应阶段。给定输出长度 y 时，动作 a_k 的条件即时成本定义为

$$\bar{c}_k(s_k, a_k, y) = w_1 T_k^{\text{E2E}}(s_k, a_k, y) + w_2 D_k^{\text{request}}(a_k, y) + w_3 D_k^{\text{KV}}(a_k) + w_4 C_k^{\text{eviction}}(a_k).$$

由于输出长度未知，决策时的当前期望成本为

$$c_k(s_k, a_k) = \mathbb{E}_{Y_k \sim \rho_k} [\bar{c}_k(s_k, a_k, Y_k)].$$

其中 T_k^{E2E} 衡量完整请求时延； D_k^{request} 是入口与计算节点不同时传输的输入及输出字节数； D_k^{KV} 是为当前动作同步的 KV 字节数； C_k^{eviction} 衡量当前动作占用显存并驱逐其他 session 的 KV blocks 后，对后续请求造成的额外 recompute、CPU memory swap-in 或远端同步开销。权重 $w_1, \dots, w_4$ 用于完成单位归一化并反映时延、带宽和跨 session 缓存外部性的重要性。请求完成后，系统用实际 Y_k 和实测开销生成 replay 样本。

系统使用输出长度高分位预测 $\hat{Y}_k^{(p)}$ 构造可行动作集合

$$\mathcal{A}_k^F(s_k) = \left\{ a : \hat{T}_k^{\text{E2E}}(a, \hat{Y}_k^{(p)}) + \Delta_k(a) \leq SLA_k, \quad \hat{M}_k(a, \hat{Y}_k^{(p)}) + M^{\text{reserve}} \leq M_{J_k}^{\text{free}}, \quad J_k \in \mathcal{N}_k^{\text{avail}}, \quad \chi_k^{\text{KV}}(a) = 1 \right\},$$

其中 $\mathcal{N}_k^{\text{avail}}$ 表示当前可用且部署了匹配模型/adapter 版本的节点集合， $\chi_k^{\text{KV}}(a)$ 表示 KV 获取方式的版本与 block 可用性检查：reuse 要求 J_k 已有所需连续 prefix/blocks，sync 要求至少存在可同步的版

本一致副本且链路可用，recompute 要求目标节点可访问完整上下文并有足够计算与显存余量。本文对输出长度采用分层处理：可行动作过滤使用高分位预测 $\hat{Y}_k^{(p)}$ ，以降低输出长度低估造成的 SLA violation 与显存不足；长期价值估计则保留长度分布 ρ_k ，对不同输出长度造成的即时成本和下一状态变化取期望，避免策略过度保守。

Greedy baseline 仅在可行动作中选择当前成本最低者：

$$a_k^{\text{greedy}} = \arg \min_{a \in \mathcal{A}_k^F(s_k)} c_k(s_k, a).$$

它能够响应当前负载和链路状态，但不考虑本次同步或重算对后续 KV 复用的影响，因此作为本文长期策略的短视对照，也作为 RL 异常时的安全回退。

6.4.6 长期优化与动作掩码 Double DQN

将第 k 个请求到达时的状态写为

$$s_k = (k, U_k, b_k, \rho_k, \mathcal{C}_k, \mathbf{L}_k, \mathbf{M}_k, \mathbf{W}_k),$$

分别描述入口及其预测、输出长度分布、KV 状态、负载、显存和链路。有限时域优化问题为

$$\begin{aligned} \min_{\pi} \quad & \mathbb{E}_{\pi, U, Y} \left[\sum_{k=1}^K \gamma^{k-1} c_k(s_k, a_k) \right] \\ \text{s.t.} \quad & a_k = \pi(s_k), \quad a_k \in \mathcal{A}_k^F(s_k), \\ & U_{k+1} \sim P(\cdot \mid U_k, \Delta\tau_k), \\ & Y_k \sim \rho_k(\cdot \mid x_k). \end{aligned}$$

其中 $c_k(s_k, a_k)$ 是对当前输出长度分布取期望后的即时成本。为避免在优化问题中展开完整系统动力学，记

$$s_{k+1}^{a, y, u'} = H(s_k, a_k, y, u', \Delta\mathcal{B}_k(a_k, y), \psi_k^{\text{cache}})$$

为在当前动作 a_k 、实际输出长度 y 、下一入口 u' 和 KV 增量共同作用后的下一状态。该转移函数内部同时更新入口节点、负载、显存、链路状态和版本化 KV 状态；其中 KV 状态由本轮动作、真实输出长度、新增 KV blocks 以及副本保留/驱逐策略共同决定。对应的理论最优状态-动作价值满足

$$\begin{aligned} \mathcal{Q}^*(s_k, a_k) = & \sum_y \rho_k(y) [\bar{c}_k(s_k, a_k, y) \\ & + \gamma \sum_{u' \in \mathcal{N}} P_{U_k u'}(\Delta\tau_k) \min_{a' \in \mathcal{A}_{k+1}^F(s_{k+1}^{a, y, u'})} \mathcal{Q}^*(s_{k+1}^{a, y, u'}, a')] \end{aligned}$$

移动性既影响当前转发成本，也通过 $P_{U_k u'}$ 影响未来价值；输出长度不仅影响当前 decode 和显存，还通过 $\Delta \mathcal{B}_k(a_k, Y_k)$ 改变后续 KV 分布。上述 Bellman 形式把条件即时成本和下一状态价值放在同一个 $\rho_k(y)$ 期望中，因此单位始终为 cost；而 action mask 中的 $\hat{Y}_k^{(p)}$ 只用于硬约束过滤。这延续了 QoS-LLM-Router 的长期 QoS 优化 [60] 和 MGCO 的非短视移动卸载思想 [66]，并将 CA-AIGC-Migration 的移动 context 管理 [73] 细化到版本化 KV blocks。

上式定义了希望逼近的 Q^* 。对短 session、离散状态且已知真实轨迹的小规模实验，可用有限时域动态规划或搜索得到离线 oracle；但是一般场景中同一 session 的 request 数量 K 不确定，且存在动作序列和状态爆炸，因此采用 Dueling Double DQN 近似求解。DQN 对所有固定动作输出 Q 值，在执行 $\arg \min$ 前将 $a \notin \mathcal{A}_k^F(s_k)$ 的 Q 值置为 $+\infty$ ；

Replay buffer 中记录的是执行后的实际转移 s_{k+1}^{obs} ，其中已经包含本轮真实 Y_k 、KV 增量、显存变化和下一入口。Double DQN 在该观测下一状态上先用在线网络选择动作，再用目标网络评价：

$$\begin{aligned} a_{k+1}^{\text{sel}} &= \arg \min_{a' \in \mathcal{A}_{k+1}^F(s_{k+1}^{\text{obs}})} Q_{\theta}(s_{k+1}^{\text{obs}}, a'), \\ V_{k+1} &= Q_{\bar{\theta}}(s_{k+1}^{\text{obs}}, a_{k+1}^{\text{sel}}), \\ z_k &= \bar{c}_k(s_k, a_k, Y_k^{\text{obs}}) + \gamma V_{k+1}. \end{aligned}$$

即 replay 训练目标等于“观测输出长度下的即时成本 + 折扣后的观测下一状态价值”。若采用模型化 roll-out，则不使用单个观测下一状态，而是按照 Bellman 式对输出长度 y 和下一入口 u' 共同取期望，同时计算对应的 $\bar{c}_k(s_k, a_k, y)$ 与下一状态价值。在线动作则为 $a_k^* = \arg \min_{a \in \mathcal{A}_k^F(s_k)} Q_{\theta}(s_k, a)$ 。若模型异常则回退至可行动作中的 Greedy 方案；若集合为空，则排队、降级或拒绝。

6.4.7 分布式 KV 管理

KV manager 是上层卸载策略和底层 KV runtime 之间的执行层。上层 RL 只决定“计算节点 + KV 获取方式”，而 KV manager 负责把 reuse、sync 和 recompute 落到 block 级目录查询、源节点选择、状态传输和副本管理上。借鉴 Mooncake 的 KVCache-centric 全局存储思想 [13]，系统建立分布式 block directory，记录 model/adaptor 版本、prefix hash、token 区间、block 版本、大小和所在节点。执行 sync 前，目标节点先查询已有 blocks，只对缺失部分选择一个或多个源节点并传至计算节点；这也吸收了 prefill/decode 分离系统中 KV handoff 的思路 [4]。同步不等于删除，源 blocks 会继续保留，直至 cache policy 根据 TTL、显存压力、未来入口与复用概率、增量同步收益和重建成本决定驱逐。

在被动执行上层动作之外，KV manager 还可以利用移动预测做 proactive KV placement。具体而言，系统可根据未来入口分布、session continuation probability 和 block 复用概率，提前把高价值 KV blocks 复制到未来更可能访问的计算节点，或放置到与这些节点网络距离更近的邻近节点/共享

存储中。这样后续 request 到达时可以直接 reuse，或只需要少量增量同步，从而把原本位于实时路径上的 recompute 和大规模迁移提前到后台完成。Mooncake 将 CPU/DRAM/SSD/RDMA 组织为全局 KVCache Store，并按 KV 本地性、节点负载和 SLO 调度请求 [13]；在 CEC 场景中，这一思路进一步与用户移动预测结合，形成面向未来入口的 KV 预放置。

当显存或链路成为瓶颈时，KV manager 还负责 KV 生命周期管理。热 blocks 优先保留在 GPU memory，温/冷 blocks 可下沉到 CPU memory、远端内存或本地存储；跨节点同步前，可对低复用概率或可容忍精度损失的 KV 进行压缩或量化，以减少传输字节数和显存占用。相关 KV compression/streaming 工作如 CacheGen 说明，压缩 KV cache 可以降低长上下文服务中的状态传输与存储成本 [75]。对于质量约束允许的场景，KV manager 还可以结合 token importance、attention score 或 session 摘要，对低价值 token/block 做剪枝或降精度保留。

最后，KV manager 可以把一次上层 sync 动作细化为混合恢复：对带宽开销较低或重算代价较高的缺失 blocks 执行迁移，对其余 blocks 从目标节点已有的连续 prefix/checkpoint 开始重算。两类操作不必串行完成，系统可以按 block 组织流水，使远端 KV 同步、局部 recompute 和后续 prefill/decode 尽量重叠，从而减少单纯等待传输或等待重算造成的 bubble。上述机制均不扩大上层 RL 动作空间，但会改变实际 D_k^{KV} 、 $C_k^{eviction}$ 和状态准备时间；这些观测结果会回写到成本模型和 replay buffer，使后续路由逐步学习到不同 KV 管理策略的真实收益。

6.4.8 在线闭环与实验设置

请求完成后，系统用实际输出长度、E2E latency、KV 增量、传输量和显存变化更新长度/移动预测器、成本模型与 replay buffer。

主对比设置就近路由、当前成本 Greedy 和本文 RL 三种策略；补充实验只增加“全程无移动”对照，以及 10%/20%/30% 改发比例的敏感性分析。核心指标为 SLA 下最大并发数、平均/P95 E2E latency 和 SLA violation；KV 传输量、prefix/block 命中率、同步/重算次数与显存峰值仅用于解释结果。

6.5 小结

总体来看，CEC 中的 LLM inference task offloading 已超出传统“数据量、CPU cycles 与 deadline”驱动的路由节点选择问题。系统既要路由完整请求和连续 session，也要编排多模型/多 agent workflow，并管理 context、prefix cache、KV cache 和 agent memory。用户移动进一步改变请求入口、网络路径和历史状态位置；输出长度不确定性则影响 decode 时间、返回流量、显存占用和新增 KV 规模。二者共同使一次卸载动作持续影响后续多轮服务。

现有研究呈现三条主线：request/session routing 从就近执行转向质量、SLA、队列和显存联合感

知；multi-model/multi-agent orchestration 从单模型选择扩展到角色、模型、工具和中间状态的跨节点编排；runtime-state management 则将 KV 和 context 视为可驻留、复制、同步、驱逐或重算的系统资源。三者共同表明，LLM 卸载需要从 computation-aware 走向 computation-state co-optimization。

面向项目场景，本章构建“状态同步-长度与移动预测-显式可行性过滤-RL 长期路由-block-level KV 管理-在线反馈”的闭环。CEC network/state info manager 为过滤和决策提供负载、链路、显存与 KV 目录快照；移动概率和输出长度分布共同进入长期价值估计，其中输出长度高分位用于 SLA/显存等硬约束过滤，完整长度分布用于 Bellman 期望。动作掩码 Double DQN 在满足 SLA、显存、节点可用性和 KV 版本一致性的候选中，联合选择计算节点与 reuse、sync 或 recompute 方式，从而避免 Greedy 策略的状态黏附和无效数据移动。

KV 管理层则把上层动作落实到 block directory 查询、缺失 blocks 同步、副本保留/驱逐和混合恢复等机制，并可进一步结合移动预测进行 proactive placement、KV 压缩/量化和低价值 block 剪枝。后续工作可进一步研究请求执行期间 handover、多用户资源竞争、prefill/decode 分离和 CEC-native agent workflow，并探索移动与输出长度预测同卸载 regret 的联合训练。由此，CEC 中的 LLM inference offloading 将从单次执行位置选择发展为面向请求、workflow 和版本化运行时状态的长期协同优化。

七、三类算法的协同优化框架

前文分别从模型切分部署、业务感知 batching 和 LLM inference task offloading 三个角度讨论了 CEC-LLM 推理系统的关键算法。真实系统中，这三类算法并不是彼此独立的模块：模型切分部署决定模型组件、服务实例和推理阶段的空间分布；业务感知 batching 决定请求、prefill chunk 和 decode token 在实例内部或不同阶段资源池中的时间组织；task offloading 决定完整请求、复杂 workflow 以及 context/KV/runtime state 在端、边、云之间的路由、迁移和复用。三者共享模型位置、队列长度、KV cache 位置、链路状态、用户移动趋势和 SLO slack 等同一组系统状态，因此任何一类算法的局部优化都可能改变另外两类算法的可行域和收益。

本章不再重新划分第四、五、六章的所有单点方法，而是围绕“三类算法之间怎样协同”组织已有工作。具体而言，协同优化关注的不是某个模型如何单独切分、某个 batch size 如何单独选择，或某个请求如何单独卸载，而是这些决策在多时间尺度、多资源层级和多运行时状态下如何形成闭环。已有工作已经从局部角度揭示了这种耦合关系：model caching 会影响 request offloading 的可选路径，prefill/decode disaggregation 会改变 batching 和 KV handoff，KVCache-centric serving 会把状态位置纳入 request scheduling，移动场景下的 context migration 又会反过来影响长期 session routing。

本文将 CEC-LLM 推理中的三类算法协同关系划分为四类：

1. **Model / Service Placement – Request Offloading 协同**：低频模型缓存、服务部署和高频请求路由之间的双时间尺度闭环。
2. **Model Partitioning – Batching / Scheduling 协同**：模型切分、阶段解耦、continuous batching 和 serving scheduling 之间的容量匹配与时延控制。
3. **Request / Workflow Routing – Runtime State Management 协同**：请求路径、agent workflow、KV cache、prefix cache、session state 和 context migration 之间的状态感知路由。
4. **Mobility-aware Hierarchical Control**：用户移动、负载热点变化、handover 和状态位置变化共同驱动的跨层闭环控制。

这四类关系不是新的孤立算法类别，而是前文三类算法在系统运行时的交叉界面。第一类主要连接第四章的模型/服务部署和第六章的请求卸载；第二类连接第四章的模型切分和第五章的 batching；第三类连接第五章的状态感知调度和第六章的请求/workflow offloading；第四类则把移动 CEC 中的外部扰动纳入统一控制框架。表 13 汇总了与这些协同关系直接相关的代表工作、主归属和方法关键词。

表 13 CEC-LLM 三类算法协同优化相关工作总览

序号	代表工作	发表在哪	主归属	相关机制	方法关键词
----	------	------	-----	------	-------

1	SLM-LLM-Collab [65]	TMC 2026	Model / Service Placement – Request Offloading	SLM caching; D2D / edge inference offloading	长期缓存 SLM 或插件模型，短期在 local、D2D 和 edge LLM 之间选择卸载路径，体现模型缓存与请求卸载的双时间尺度协同
2	T2DRL [74]	TON 2026	Model / Service Placement – Request Offloading; Mobility-aware Control	long-context model caching; request management	在 mobile edge 中联合管理 cached models、service requests 和资源匹配，使 long-context LLM agent 服务同时感知模型缓存、上下文长度和边缘资源
3	H-MEC-Placement [76]	TMC 2025	Model / Service Placement – Request Offloading	service placement; task scheduling; resource allocation	面向 general MEC 说明服务部署、任务调度和资源分配在规划周期内存在强耦合，可作为 LLM 服务布局协同的基础参考
4	LVM-Offload [77]	INFOCOM 2026	Model / Service Placement – Request Offloading	foundation model selection; step offloading; resource allocation	在 foundation model 服务中联合考虑模型选择、推理步数、任务卸载和资源分配，说明模型能力选择会改变后续卸载路径
5	DistServe [4]	OSDI 2024	Model Partitioning – Batching / Scheduling	prefill/decode disaggregation; phase-specific workers	将 prefill 和 decode 分离到不同 worker pool，并分别优化并行策略、batching 和资源分配，收益取决于 KV handoff 与阶段队列协调
6	Splitwise [27]	ISCA 2024	Model Partitioning – Batching / Scheduling	prompt pool; token pool; KV transfer	把 prompt computation 和 token generation 放到不同机器池，利用阶段资源画像差异，同时需要处理阶段间 KV transfer 和调度边界
7	Orca [1]	OSDI 2022	Model Partitioning – Batching / Scheduling	iteration-level scheduling; selective batching	在 decode iteration 边界更新 active batch，说明 serving 调度粒度会直接影响模型实例利用率和请求等待时间
8	vLLM [2]	SOSP 2023	Model Partitioning – Batching / Scheduling; Runtime State Management	PagedAttention; KV block management	通过 KV block 管理支撑 continuous batching，使 batching 容量不只取决于计算资源，也取决于 KV 显存碎片和状态管理能力
9	Llumnix [49]	OSDI 2024	Model Partitioning – Batching / Scheduling; Runtime State Management	live request migration; instance rescheduling	在多个 LLM serving 实例之间迁移运行中请求及其状态，体现实例调度、batch 负载均衡和状态迁移之间的耦合
10	Mooncake [13]	USENIX FAST 2025	Request / Workflow Routing – Runtime State Management	distributed KV cache; global scheduling; PD disaggregation	将 KV cache 作为全局状态层管理，调度器按 KV 本地性、节点负载和 SLO 选择 prefill/decode 路径
11	Preble [46]	ICLR 2025	Request / Workflow Routing – Runtime State Management	distributed prompt scheduling; prefix/KV locality	把 prefix/KV cache 命中和负载均衡共同纳入请求分配，使 routing action 与后续 prefill 复用和 batch 组成相关
12	CachedAttention [78]	USENIX ATC 2024	Request / Workflow Routing – Runtime State Management	attention / prefix cache reuse	说明 attention/prefix cache 的复用机会会改变请求服务路径和重复计算成本，是状态感知调度的重要依据
13	FuseSpill [71]	TPDS 2026	Request / Workflow Routing – Runtime State Management	KV spillover; auxiliary GPU scheduling	显存受限时根据代价模型选择 KV cache spillover，并把溢出序列调度到辅助 GPU，改变原有 decode 执行位置
14	QoS-LLM-Router [60]	TMC 2025	Request / Workflow Routing – Runtime State Management	QoS-aware routing; queue / memory state	将 edge LLM expert routing 建模为长期 QoS 优化，路由决策同时感知质量、响应长度、队列状态和 GPU memory
15	CES-CB [61]	TSC 2025	Request / Workflow Routing – Runtime State Management	request cost prediction; edge batch selection	利用 prompt embedding 和历史执行数据预测请求成本，在 edge/cloud 间选择请求集合，说明 routing 会影响队列阻塞和 batch 组成

16	MasRouter [67]	ACL 2025	Request / Workflow Routing – Runtime State Management	multi-agent routing; model / role selection	根据 query 决定 multi-agent 协作模式、agent 数量、角色和底层 LLM；在 CEC 中需要进一步映射为 workflow offloading
17	Conductor [68]	ICLR 2026	Request / Workflow Routing – Runtime State Management	natural-language workflow control	训练控制器生成多模型 workflow、subtask 和上下文访问范围，提示 agent workflow 需要同时考虑模型选择和状态可见性
18	AFLOW [69]	ICLR 2025	Request / Workflow Routing – Runtime State Management	automatic agentic workflow search	将 agentic workflow 表示为代码形式的 LLM 调用图，并用执行反馈搜索更优 workflow；在 CEC 中还需加入物理节点和状态路径
19	MGCO [66]	TSC 2026	Mobility-aware Hierarchical Control	mobility-aware generative computation offloading	用 Transformer MHA encoder 建模历史决策和未来移动状态，说明请求卸载不能只根据当前时隙状态短视决策
20	DT-MADTO [70]	TMC 2025	Mobility-aware Hierarchical Control	digital twin; DAG offloading; mobility prediction	通过 digital twin 预测移动和资源状态，联合决策 DAG 子任务卸载、带宽分配和中间数据等待时间
21	CA-AIGC-Migration [73]	TSC 2026	Mobility-aware Hierarchical Control; Runtime State Management	context migration; mobility-aware service migration	将完整模型迁移转化为 long context migration，在回复质量、AoI、计算延迟和传输成本之间做权衡

7.1 三类算法的基本耦合关系

模型切分、batching 和 task offloading 共享同一组系统状态，包括模型副本位置、模型切分模板、prefill/decode worker 资源、batching window、实例队列长度、KV cache 位置、prefix cache 命中情况、session state、链路质量和用户移动趋势。模型切分如果改变 prefill 或 decode 的执行位置，就会改变 batching 的阶段容量和 KV handoff 成本；batching 如果改变请求等待时间和 token 生成节奏，就会影响 request routing 观察到的实例时延和 admission 边界；request/session routing 如果改变流量入口和服务实例选择，又会反过来改变模型实例负载、prefix cache 命中率和后续 context/KV 迁移成本。

因此，三类算法之间不是简单的前后级联关系，而是多时间尺度上的闭环关系。低频部署层决定模型切分、模型副本、SLM/LLM 插件和服务能力布局；中频 serving 层根据 workload 调节 batching、admission、prefill/decode 比例和实例负载均衡；高频 offloading 层根据请求复杂度、用户位置、链路状态和状态位置执行 request routing、workflow placement 和 state migration。协同优化的关键不是把每一层都独立做到局部最优，而是让它们在统一 SLO、资源约束和状态约束下避免互相抵消。

从控制变量看，三类算法的输入和输出也存在明显依赖。模型切分层输出可部署模板、阶段容量、KV handoff cost 和 fallback path；batching 层输出队列等待、active batch occupancy、TTFT/TPOT、KV 水位和阶段 backlog；offloading 层输出请求入口、计算节点、状态处理动作和后续 session 的 KV owner。这些输出又会成为其他层下一轮决策的输入。换言之，协同优化不应只追求一个全局静态最优解，而应维护一个持续更新的系统状态视图。

7.2 Model / Service Placement – Request Offloading 协同

模型/服务部署与请求卸载之间的耦合，是 CEC-LLM 推理系统中最直接的协同优化关系。由于边缘节点无法缓存所有 LLM、SLM、expert、adapter 或工具服务，模型能力是否提前部署会直接决定请求能否在近端被处理，以及是否需要跨边缘或上云。反过来，请求流量、用户会话分布和模型调用频率又会影响下一周期的模型缓存和服务部署收益。

已有工作已经开始显式处理部署规划与请求卸载之间的时间尺度差异。SLM-LLM-Collab [65] 将 SLM caching 与 local/D2D/edge inference offloading 进行双时间尺度建模，说明模型缓存不是独立部署问题，而是为后续请求卸载提供可行路径。T2DRL [74] 进一步将 long-context LLM serving 中的 model caching、request management 和资源匹配联系起来，强调上下文长度和移动边缘资源会共同影响模型缓存与请求处理策略。H-MEC-Placement [76] 虽然不是 LLM-specific 工作，但从 general MEC 角度说明 service placement、task scheduling 和 resource allocation 在规划周期内存在强耦合。LVM-Offload [77] 则在 foundation model 服务中联合考虑模型选择、推理步数、任务卸载和资源分配。

这一类协同的核心变量包括 model/service placement、SLM/LLM cache、adapter/expert residency、service capacity、request arrival pattern、routing action、cloud fallback 和 cache update period。低频部署层如果只根据历史平均流量放置模型，可能无法应对移动热点和突发请求；高频请求卸载如果只根据当前节点负载选择路径，又可能频繁调用未缓存模型，造成冷启动、跨域传输和质量回退。因此，部署层和卸载层需要形成“长期能力布局 + 短期路径选择 + 在线反馈更新”的闭环。

在 CEC 中，模型部署层应当被视为协同优化框架中的低频规划层，而不是一次性静态配置。其输出不应只是某个模型放在哪个节点，而应包括模型能力布局、服务容量边界、可服务请求类型、切换条件和回退路径。请求卸载层则在给定部署状态下进行高频决策，并把流量、命中率、队列和质量反馈回部署层，用于下一周期更新模型缓存和服务布局。

7.3 Model Partitioning – Batching / Scheduling 协同

模型切分与 batching/scheduling 的耦合主要体现在推理阶段的容量匹配和时延控制上。LLM inference 中 prefill、decode、verification 等阶段具有不同资源画像：prefill 更偏计算密集，decode 更偏 memory bandwidth 和低时延 token streaming。模型切分或阶段解耦改变这些阶段所在的资源池，而 batching 和 scheduling 决定请求如何进入这些资源池以及 token 如何被迭代生成。

已有 LLM serving 工作从不同角度揭示了这种耦合。Splitwise 和 DistServe 通过 prefill/decode disaggregation 说明，将两类阶段部署到不同资源池可以提高 goodput，但也会引入阶段间协调和 KV handoff 成本 [27], [4]。Orca [1] 和 vLLM [2] 等系统说明，iteration-level scheduling 和 continuous batching 可以显著提高 GPU 利用率，但其效果受到模型部署方式、decode worker 容量和 KV cache 管理策略影

响。Llumnix 进一步展示了动态请求迁移、实例调度和 serving 稳定性之间的关系 [49]。Mooncake 从 KVCache-centric serving 出发，将 KV cache 管理、prefill/decode 分离和请求调度结合起来，说明状态位置会影响 batching 和阶段执行路径 [13]。Jupiter 将 generative LLM 在 edge devices 上的 collaborative inference 与 pipeline parallelism、speculative decoding 结合起来，进一步说明在 CEC 场景中，阶段拆分和 serving scheduling 必须一起考虑 [19]。

这一类协同的关键变量包括 partition template、stage boundary、prefill/decode pool capacity、batching window、active sequence set、chunk size、KV transfer time、TTFT/TPOT target 和 worker scaling。模型切分模板不能只描述模型层或阶段放在哪里，还需要暴露阶段容量、KV handoff 成本、batching 可容纳规模和 TTFT/TBT 估计。Batching 层也不能只根据当前队列最大化吞吐，而需要感知切分拓扑、状态位置和用户 SLA。

在 CEC 系统中，模型切分与 batching 的协同尤其需要避免局部优化互相抵消。例如，阶段切分把 prefill 放到区域边缘可以提升 prefill 吞吐，但如果 KV 传回接入边缘的链路不稳定，decode TTFT 和后续 token latency 可能恶化；batching 策略为了追求高吞吐扩大等待窗口，可能让移动用户在 handover 前无法及时完成关键阶段；continuous batching 扩大 active set 能提升利用率，但也会提高 KV 显存压力，使请求迁移和状态复制更困难。因此，协同控制器需要同时观察阶段负载、batch 状态、KV 水位和链路状态。

7.4 Request / Workflow Routing – Runtime State Management 协同

请求路由和 runtime state 管理之间的耦合，是 LLM inference 相比传统 task offloading 最突出的新问题之一。传统 offloading 中，任务完成后状态通常不再影响后续请求；而在 LLM 多轮对话、长上下文和 agent workflow 中，context、KV cache、prefix cache、agent memory 和 tool outputs 会持续影响后续推理。此时，请求应该路由到哪里，不仅取决于哪个节点算力空闲，还取决于相关状态是否已经在该节点或邻近节点存在。

Mooncake、Preble 和 CachedAttention 等 cache-centric serving 工作说明，prefix/KV cache 已经从单纯的内存优化对象变成影响请求调度和服务路径选择的核心资源 [13], [46], [78]。FuseSpill [71] 从显存受限场景出发说明，KV cache spillover 可能导致 decode 请求迁移到辅助 GPU 或其他资源位置，从而改变原有的执行路径。CA-AIGC-Migration [73] 则直接面向移动边缘场景，将用户 long context 视为可迁移状态，而不是迁移完整模型，从而在回复质量和迁移成本之间做权衡。QoS-LLM-Router [60] 和 CES-CB [61] 虽然主要关注 request-level routing，但它们也说明了 routing action 会改变队列状态、batch 组成和后续请求的等待时延。

这一类协同的关键变量包括 session ID、prefix hash、KV block location、state size、state freshness、remote access latency、migration cost、recompute cost、queue state 和 request quality target。对于连续

session，路由动作和状态动作应被联合枚举：在已有 KV 的节点本地执行、将 KV 迁移到新节点执行、远程访问旧节点状态，或在新节点重新 prefill / recompute。不同动作的当前成本和长期收益不同，不能只用单次请求的最小时延判断。

对于 multi-agent workflow，这种耦合更加复杂。MasRouter、Conductor 和 AFLOW 等工作主要从算法层研究如何选择 agent 数量、角色、模型和工作流结构，但在 CEC 系统中，每个 agent 背后的模型、工具、数据库和 memory 可能位于不同物理节点。因此，workflow orchestration 必须进一步转化为 workflow offloading：不仅决定 planner、solver、critic、verifier 等角色如何协作，还要决定这些组件在哪里执行、哪些 context 可以共享、哪些中间状态需要迁移，以及何时调用云端强模型进行 verification。

7.5 Mobility-aware Hierarchical Control

多用户移动性会同时影响模型部署、batching、请求路由和状态管理，因此它是三类算法协同优化中最重要的外部扰动之一。用户移动会改变区域请求热点，使原有 model placement 失效；会改变接入链路和候选 edge，使 request routing 的当前最优路径失效；会在请求或 workflow 执行过程中触发 handover，使中间状态和后续阶段的执行路径需要重新选择；也会使 long context、KV cache 和 agent memory 留在旧 edge，从而产生状态迁移、远程访问或重算之间的权衡。

已有移动性相关工作提供了局部研究基础。MGCO [66] 说明，generative computation offloading 不能只根据当前状态短视决策，而需要利用长程移动信息进行规划。DT-MADTO [70] 通过 digital twin 建模 DAG task、用户移动和中间数据流，说明移动性会改变 workflow/subtask offloading 的传输代价和拥塞状态。CA-AIGC-Migration [73] 进一步将移动性与 long context migration 结合，说明状态迁移可以成为替代完整服务迁移的轻量机制。T2DRL [74] 则提示，在 mobile edge 场景中，long-context serving 的模型缓存、请求管理和资源匹配需要与移动场景共同考虑。

这一类协同的关键变量包括 user trajectory、handover probability、future ingress node、service hotspot、state owner、migration bandwidth、session continuation probability 和 workload forecast。低频层根据用户移动和服务热点调整模型/服务布局；中频层根据负载和状态命中率调整 batching 与实例分流；高频层根据 handover、链路抖动和 context/KV 位置进行 request/workflow/state routing。三个时间尺度不需要每次都同时重优化，但必须共享预测和反馈。

现有研究还没有形成完整的 CEC-LLM 跨层闭环。很多工作只优化其中一层：有的关注 request offloading，有的关注 DAG subtask，有的关注 context migration，有的关注 model caching。未来更需要一个 mobility-aware hierarchical controller，使模型能力、请求路径、workflow 结构和状态位置能够随用户移动和 workload 变化自适应调整。

7.6 算法方案对比与面向华为验证环境的建议

结合第四、五、六章的项目约束，第七章不宜把协同优化写成一个不可落地的全局大优化问题。更合理的定位是：在已有模型切分规划器、业务感知 batching 控制器和长期成本感知请求卸载策略之上，实现一个**分层状态感知协同控制器**。它不替代底层 runtime 的 continuous batching、PD 分离、KV cache 管理或路由执行逻辑，而是统一读取部署模板、batch 状态、节点负载、链路状态和 KV/session 元数据，在不同时间尺度上修正各层参数。

该控制器的第一部分是统一状态视图。系统需要收集模型/服务部署状态、可用切分模板、prefill/decode worker 容量、batching 参数、队列长度、TTFT/TPOT/E2E latency、KV cache 位置、prefix 命中率、节点显存、链路 RTT/带宽、请求优先级和用户人口变化。只有把这些信息放入同一个状态表，才能判断某个请求到底应该近端执行、跨节点迁移状态、进入更大 batch，还是回退到云端/区域边缘。

第二部分是分层决策周期。低频周期执行模型/服务 placement 与切分模板选择，重点看历史请求分布、未来热点、节点容量和模型能力覆盖；中频周期调节 batching window、batch token budget、prefill/decode ratio、admission threshold 和实例分流；高频周期执行 request routing、workflow placement、KV migrate/recompute/local 动作和异常回退。不同周期之间通过显式接口传递容量、成本和状态，而不是各自维护不一致的局部视图。

第三部分是冲突检测与受控重配置。协同控制器需要识别三类典型冲突：模型切分带来的 KV handoff 成本超过 batching 收益；batching 为吞吐扩大等待窗口导致高优先级请求或移动用户违反 SLO；request routing 因已有 KV 黏附远端节点，长期造成跨节点通信。对于这些冲突，控制器应通过候选模板切换、batch 参数收缩、状态迁移/重算、路由惩罚项和冷却时间进行修正，避免频繁重配置造成调度震荡。

验证设计上，建议第七章对应的实验分成四组。第一组是独立算法组合基线：分别启用第四章切分部署、第五章动态 batching、第六章请求卸载，但不共享状态，只用各自局部观测做决策。第二组是共享状态但无长期预测：三类算法读取统一节点、队列和 KV 元数据，但只优化当前窗口。第三组是分层协同控制：低频更新部署/切分模板，中频调节 batching 参数，高频执行状态感知请求卸载，并把移动入口、KV owner 和未来 session 成本纳入决策。第四组是扰动与消融：改变用户移动比例、链路带宽、KV cache 规模、长 prompt 比例、请求优先级分布和节点显存水位，分别关闭状态共享、移动预测、KV 迁移、batch 参数联动和切分模板联动，观察 P99 TTFT、平均 E2E latency、SLA violation、跨节点通信量、KV 命中率和满足 SLA 的并发数。

因此，第七章面向验证环境最适合落地的是**统一元数据驱动的分层协同优化框架**。它的算法主线不是一次性求解所有 placement、batching 和 offloading 变量，而是建立“低频部署布局—中频 serving

参数-高频请求/状态路由”的闭环。这样既能和第四、五、六章的单项算法保持边界清晰，也能把协同收益落实到可测指标上：在满足 P99 TTFT SLA 的前提下提升并发能力，降低平均 E2E 时延，减少跨节点状态通信，并提高移动 session 的服务连续性 [59]。

7.7 小结

总体来看，三类算法的协同优化不是简单地把模型切分、batching 和 task offloading 合并成一个大优化问题，而是要在实际工作揭示的局部耦合关系基础上，建立分层、多时间尺度、状态感知的控制框架。本章将相关耦合关系组织为四类：Model / Service Placement – Request Offloading 协同、Model Partitioning – Batching / Scheduling 协同、Request / Workflow Routing – Runtime State Management 协同，以及 Mobility-aware Hierarchical Control。

已有工作已经分别展示了 model caching 与 inference offloading、prefill/decode disaggregation 与 scheduling、KV cache 与 request routing、context migration 与 mobility-aware serving 之间的耦合，但这些机制仍然缺少面向 CEC-LLM inference 的统一组织。CEC 场景的核心挑战在于，模型能力、batch 状态、请求路径和运行时状态会随着用户移动、负载变化和链路波动持续变化；局部最优策略如果缺少共享状态和长期反馈，容易造成阶段失衡、尾延迟放大、KV 状态黏附或频繁迁移。

面向工程落地，协同优化应采用“统一状态视图 + 分层决策周期 + 受控重配置”的方式推进。低频层负责模型/服务部署和切分模板选择，中频层负责 batching 参数、admission 和阶段队列调节，高频层负责 request/workflow routing 与 context/KV/runtime state 处理。三层通过 TTFT/TPOT、queue length、KV 命中、链路状态、handover 和显存水位等指标持续反馈，从而使模型切分、batching 和 task offloading 不再只是并列算法，而是服务于同一 CEC-LLM 推理目标的协同控制闭环。

八、面向 Ascend/CANN 验证环境的 CEC 推理工程实现方案

8.1 工程定位

面向验证环境的工程实现需要把前文三个算法落到同一套可执行闭环中：第四章的动态规划切分策略依赖 Ascend GPU/NPU 上可运行的 layer/block 级 pipeline stage，并在此基础上搜索 layer-wise stage boundary、stage placement 和混合并行动作；第五章的 RL/bandit batching 优化器以参数自适应、请求排序/准入、阶段干扰控制为递进路线；第六章的请求调度则把就近路由扩展为 SLA、显存、链路和 KV 状态共同约束的长期成本决策。第七章已经将三者组织为低频部署、中频 serving 参数和 高频请求/状态路由的协同控制闭环。本章的任务是说明这些规划与策略学习算法如何映射到 vLLM-Ascend、MindIE、CANN 和异构边缘验证环境中的可执行接口。

vLLM-Ascend 是 vLLM 面向 Ascend NPU 的硬件插件，依赖 Linux、Ascend NPU、CANN、torch-npu、torch 和 NNAL 等基础环境 [79], [80]。其功能文档已经列出 dynamic batch、fine-grained tensor parallelism、layer sharding、context parallel 和 dynamic chunked pipeline parallel 等能力 [81]；KV Cache Pool 和 speculative decoding 也分别有独立说明 [82], [83]。这些能力可以作为工程底座，但不能直接等同于 CEC 场景下的资源自适应推理系统：它们需要被暴露为策略可调用的安全动作，并由外层控制面根据节点资源、链路状态、请求负载和业务 SLO 做受控决策。

本章建议采用“执行底座 + 规划/学习器 + 安全过滤器 + 分层控制”的实现路线。执行底座先保证 Ascend 侧能够稳定运行 layer/block 级 pipeline stage；模型切分规划器把 layer-wise split、PP/TP/EP/DP、context parallel、KV Cache Pool、PD 分离和 speculative decoding 等能力作为可部署候选；batching 学习器把 max running requests、max batched tokens、chunked prefill、等待窗口和准入阈值作为动作空间；安全过滤器根据显存、链路、SLA 和框架能力裁剪不可执行动作；分层控制器再根据统一元数据在低频、中频和高频三个周期上执行模型切分、batching 参数控制和请求/状态路由。工程实现的核心是在现有 runtime 能力之上补齐 CEC 场景需要的资源画像、动态规划、策略学习、状态管理和在线安全闭环，使算法输出能够被部署、压测、回退和解释。

8.2 总体架构

图 20 给出了推荐的工程架构。与旧式“集中入口加调度器”的架构不同，这里把系统拆成四个互相反馈的平面：执行面负责 vLLM-Ascend / MindIE 等 CANN 兼容实例运行；画像面负责模型、硬件、拓扑和 workload profiling，为模型切分动态规划生成 stage/link 代价表，并为 RL/bandit batching 提供训练和评估数据；状态面负责队列、KV、链路、session 和故障元数据；控制面负责低频切分策略、中频 batching 策略和高频请求调度策略。

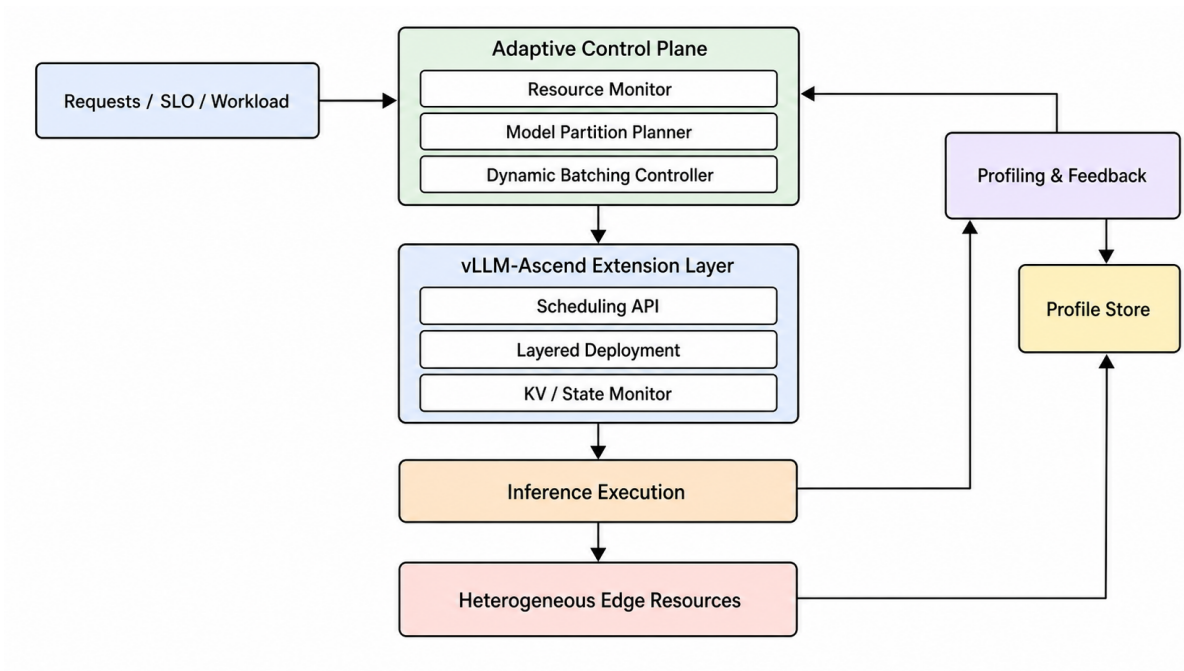


图 20 面向 Ascend/CANN 验证环境的 CEC 推理工程框架

从请求路径看，业务请求从多个流量入口分别进入系统，携带 session ID、业务优先级、SLO、隐私标签、输入长度估计和上下文标识。入口层保持分布式接入方式，每个入口节点查询共享状态面并执行本地路由决策，同时接受控制面的全局约束和策略参数。入口层先判断已有 KV 或 prefix cache 位于哪个节点，再把节点显存、队列长度、链路状态和状态位置输入请求调度策略。若请求进入本地或近端实例，batching 策略决定其是否立即接纳、等待合批、进入长请求隔离队列或触发 chunked prefill；若请求需要跨节点执行，请求调度策略同时决定 KV 是本地读取、迁移、复制还是重算。

从反馈路径看，推理实例持续上报 TTFT、TPOT、ITL、E2E latency、active sequence 数、waiting queue 长度、KV block 占用、显存峰值、NPU 利用率、stage transfer time、链路 RTT/带宽和错误码。画像面将离线 profiling 与在线观测合并，形成 stage/link 代价、状态特征、动作效果、奖励和安全约束数据；状态面维护 KV owner、prefix block hash、session 入口变化和实例健康状态；控制面据此更新动态规划代价表、RL/bandit 策略、动作裁剪边界和请求路由代价。这个闭环需要为每次动作保留可追溯原因，例如选择某个切点和 placement、缩小 batch、迁移 KV 或重算 KV 时对应的状态、约束和收益估计。

8.3 分阶段实现路线

第一阶段是 Ascend 层级切分执行能力。优先目标是让 CodeLlama34B、Qwen2-VL-7B-Instruct 等目标模型能够按连续 layer/block 切成多个 pipeline stage，并映射到 230T/64GB、70T/24GB、310P、910B4 或多机多卡 Ascend 节点上执行 [59]。工程上需要解决四个接口：分段权重加载、stage 间 hidden state

传输、pipeline stage 启动参数和异常回退。接口稳定后，第四章的拓扑感知 layer-wise split 才能从规划结果转化为真实可部署方案，并进一步支撑 TP、EP、DP、PD 分离和 KV pool 等高阶动作。

第二阶段是 EdgeShard 风格的动态规划切分规划器。系统先对单实例、平均 layer split、固定 PP、固定 TP、PP+TP、layer sharding、context parallel、KV Cache Pool、PD 分离和 speculative decoding 等运行形态做离线 profiling，记录每层 latency、activation 大小、KV 增长、stage transfer cost、显存峰值和可支持并发，并用这些数据校准 stage 代价表、链路代价表和安全过滤器。切分规划的顺序应与第四章保持一致：先用动态规划搜索 layer-wise stage boundary、stage placement 和 pipeline balance；再判断单个 stage 是否需要叠加 TP，MoE 或组件化模型是否需要 EP，流量承载是否需要 DP；最后才把 PD 分离、KV Cache Pool 和 semantic/token-level collaboration 作为高阶候选。

第三阶段是分阶段增强的 RL/bandit 在线 batching 优化器。实现上先只训练参数自适应策略，保持框架默认请求顺序或 FIFO 不变，让策略根据并发、prompt 长度分布、KV 水位和 TTFT/TPOT/ITL 反馈调节 max running requests、max batched tokens、等待窗口、chunk size 和 prefill/decode quota。这样可以先确认收益是否来自 batching 参数本身。第二步再加入请求排序和准入控制，用 SLO slack、预测 prefill/decode 时间、KV 增长和对其他请求 ITL 的边际影响学习排序权重，使出队顺序同时反映业务紧迫度和系统边际成本。第三步再启用联合策略：短 prompt 和紧 SLO 时策略收紧 active set 与等待窗口，高并发且 slack 充足时扩大 token budget，长 prompt 或 VLM 比例升高时启用 chunked prefill 或长请求隔离，具备稳定 KV handoff 时才允许策略选择 PD 分离动作。

第四阶段是长期成本感知请求调度和 KV 管理。三个节点都应维护统一的节点负载、显存、网络 and KV 元数据；请求到达后，系统枚举目标计算节点以及 local、migrate、recompute 等状态处理动作，并先用 SLA 和 memory 约束过滤不可行动作。正常路由不只选择当前请求时延最低的节点，还要估计动作执行后 KV owner、节点负载、未来入口位置和后续 session 成本的变化。KV 管理应采用 block-level 索引和增量同步：迁移前先查询目标节点已有 prefix/KV blocks，只传输缺失部分；旧节点短期保留状态副本，以支持迁移失败、链路退化或负载突变时回退。

第五阶段是统一元数据驱动的协同控制。低频周期更新模型切分策略和服务能力布局，可按分钟级或配置变更事件触发；中频周期调节 batching 参数、admission threshold、prefill/decode quota 和实例分流，可按数秒到十秒级窗口触发；高频周期执行 request routing、workflow placement、KV migrate/recompute/local 动作和异常回退，可按单请求或 session 边界触发。三类决策必须共享同一状态视图：模型切分层输出 stage 容量和 KV handoff cost，batching 层输出队列等待和 KV 水位，请求调度层输出请求入口和状态位置。控制器重点处理三类冲突：切分带来的状态传输成本超过并行收益，batching 扩大等待窗口导致高优先级请求违约，路由因 KV 黏附远端节点造成长期跨节点通信。

8.4 关键工程模块

第一个模块是 vLLM-Ascend / MindIE 接口适配层。它需要梳理模型加载、并行配置、scheduler、KV Cache 管理、metrics 暴露和错误码路径，并把底层能力转化为控制面可调用的动作接口。例如，模型切分策略输出的动作需要能够落到 stage 数、每个 stage 的 layer 范围、节点映射、tensor parallel degree、context parallel 开关、KV pool 策略和回退动作；batching 策略输出的动作需要能够落到 max running requests、max batched tokens、等待窗口、chunk size、prefill/decode quota 和 admission threshold；请求调度策略输出的动作需要能够落到目标实例、KV local/migrate/recompute、状态副本保留时间和失败回退路径。MindIE 可作为 Ascend 生态中生产化推理、调试、调优和迁移接口的参考底座 [84]；vLLM 的 OpenAI-compatible server 可作为服务入口和接口兼容性的参考 [85]。

第二个模块是 Profile Store、Cost Table / Replay Buffer 与 Safety Guard。Profile Store 保存模型-硬件-拓扑的观测数据，包括 layer latency、activation、KV 增长、stage transfer cost、prefill/decode 吞吐、OOM 边界和链路状态；Cost Table 保存动态规划需要的 stage 代价、link 代价、显存约束和切换成本；Replay Buffer 保存 batching 与请求调度的状态、动作、奖励、违约和回退记录，用于离线训练、在线微调 and 策略回放评估；Safety Guard 保存显存上限、链路带宽下限、P99 TTFT 约束、动作冷却时间和回退条件。三者需要区分离线压测值和在线校正值：离线值用于生成初始代价表和安全边界，在线值用于修正当前窗口的动作裁剪、代价估计和奖励估计。这样可以避免系统在负载波动时盲目切换，也能避免长期使用过期画像。

第三个模块是在线策略决策器。模型切分策略只在低频周期或明显资源变化时输出动作，并设置冷却时间和最小收益阈值；batching 策略在中频周期更新参数、排序权重和准入阈值，但不直接替代 runtime 内部 token 调度；请求调度策略在高频周期处理每个 session/request 的目标节点和 KV 动作。三个决策器共享同一组约束：显存不溢出、SLA 不违约、链路带宽可承载、动作可启动、状态迁移成本可控。若安全过滤后可行动作为空，系统必须显式进入回退路径，例如降低并发、缩小 batch、回退固定 PP/TP、转发区域边缘或云端完整实例。

第四个模块是监控、审计和回退。验证环境中的算法收益必须能被复现，因此每次策略动作、batch 参数调整、请求迁移和 KV 重算都要记录触发状态、策略输出、安全过滤结果、奖励估计、最终动作和实际结果。监控指标至少包括 P90/P99 TTFT、TPOT、ITL、平均 E2E latency、SLA violation、满足 SLA 的 rps、最高并发用户数、显存峰值、OOM 次数、NPU 利用率、KV 命中率、状态迁移量和回退次数。对外报告时，应能区分收益来自层级切分策略、batching 参数自适应、请求排序策略、KV 迁移还是三者协同。

8.5 验证路线

验证应按能力递进组织，先验证执行底座，再验证单类策略，最后验证协同闭环。第一组实验验证 Ascend 层级切分执行能力：比较单实例、平均 layer split、固定 PP/TP 和拓扑感知 layer-wise split，观察 OOM、P99 TTFT、TPOT、stage bubble、stage transfer time 和满足 SLA 的并发用户数。该组实验回答的是“Ascend 推理栈是否已经具备可执行的层级切分底座”。

第二组实验验证动态规划模型切分策略：在完成 profiling、stage/link 代价表和安全边界校准后，比较平均切分、固定 PP/TP、负载感知 placement、启发式拓扑感知 layer-wise split、EdgeShard 风格动态规划切分、动态规划加安全过滤和收益阈值。重点指标是满足 SLA 的请求处理数、最高并发用户数、显存峰值、通信开销、不可执行方案比例、pipeline bubble 和回退次数。这里的目标与第四章一致：相对于多节点平均切分基线，验证 goodput 和 SLA-constrained concurrency 是否有稳定提升 [59]。

第三组实验验证 RL/bandit 在线 batching 优化器：先比较固定参数、离线调优参数、contextual bandit 和 actor-critic 参数自适应；再比较 FIFO、EDF/slack baseline、priority queue、边际代价排序和 RL 学习到的排序权重；最后在短 prompt、长 prompt、VLM 和多优先级混合 workload 上测试联合策略。指标应覆盖满足 SLA 的请求处理速度、平均 E2E latency、P99 TTFT、ITL 抖动、高优先级请求违约率、普通请求饥饿比例、策略切换次数和回退次数。

第四组实验验证长期成本感知请求调度和 KV 管理：比较就近执行、当前最小时延 Greedy、长期成本路由、长期成本路由加 block-level KV 管理。除 TTFT、E2E latency、吞吐和并发数外，还要统计用户移动后的累计通信时延、请求转发次数、KV 迁移/重算次数、状态传输量、prefix/KV 命中率和 session 节点切换次数。该组实验用于说明为什么 CEC 中的请求调度不能只看当前节点负载。

第五组实验验证协同控制：分别启用第四章切分策略、第五章 batching 策略和第六章请求调度但不共享状态，作为独立算法组合基线；再启用统一元数据但不做长期预测；最后启用完整的低频部署策略、中频 batching 策略、高频请求/状态路由闭环。扰动项包括链路带宽、RTT、可用显存、长 prompt 比例、移动用户比例和请求优先级分布。最终报告应同时给出性能收益和稳定性成本，尤其是策略切换频率、策略震荡、回退次数和状态传输开销。

8.6 小结

工程结论是：面向 Ascend 验证环境的 CEC LLM 推理系统应先补齐层级切分执行底座，再把模型切分、batching 和请求调度分别实现为可观测、可回退、可解释的在线决策能力。第四章对应低频的动态规划混合切分部署；第五章对应中频的分阶段 RL/bandit batching 优化器；第六章对应高频的长期成本感知请求路由和 KV 管理；第七章则把三者收敛为统一元数据驱动的分层协同控制。

这样的工程路线与现有 vLLM-Ascend 能力并不冲突。vLLM-Ascend 继续负责模型加载、算子执

行、KV 管理和实例内调度；外层系统负责画像、动态规划、策略学习、状态、路由、动作裁剪和回退。只有把“可执行动作接口”和“在线状态”放在同一个闭环中，前文提出的算法建议才会从综述中的方法对比，转化为验证环境中可以部署、压测和解释的工程方案。

九、实验评估与对比指标

9.1 实验场景

实验设计需要先围绕 SOW 中三项算法交付展开，而不是把泛化端-边-云场景全部铺开。建议将实验划分为三组主实验和一组协同实验。第一组是资源自适应模型切分实验，用于验证 layer-wise stage boundary、stage placement、pipeline balance 以及必要的 TP/EP/DP 补充能力是否优于多节点平均层级切分和固定并行模板；第二组是业务感知 batching 实验，用于验证在线参数自适应、请求重排和混合 batching 策略相对静态 batching/FIFO 的收益；第三组是用户移动场景下的请求调度实验，用于验证多流量入口、多推理实例、链路拓扑和 prefix/KV 状态共同约束下的路由策略是否优于就近实例路由；第四组才是协同实验，用于观察切分部署、batching 和移动请求调度共享状态后的叠加收益与稳定性。相关场景设置可以参考端-边-云协同智能、移动服务放置和 LLM serving benchmark 中对异构资源、移动性和 SLO/goodput 的评估方式 [4], [9], [86]。

验证环境应尽量贴近任务书约束：OpenEulerOS、CANN 和任一可在 Ascend 上运行的推理框架，模型覆盖 CodeLlama34B 与 Qwen2-VL-7B-Instruct，硬件包含 ARM CPU、230T/64GB 或 70T/24GB Ascend NPU、310P/910B4 组合以及可调 vNPU，链路覆盖 1–100 Gbps 带宽和 1 us–20 ms 时延范围。模型切分实验需要单独覆盖异构拓扑，而不是只在同构多卡环境中验证。建议至少包含 A800T-A2 两机四卡 100 Gbps 直连、A800T-A2 两机四卡 25 Gbps host RDMA、单机两卡 56 GBps HCCS、半算力或四分之一算力卡，以及 310P–910B4 联合组网等配置。每组实验都应记录 layer-level profile、stage 耗时、stage 间 activation/hidden state 传输、pipeline bubble、显存峰值和回退次数。这样才能判断收益来自合理的层级切分和拓扑放置，而不是来自单纯增加副本数或换用更强设备。

用户移动场景应单独作为重点实验，而不是混在普通多节点实验中。SOW 中的关键设定是三个同模型推理实例、三个请求客户端、默认就近实例、prefix cache 开启；测试运行超过 50% 后，将 20% 请求发送到另外两个实例，以模拟入口变化和历史状态位置分离。该场景需要记录入口变化、服务实例变化、状态访问路径变化和 token streaming 中断情况。这样才能判断 mobility-aware routing 是否真的改善了服务连续性，而不是只是在平均时延上略有收益。

Workload 也需要按 LLM 推理特征构造。短 prompt + 短输出用于模拟简单问答和低复杂度交互；长 prompt + 短输出用于考察 prefill 压力和 prefix cache；短 prompt + 长输出用于考察 decode 阶段的持续吞吐；长上下文多轮会话用于考察 KV Cache 管理和状态迁移；混合 SLO 请求用于考察业务感知调度是否会牺牲低优先级请求。若只使用均匀长度请求，实验会低估 CEC 场景下 batching、状态管理和路由策略的真实难度。

9.2 Baseline 设置

Baseline 应覆盖四类对照。第一类是模型切分对照，包括单实例部署、平均 layer split、固定 layer split、固定 PP、固定 TP、固定 PP+TP、只按负载做 layer placement、拓扑感知 layer-wise split，以及拓扑感知 layer-wise + TP 混合切分。若模型包含 MoE，还应加入固定 EP 与 expert/component-aware placement；若启用阶段解耦，则增加 PD disaggregation / KV Pool 作为扩展 baseline。它们用于回答第 4 章的主线算法是否真的优于平均层级切分和固定模板，而不是只优于“完全不切分”。

第二类是移动请求调度对照，包括就近实例路由、只按负载路由、只按 RTT 路由、当前请求最小时延 Greedy 路由和长期成本感知路由。SOW 明确不应把所有入口流量先汇聚到中心负载均衡节点，因此 centralized load balancer 更适合作为不推荐的工程对照或仿真上界，而不应作为默认实现基线。第三类是 runtime 对照，包括固定 batch size、FIFO 队列、普通 dynamic batching、continuous batching、chunked prefill 关闭和 KV-aware admission 关闭。它们用于拆分 batching 机制带来的收益。第四类是状态对照，包括无状态迁移、用户移动后重新 prefill、远程访问旧状态、迁移 session state、迁移 KV Cache 和只使用 prefix cache。它们用于判断移动场景下状态连续性策略的边界。上述 baseline 的设置可以借鉴 Orca 的 iteration-level scheduling、vLLM 的 KV memory management、DistServe 的 prefill/decode disaggregation、Preble 的 prefix-aware prompt scheduling，以及 Follow Me at the Edge 的 mobility-aware placement 实验思路；但这些工作并不与固定 batching、continuous batching 或状态迁移 baseline 形成简单的一一对应关系 [1], [2], [4], [46], [86]。

对模型部署相关算法，应避免只和“完全不切分”比较。更合理的 baseline 应从弱到强逐步设置：完全不切分、平均层级切分、固定 layer split、固定 PP/TP/PP+TP、负载感知 layer placement、拓扑感知 layer-wise placement、拓扑感知 layer-wise + TP/EP 混合切分。端侧小模型独立推理、边缘中模型独立推理和云端大模型兜底可以作为服务形态对照，但不应替代模型切分 baseline。这样可以区分三类收益：一类来自模型规模或部署位置变化，一类来自层级切点和 stage 放置优化，另一类来自 TP/EP/PD/KV 等补充能力。

对于综合方案，还应设置逐步叠加 baseline：仅 batching、batching + state-aware routing、batching + mobility-aware routing、batching + routing + 模板化部署。逐步叠加比一次性比较更有解释力，因为它能回答每个模块的边际贡献，也能发现模块之间是否存在互相抵消。例如 routing 提高 prefix cache 命中但增加跨边缘转发，或者 chunked prefill 改善 TPOT 但拉长部分长 prompt 的 TTFT，这些现象都需要通过分层 baseline 暴露出来。

9.3 指标体系

指标体系应分为服务体验、模型切分质量、系统效率、网络状态和质量约束五层。服务体验层关注用户直接感知到的指标，包括平均 E2E 时延、P90/P99 TTFT、TPOT、ITL、SLA 达标率、请求失败率和 token streaming 中断时间。对于交互式 LLM 服务，平均时延不够，P90/P99 TTFT 和 ITL 更能反映尾部体验；对于工具调用或工作流型请求，deadline miss ratio 比单纯 TPOT 更重要。

模型切分质量层用于评估 layer-wise-first 算法是否真正有效，包括 stage latency imbalance、pipeline bubble ratio、stage 显存峰值、跨 stage activation transfer、stage 间等待时间、切分方案不可部署率、回退次数、重配置耗时、满足 SLA 的并发用户数和相对平均层级切分的 goodput 提升。若只记录端到端时延，无法解释一个切分方案到底是因为切点合理而变快，还是因为某个强设备承担了过多计算。

系统效率层关注资源是否被充分使用，包括 GPU/NPU/CPU 利用率、显存/内存利用率、prefill 吞吐、decode batch occupancy、active batch 平均请求数、KV Cache 占用率、KV eviction 次数、prefix cache 命中率和 admission rejection rate。这里需要特别区分 prefill 和 decode，因为两者瓶颈不同。一个方案如果只提高 prefill 吞吐，却让 decode 队列排队变长，整体交互体验未必改善。

网络与状态层用于评估 CEC 特有成本，包括端-边传输量、边-边传输量、边-云传输量、activation transfer、KV transfer、状态迁移耗时、远程状态访问次数、handover 额外时延、通信-计算 overlap 比例和状态命中后节省的 prefill token 数。对于模型切分实验，还需要区分 layer-wise stage 间 activation/hidden state 传输和 prefill/decode 阶段间 KV 传输，因为前者决定层级切分是否划算，后者决定阶段解耦是否划算。这些指标不能只作为附录，因为 CEC 方法的很多收益都来自“少算一次 prefill”“少传一次 activation”或“少迁一次状态”，如果不记录状态和中间结果流动，就无法解释时延变化的来源。

质量与约束层则用于防止系统优化以牺牲生成效果、隐私或公平性为代价。应记录任务成功率、生成质量代理指标、模型降级/回退比例、隐私敏感请求本地处理比例、不同 SLO 等级用户的违约率差异，以及移动用户与静态用户之间的体验差异。如果只保留核心验收指标，建议优先选择 P90/P99 TTFT、TPOT/ITL、平均 E2E latency、SLA 达标率、满足 SLA 的 rps 或用户数、GPU/NPU 利用率、prefix cache 命中率、KV/state 迁移成本和 handover 中断时间。这组指标既能体现 LLM serving 性能，也能体现无线 CEC 的场景价值。

9.4 消融实验

消融实验不应只是“去掉某个模块”，还要说明去掉之后系统退化到什么策略。对于 batching，可比较 FIFO + 固定 batch、dynamic batching、continuous batching、continuous batching + chunked prefill、continuous batching + SLO priority、continuous batching + KV-aware admission。这样可以清楚看到迭代级调度、长 prompt 切块、业务优先级和状态感知分别贡献了什么。类似的模块化消融思路在 Sarathi-

Serve、DistServe、Mooncake 和 Preble 等系统论文中较常见，适合用于说明单项机制与组合机制的增益差异 [3], [4], [13], [46]。

对于路由和状态管理，可从就近路由开始，逐步加入实例负载、RTT、prefix cache 命中、session state 位置、KV 迁移代价和移动趋势。每加入一个因素，都观察平均 E2E latency、P99 TTFT、handover 中断时间、跨边缘传输量和 SLA 达标率的变化。特别要比较“迁移状态”“远程访问状态”和“重新 prefill”三种策略，因为它们分别代表高一次性成本、低启动成本但持续远程访问、以及高计算重算成本。哪个策略更优取决于状态大小、链路带宽、剩余生成长度和用户驻留时间。

对于模型部署，可从 layer-wise 主线出发做消融。第一步比较平均 layer split、固定 layer split 和拓扑感知 layer-wise placement，观察切点选择和 stage 放置本身的收益；第二步在相同 layer-wise placement 上叠加 TP，观察单个 stage 内部显存和算力压力是否被缓解；第三步在 MoE 或组件化模型上叠加 EP，观察 expert/component placement 是否降低换入和远程访问成本；第四步再加入 PD/KV 扩展模板，观察阶段解耦是否真的弥补 prefill/decode 干扰。消融时应避免过多组合导致结果难以解释，建议每次只改变一类决策变量：先固定 routing 看模型切分收益，再固定模型切分看 batching 收益，最后叠加形成综合方案。这样写出来的实验结论会更像系统论文，而不是参数扫描。

9.5 预期收益分析

预期收益应围绕三项算法交付分别验证，而不是按研发远近粗略拆分。模型切分部署算法的收益主要体现在异构资源利用和可支持并发规格上：通过以 layer-wise stage boundary 和 stage placement 为主轴的拓扑感知切分，先避免平均层级切分在异构算力、半算力卡、跨主机 RDMA 或 310P-910B4 联合组网中的 stage 不均衡和通信浪费；再按需叠加 TP/EP/DP 等补充能力，缓解单个 stage 的显存压力、MoE expert 驻留压力和流量承载压力。该算法的收益应主要通过相对多节点平均切分基线的满足 SLA 请求处理数、SLA-constrained concurrency、pipeline bubble ratio 和回退次数来体现，并支撑满足 SLA 请求处理能力提升 50%、支持并发用户数达到 1.5X 基线的规格目标。

业务感知 batching 算法的收益主要体现在 SLA goodput 和平均 E2E 时延上。通过 SLO-aware admission、请求重排、动态等待窗口、batch/token budget 控制、chunked prefill 和 KV 水位保护，系统可以在随机并发、不同输入输出长度和 HTTP header 特殊时延要求下减少高优先级请求违约，降低长 prompt 对流式 decode 的干扰。验收时应相对于“静态 batching 配置、不调整请求处理顺序”基线，验证满足 SLA 的请求处理速度或 rps 提升 20%，所有请求平均 E2E latency 降低 20%。

移动请求调度算法的收益主要体现在多实例协同和服务连续性上。它不一定显著提高单实例峰值吞吐，但可以在多流量入口、不同链路距离和用户入口切换条件下减少不必要的就近拥塞，降低移动用户服务中断和重复 prefill，提升 prefix cache 或 session state 的利用率，并把请求分散到更合适的边缘实例。验收时应相对于“就近实例路由”基线，验证满足 SLA 的用户数达到 1.5X，所有请求

平均 E2E latency 降低 20%。对于无线业务，这类收益比单点吞吐更贴近用户感知，因为真实体验受路径稳定性、响应连续性和尾部时延共同影响。

十、挑战与未来方向

10.1 异构资源与性能建模

CEC LLM 推理中的性能建模比云端 serving 更难，因为影响时延的因素不再只是模型大小和 GPU 利用率。端侧 NPU、边缘 GPU、CPU 内存、显存容量、无线链路、边缘间链路和云端队列都会进入同一条服务路径。即使模型和请求相同，只要入口位置、batch 组成或 KV 状态位置不同，TTFT 和 TPOT 都可能出现明显差异。HexGen、HexGen-2 和端-边-云协同智能综述均表明，异构资源建模是跨设备、跨节点推理优化的基础难点 [12], [87], [9]。

因此，未来更可行的方向不是建立一个全局精确模型，而是构建可在线校准的分层性能画像。模型画像记录不同 prompt/output 长度下的 prefill、decode 和 KV 增长特征；硬件画像记录不同 batch、并行度和量化配置下的吞吐与显存水位；链路画像记录端-边、边-边、边-云的 RTT、带宽和抖动；状态画像记录 prefix cache、KV Cache 和 session state 的位置与复用价值。调度器再基于这些画像做近似决策，并用线上观测持续修正误差。这个方向的关键不是预测得多精细，而是预测误差不能把系统带入错误的路由或切分策略。

10.2 状态迁移与一致性

KV Cache、prefix cache 和 session state 让 LLM 推理天然带有强状态性。传统边缘 offloading 往往把任务看成可独立迁移的计算单元，但 LLM 会话不是这样：一次 prefill 生成的 KV 可能在后续数百个 decode step 中持续使用，多轮对话还会不断累积上下文。用户移动后，如果只把新请求路由到最近节点，却忽略旧状态的位置，就可能出现重复 prefill、远程访问延迟升高或上下文连续性破坏。PagedAttention、CacheGen、Mooncake、Preble 和 CachedAttention 分别从 KV 分页、KV 压缩/流式传输、KVCache-centric architecture、prompt scheduling 和多轮会话缓存复用角度说明了该问题的重要性 [2], [13], [46], [75], [78]。

未来需要研究的是状态生命周期管理，而不只是状态迁移。系统应能判断哪些状态值得保留，哪些状态可以压缩、降精度、迁移、复制或淘汰；还要判断在 handover 前是否预热候选边缘节点，在 handover 后是否临时远程访问旧状态，何时再把状态真正迁移过去。更进一步，状态一致性也不能只用数据库式强一致性来理解。LLM serving 更需要的是服务连续性一致性：用户看到的上下文、权限、个性化信息和生成流不能中断，但底层 KV 是否完全同步可以根据成本和剩余生成长度做权衡。

10.3 在线策略稳定性

动态 batching、动态路由和动态切分都可能带来策略震荡。一个典型例子是：调度器看到某个边缘实例负载低，于是把更多请求转过去；转发后该实例排队升高，下一轮调度又把请求切回原实例；与此同时 prefix cache 命中下降，KV 状态在多个实例之间来回移动，最终平均负载看似均衡，尾延迟却变差。类似问题在无线移动场景中会更明显，因为链路抖动和入口变化本身就是高频扰动。动态调度和迁移相关系统通常会显式讨论 SLO、尾延迟、迁移开销或公平性边界，这也是 CEC 推理调度需要继承的工程原则 [4], [49], [47]。

因此，在线策略需要从“每次都选当前最优”转向“在收益足够大时才切换”。可研究的机制包括切换冷却时间、收益阈值、置信区间、状态迁移预算、SLA 违约保护和灰度发布。对于生产系统，还应保留影子调度能力：新策略先在旁路计算推荐决策，与线上实际决策比较一段时间，再逐步接管小比例流量。这样的机制看起来不如端到端强化学习漂亮，但在无线软件环境中更容易解释、回滚和验收。

10.4 隐私与合规

无线场景中的用户位置、业务上下文、终端标识和会话历史都可能具有隐私敏感性。CEC 推理如果只按时延和负载选择路径，可能把敏感上下文转发到不合适的节点或区域。尤其是长上下文会话中，KV Cache 与 prefix cache 虽然不是原始文本，但仍可能包含与用户输入相关的隐含信息，因此状态放置和状态迁移也应纳入隐私约束。

未来更合理的做法是把隐私边界作为调度约束，而不是事后补丁。请求进入系统时就应带有数据等级、可出域范围、是否允许云端处理、是否允许跨边缘复制状态等标签。路由、batching 和模型切分都必须在这些约束下优化：隐私敏感请求可以优先端侧或接入边缘处理，复杂但敏感的请求可以采用本地小模型 + 云端非敏感摘要的协同方式，状态迁移也可以只迁移必要的 session metadata，而不迁移完整 KV。这个方向的难点在于同时维持性能、质量和合规，而不是简单地把敏感请求全部留在本地。

10.5 未来研究方向

后续研究可以沿着五条线推进。第一是云边端一体化 LLM serving，把模型模板、batching 策略、状态索引和路由策略纳入统一控制面，而不是分别优化。第二是面向无线移动性的 KV/state 管理，重点研究状态何时保留、何时迁移、何时远程访问、何时重算。第三是多模型多租户调度，在 SLM、LLM、LoRA adapter、MoE expert 和云端大模型共存时，同时保证资源隔离、公平性和 SLA。第四是隐私保护协同推理，把数据本地性、状态脱敏、token-wise partition 和模型路径选择结合起来。第五

是面向生产系统的 MLOps 与策略治理，包括监控、灰度、回滚、异常检测和策略审计。其中，云边缘协同和移动服务连续性可由端-边-云协同智能综述与 mobility-aware service placement 工作支撑 [9], [86]; KV/cache runtime 可由 Mooncake、CacheGen、Preble 和 CachedAttention 等工作支撑 [13], [46], [75], [78]; 多用户公平调度可由 Fairness in Serving LLMs 支撑 [47]。

从项目落地角度看，最值得优先投入的是“预测-决策-回退”一体化框架。预测模块不追求完美，只要能够估计请求长度、实例服务时间、链路状态和移动风险；决策模块基于有限模板做 batching、routing 和部署选择；回退模块在预测失准、SLA 风险升高或状态迁移失败时快速切回保守策略。这样的框架既能承接前文三类算法，也能符合无线系统对稳定性和可解释性的要求。

十一、总结

本文围绕 CEC 场景下的大语言模型推理优化进行了系统综述。与传统数据中心 LLM serving 相比，CEC 推理需要额外处理端-边-云异构资源、无线链路波动、边缘节点负载变化、用户移动、KV/状态连续性、隐私边界和业务 QoS。现有研究已经分别在异构推理、LLM serving runtime、KV/cache 管理和移动边缘服务连续性方面提供了可借鉴基础，但这些机制需要被重新组织到一个可观测、可回退、可验证的协同推理框架中 [1], [2], [3], [4], [12], [9], [27], [13], [46], [47], [75], [78], [86]。

从算法主线看，资源自适应模型切分部署解决“模型如何跨异构节点展开”，业务感知 batching 解决“请求和 token 如何在实例内组织执行”，LLM inference task offloading 解决“请求、workflow 和运行时状态如何在多层网络中移动”。三者不是彼此独立的模块：模型切分会改变阶段容量和状态传输代价，batching 会改变实例队列和 SLO 风险，请求路由又会改变负载分布、prefix/KV 命中和后续状态位置。因此，CEC LLM 推理的系统价值来自分层闭环，而不是某一个单点算法。

从工程实现看，更稳妥的路线是在 vLLM-Ascend 等现有 Ascend 推理栈上扩展资源自适应控制能力。底层 runtime 继续负责模型加载、算子执行、token 生成、KV 管理和实例内调度；外层控制面补齐多节点资源感知、layer-level profiling、模型分层部署规划、动态 batching 参数控制、状态索引、监控反馈和异常回退。EdgeShard 提供了 layer-level profiling、设备选择、model shard placement 和动态规划搜索的基础，但在 Ascend 环境中需要转化为 vLLM-Ascend 可执行的模板化部署方案。

面向验证环境，报告建议将主问题落到三个可测闭环上：第一，以 layer-wise stage boundary 和 stage placement 为主变量，叠加 TP/EP/DP 等补充能力，验证拓扑感知混合切分是否优于平均层级切分和固定并行模板；第二，以 SLO slack、请求优先级、队列状态和实例画像为输入，验证动态 batching 控制是否优于固定 batch size 与 FIFO；第三，在用户人口变化和 KV 状态分离场景下，验证长期成本感知 routing 是否能够减少重复 prefill、远程状态访问和跨节点通信开销。只有把算法收益落实到 TTFT、TPOT/ITL、P95/P99 E2E、SLA 达标率、goodput、显存水位、KV/state 迁移成本和回退次数等指标上，才能判断其是否具备工程部署价值。

总体而言，CEC LLM inference 的核心并不是把云端 serving 简单搬到边缘，而是在异构资源和动态网络中重构模型部署、请求调度和状态管理之间的关系。资源画像、有限候选模板、在线代价选择、动态 batching 控制、状态感知 routing 和可解释回退机制，是把综述中的研究方法转化为可运行系统的关键路径。

参考文献

- [1] Y. Sheng et al., "FlexGen: High-throughput generative inference of large language models with a single GPU," in Proc. Int. Conf. Machine Learning (ICML), 2023.
- [2] A. Borzunov et al., "Petals: Collaborative inference and fine-tuning of large models," in Proc. 61st Annu. Meeting Assoc. Comput. Linguistics (ACL), vol. 3, 2023, pp. 558-568.
- [3] Y. Jiang et al., "HexGen: Generative inference of large language model over heterogeneous environment," in Proc. Int. Conf. Machine Learning (ICML), 2024, pp. 21946-21961.
- [4] G.-I. Yu et al., "Orca: A distributed serving system for Transformer-based generative models," in Proc. USENIX Symp. Operating Systems Design and Implementation (OSDI), 2022, pp. 521-538.
- [5] W. Kwon et al., "Efficient memory management for large language model serving with PagedAttention," in Proc. ACM Symp. Operating Systems Principles (SOSP), 2023, pp. 611-626.
- [6] A. Agrawal et al., "Taming throughput-latency tradeoff in LLM inference with Sarathi-Serve," in Proc. USENIX Symp. Operating Systems Design and Implementation (OSDI), 2024, pp. 117-134.
- [7] Y. Zhong et al., "DistServe: Disaggregating prefill and decoding for goodput-optimized large language model serving," in Proc. USENIX Symp. Operating Systems Design and Implementation (OSDI), 2024.
- [8] L. Zheng et al., "SGLang: Efficient execution of structured language model programs," in Proc. Adv. Neural Inf. Process. Syst. (NeurIPS), 2024.
- [9] NVIDIA, "TensorRT-LLM documentation," 2026. [Online]. Available: <https://nvidia.github.io/TensorRT-LLM/>
- [10] NVIDIA, "Triton Inference Server: Dynamic batcher," 2026. [Online]. Available: <https://docs.nvidia.com/deeplearning/triton-inference-server/>
- [11] Hugging Face, "Text Generation Inference documentation," 2026. [Online]. Available: <https://huggingface.co/docs/text-generation-inference/>
- [12] Y. Jiang, R. Yan, and B. Yuan, "HexGen-2: Disaggregated generative inference of LLMs in heterogeneous environment," in Proc. Int. Conf. Learn. Representations (ICLR), 2025.
- [13] S. Duan, D. Wang, J. Ren, F. Lyu, Y. Zhang, H. Wu, and X. Shen, "Distributed artificial intelligence empowered by end-edge-cloud computing: A survey," IEEE Communications Surveys & Tutorials, vol. 25, no. 1, pp. 591-624, 2023.
- [14] Y. Huang et al., "GPipe: Efficient training of giant neural networks using pipeline parallelism," in Proc. Adv. Neural Inf. Process. Syst. (NeurIPS), 2019, pp. 103-112.

- [15] D. Narayanan et al., "Efficient large-scale language model training on GPU clusters using Megatron-LM," in Proc. Int. Conf. High Performance Computing, Networking, Storage and Analysis (SC), 2021.
- [16] L. Zheng et al., "Alpa: Automating inter- and intra-operator parallelism for distributed deep learning," in Proc. USENIX Symp. Operating Systems Design and Implementation (OSDI), 2022, pp. 559-578.
- [17] R. Y. Aminabadi et al., "DeepSpeed-Inference: Enabling efficient inference of transformer models at unprecedented scale," in Proc. Int. Conf. High Performance Computing, Networking, Storage and Analysis (SC), 2022.
- [18] P. Patel et al., "Splitwise: Efficient generative LLM inference using phase splitting," in Proc. ACM/IEEE Int. Symp. Computer Architecture (ISCA), 2024, pp. 118-132.
- [19] B. Sun et al., "Llumnix: Dynamic scheduling for large language model serving," in Proc. USENIX Symp. Operating Systems Design and Implementation (OSDI), 2024, pp. 173-191.
- [20] R. Prabhu, A. Nayak, J. Mohan, R. Ramjee, and A. Panwar, "vAttention: Dynamic memory management for serving LLMs without PagedAttention," in Proc. ACM Int. Conf. Architectural Support for Programming Languages and Operating Systems (ASPLOS), 2025, pp. 1133-1150.
- [21] R. Qin et al., "Mooncake: Trading more storage for less computation – A KVCache-centric architecture for serving LLM chatbot," in Proc. USENIX Conf. File and Storage Technologies (FAST), 2025, pp. 155-170.
- [22] Y. Liu et al., "CacheGen: KV cache compression and streaming for fast large language model serving," in Proc. ACM SIGCOMM, 2024, pp. 38-56.
- [23] V. Srivatsa et al., "Preble: Efficient distributed prompt scheduling for LLM serving," in Proc. Int. Conf. Learn. Representations (ICLR), 2025.
- [24] B. Gao et al., "Cost-efficient large language model serving for multi-turn conversations with Cache-dAttention," in Proc. USENIX Annu. Tech. Conf. (ATC), 2024, pp. 111-126.
- [25] T. Ouyang, Z. Zhou, and X. Chen, "Follow Me at the Edge: Mobility-aware dynamic service placement for mobile edge computing," IEEE J. Sel. Areas Commun., vol. 36, no. 10, pp. 2333-2345, Oct. 2018.
- [26] Y. Sheng et al., "Fairness in serving large language models," in Proc. USENIX Symp. Operating Systems Design and Implementation (OSDI), 2024, pp. 965-988.
- [27] S. Ye, B. Ouyang, L. Zeng, T. Qian, X. Chu, J. Tang, and X. Chen, "Jupiter: Fast and Resource-Efficient Collaborative Inference of Generative LLMs on Edge Devices," in Proc. IEEE INFOCOM, 2025.
- [28] B. Zhang, J. Zhang, J. Hou, and Y. Wang, "TensAllo: Adaptive Deployment of LLMs on Resource-Constrained Heterogeneous Edge Devices," in Proc. IEEE INFOCOM, 2025.

- [29] M. Zhang, J. Cao, X. Shen, and Z. Cui, "EdgeShard: Efficient LLM Inference via Collaborative Edge Computing," *IEEE Internet of Things Journal*, 2025.
- [30] S. Ye et al., "Resource-Efficient Collaborative Edge Transformer Inference With Hybrid Model Parallelism," *IEEE Transactions on Mobile Computing*, vol. 24, no. 10, pp. 10945-10962, 2025.
- [31] J. Du, Y. Wei, S. Ye, J. Jiang, X. Chen, D. Huang, and Y. Lu, "Co-Designing Transformer Architectures for Distributed Inference With Low Communication," *IEEE Transactions on Parallel and Distributed Systems*, vol. 36, no. 4, 2025.
- [32] X. Feng et al., "E2LLM: Structure-Guided Efficient Inference for LLMs in Distributed Edge-IoT Environments," *IEEE Transactions on Mobile Computing*, 2026.
- [33] Y. Lin, S. Peng, C. Lu, C. Xu, and K. Ye, "FlexPipe: Adapting Dynamic LLM Serving Through Inflight Pipeline Refactoring in Fragmented Serverless Clusters," in *Proc. EuroSys*, 2026.
- [34] G. Qu, T. Li, Q. Chen, X. Chen, and S. Zhou, "SLIDE: Simultaneous Model Downloading and Inference at the Wireless Network Edge," *IEEE Transactions on Mobile Computing*, 2026.
- [35] D. Xu et al., "EdgeLLM: Fast On-Device LLM Inference With Speculative Decoding," *IEEE Transactions on Mobile Computing*, vol. 24, no. 4, pp. 3256-3271, 2025.
- [36] F. Chen, P. Li, T. H. Luan, Z. Su, and J. Deng, "SPIN: Accelerating Large Language Model Inference with Heterogeneous Speculative Models," in *Proc. IEEE INFOCOM*, 2025.
- [37] X. Miao et al., "SpecInfer: Accelerating Large Language Model Serving with Tree-based Speculative Inference and Verification," in *Proc. ACM Int. Conf. Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2024.
- [38] R. Kong et al., "SwapMoE: Serving Off-the-shelf MoE-based Large Language Models with Tunable Memory Budget," in *Proc. Annu. Meeting Assoc. Comput. Linguistics (ACL)*, 2024.
- [39] H. Yu, X. Cui, H. Zhang, H. Wang, and H. Wang, "Taming Latency-Memory Trade-Off in MoE-Based LLM Serving via Fine-Grained Expert Offloading," in *Proc. EuroSys*, 2026.
- [40] L. Xue, Y. Fu, Z. Lu, C. Sun, L. Mai, and M. Marina, "MoE-Infinity: Efficient MoE Inference on Personal Machines with Sparsity-Aware Expert Cache," in *Proc. Int. Conf. Machine Learning (ICML)*, 2025.
- [41] R. Hwang et al., "Pre-gated MoE: An Algorithm-System Co-Design for Fast and Scalable MoE Inference," in *Proc. ACM/IEEE Int. Symp. Computer Architecture (ISCA)*, 2024.
- [42] R. Yi, L. Guo, S. Wei, A. Zhou, S. Wang, and M. Xu, "EdgeMoE: Empowering Sparse Large Language Models on Mobile Devices," *IEEE Transactions on Mobile Computing*, 2025.
- [43] N. Xue et al., "WDMoE: Wireless Distributed Mixture of Experts for Large Language Models,"

IEEE Transactions on Wireless Communications, 2025.

[44] R. Lu, Y. Liu, and D. Wang, "P4LLM: Protecting Embedding Privacy against Model Inversion Attacks for EaaS LLM Serving via Token-wise Partition and Perturbation," IEEE Transactions on Mobile Computing, 2026.

[45] W. Zhang, Z. Wu, Y. Mu, R. Ning, B. Liu, N. Sarda, M. Lee, and F. Lai, "JITServe: SLO-aware LLM Serving with Imprecise Request Information," in Proc. USENIX Symp. Networked Systems Design and Implementation (NSDI), 2026.

[46] K. Hong et al., "SOLA: Optimizing SLO Attainment for Large Language Model Serving with State-Aware Scheduling," in Proc. Conf. Machine Learning and Systems (MLSys), 2025.

[47] A. Patke et al., "One Queue Is All You Need: Resolving Head-of-Line Blocking in Large Language Model Serving," in Proc. Workshop on Cloud Intelligence/AIOps (AIOps@ASPLOS), 2024.

[48] N. Ling, G. Chen, and L. Zhong, "TimelyLLM: Segmented LLM Serving System for Time-sensitive Robotic Applications," in Proc. ACM MobiSys, 2026.

[49] B. Wu et al., "FastServe: Iteration-Level Preemptive Scheduling for Large Language Model Inference," in Proc. USENIX Symp. Networked Systems Design and Implementation (NSDI), 2026.

[50] J. Chen et al., "TokenFlow: Responsive LLM Text Streaming Serving under Request Burst via Preemptive Scheduling," in Proc. EuroSys, 2026.

[51] K. Zhu et al., "NanoFlow: Towards Optimal Large Language Model Serving Throughput," in Proc. USENIX Symp. Operating Systems Design and Implementation (OSDI), 2025.

[52] C. Ruan et al., "Libra: Flexible Request Partitioning and Scheduling for Serving Unbalanced and Dynamic LLM Workloads," in Proc. USENIX Symp. Networked Systems Design and Implementation (NSDI), 2026.

[53] Y. Shen, Y. Zhang, and D. Yuan, "A Hybrid Online and Offline Requests Inference Serving System for LLM in Private Computer Environment," IEEE Transactions on Computers, vol. 75, no. 3, 2026.

[54] Z. Li et al., "AdaServe: Accelerating Multi-SLO LLM Serving with SLO-Customized Speculative Decoding," in Proc. EuroSys, 2026.

[55] J. Yang, Q. Wu, Z. Feng, Z. Zhou, D. Guo, and X. Chen, "Quality-of-Service Aware LLM Routing for Edge Computing with Multiple Experts," IEEE Transactions on Mobile Computing, 2025.

[56] Y. Li, J. Guo, Z. Tang, X. Ding, J. Wang, T. Wang, and W. Jia, "Cloud-Edge System for Scheduling Unpredictable LLM Requests With Combinatorial Bandit," IEEE Transactions on Services Computing, vol. 18, no. 6, pp. 3567-3580, 2025.

[57] I. Ong et al., "RouteLLM: Learning to Route LLMs from Preference Data," in Proc. Int. Conf.

Learn. Representations (ICLR), 2025.

[58] F. Wu and S. Silwal, "PORT: Efficient Training-Free Online Routing for High-Volume Multi-LLM Serving," in Proc. Adv. Neural Inf. Process. Syst. (NeurIPS), 2025.

[59] Y. He, J. Fang, F. R. Yu, and V. C. Leung, "Large Language Models (LLMs) Inference Offloading and Resource Allocation in Cloud-Edge Computing: An Active Inference Approach," IEEE Transactions on Mobile Computing, vol. 23, no. 12, pp. 11253-11264, 2024.

[60] X. Xu, G. Feng, Y. Liu, S. Qin, J. Wang, and Y. Wang, "Joint Inference Offloading and Model Caching for Small and Large Language Model Collaboration," IEEE Transactions on Mobile Computing, vol. 25, no. 2, 2026.

[61] A. Ghosh, N. Sharma, S. Mishra, R. Misra, and S. K. Das, "MGCO: Mobility-Aware Generative Computation Offloading in Edge-Cloud Systems," IEEE Transactions on Services Computing, vol. 19, no. 1, pp. 477-490, 2026.

[62] Y. Yue et al., "MasRouter: Learning to Route LLMs for Multi-Agent Systems," in Proc. Annu. Meeting Assoc. Comput. Linguistics (ACL), 2025.

[63] S. Nielsen, E. Cetin, P. Schwendeman, Q. Sun, J. Xu, and Y. Tang, "Learning to Orchestrate Agents in Natural Language with the Conductor," in Proc. Int. Conf. Learn. Representations (ICLR), 2026.

[64] J. Zhang et al., "AFlow: Automating Agentic Workflow Generation," in Proc. Int. Conf. Learn. Representations (ICLR), 2025.

[65] X. Chen, J. Cao, Y. Sahni, M. Zhang, Z. Liang, and L. Yang, "Mobility-Aware Dependent Task Offloading in Edge Computing: A Digital Twin-Assisted Reinforcement Learning Approach," IEEE Transactions on Mobile Computing, vol. 24, no. 4, pp. 2979-2994, 2025.

[66] J. Jiang et al., "Efficient KV Cache Spillover Management on Memory-Constrained GPU for LLM Inference," IEEE Transactions on Parallel and Distributed Systems, vol. 37, no. 1, 2026.

[67] S. Gao, Q. Wang, S. Zeng, Y. Lu, and J. Shu, "Weaver: Efficient Multi-LLM Serving with Attention Offloading," in Proc. USENIX Annu. Tech. Conf. (ATC), 2025, pp. 587-595.

[68] M. Xu, D. Niyato, and C. G. Brinton, "Serving Long-Context LLMs at the Mobile Edge: Test-Time Reinforcement Learning-Based Model Caching and Inference Offloading," IEEE/ACM Transactions on Networking, vol. 34, pp. 3808-3823, 2026.

[69] J. Wang et al., "Context-Aware AIGC Service Migration in Edge Intelligence Networks via Transformer DRL," IEEE Transactions on Services Computing, vol. 19, no. 2, pp. 1020-1033, 2026.

[70] LMDeploy Team, "LMDeploy documentation," 2026. [Online]. Available: <https://lmdeploy.readthedocs.io/>

- [71] A. Du, J. Jia, S. Dustdar, J. Chen, and X. Wang, "Online Service Placement, Task Scheduling, and Resource Allocation in Hierarchical Collaborative MEC Systems," *IEEE Transactions on Services Computing*, vol. 18, no. 2, pp. 983-997, 2025.
- [72] X. Zhuang, J. Wu, H. Wu, T. Zhang, and L. Gao, "Joint Optimization of Model Inferencing and Task Offloading for MEC-Empowered Large Vision Model Services," in *Proc. IEEE INFOCOM*, 2025.
- [73] Z. Zhou et al., "A Survey on Efficient Inference for Large Language Models," *arXiv:2404.14294*, 2024.
- [74] R. Zhen et al., "Taming the Titans: A Survey of Efficient LLM Inference Serving," in *Proc. Int. Natural Language Generation Conf. (INLG)*, 2025, pp. 522-541.
- [75] J. Pan and G. Li, "A Survey of LLM Inference Systems," *arXiv:2506.21901*, 2025.
- [76] 华为, 《边缘 AI 计算技术合作项目资料》, 2026.
- [77] vLLM Ascend Team, "vLLM Ascend documentation," 2026. [Online]. Available: <https://docs.vllm.ai/projects/ascend/en/latest/>
- [78] vLLM Ascend Team, "vLLM Ascend installation guide," 2026. [Online]. Available: <https://docs.vllm.ai/projects/ascend/en/latest/installation.html>
- [79] vLLM Ascend Team, "Supported features," 2026. [Online]. Available: https://docs.vllm.ai/projects/ascend/en/latest/user_guide/feature_guide
- [80] vLLM Ascend Team, "KV Cache Pool," 2026. [Online]. Available: https://docs.vllm.ai/projects/ascend/en/latest/user_guide/feature_guide/kv_pool.html
- [81] vLLM Ascend Team, "Speculative decoding," 2026. [Online]. Available: https://docs.vllm.ai/projects/ascend/en/latest/user_guide/feature_guide/speculative_decoding.html
- [82] Huawei Ascend, "MindIE," 2026. [Online]. Available: <https://www.hiascend.com/en/developer/software/mindie>
- [83] vLLM Project, "OpenAI-compatible server," 2026. [Online]. Available: https://docs.vllm.ai/en/latest/serving/openai_compatible_server.html
- [84] L. Chen, Z. Ye, Y. Wu, D. Zhuo, L. Ceze, and A. Krishnamurthy, "Punica: Multi-Tenant LoRA Serving," in *Proc. Conf. Machine Learning and Systems (MLSys)*, 2024.
- [85] Y. Sheng et al., "SLoRA: Scalable Serving of Thousands of LoRA Adapters," in *Proc. Conf. Machine Learning and Systems (MLSys)*, 2024.
- [86] J. Duan, R. Lu, H. Duanmu, X. Li, X. Zhang, D. Lin, I. Stoica, and H. Zhang, "MuxServe: Flexible Spatial-Temporal Multiplexing for Multiple LLM Serving," in *Proc. Int. Conf. Machine Learning (ICML)*, 2024, pp. 11905-11917.

- [87] Y. He, H. Yang, Y. Lu, A. Klimovic, and G. Alonso, "Resource Multiplexing in Tuning and Serving Large Language Models," in Proc. USENIX Annu. Tech. Conf. (ATC), 2025, pp. 1639-1655.
- [88] G. Oliaro, X. Miao, X. Cheng, V. Kada, M. Wu, R. Gao, Y. Huang, R. Delacourt, A. Yang, Y. Wang, C. Unger, and Z. Jia, "FlexLLM: Token-Level Co-Serving of LLM Inference and Finetuning with SLO Guarantees," in Proc. USENIX Symp. Networked Systems Design and Implementation (NSDI), 2026, pp. 1359-1379.